"[Mission statement of the institute]"

SELF-MAINTAINING API INTEGRATIONS FOR THIRD-PARTY BREAKING CHANGES

For the Degree of

Bachelor of Technology

in *Computer Science and Engineering*

By

[STUDENT NAME 1] ([ROLL NO.])
[STUDENT NAME 2] ([ROLL NO.])
[STUDENT NAME 3] ([ROLL NO.])

Under the Guidance of

[PROF./DR. GUIDE NAME]



Department of Computer Science and Engineering

[NAME OF THE INSTITUTE]

[CITY], [STATE], India

2026

CERTIFICATE

This is to certify that the project report entitled **"Self-Maintaining API Integrations for Third-Party Breaking Changes"**, which is being submitted herewith for the award of the **'Degree of Bachelor of Technology'** in **'Computer Science and Engineering'** of [NAME OF THE UNIVERSITY], is the result of the original work and contribution by

[STUDENT NAME 1] ([ROLL NO.])
[STUDENT NAME 2] ([ROLL NO.])
[STUDENT NAME 3] ([ROLL NO.])

under my supervision and guidance during the academic year 2026-27. The work embodied in this report has not formed earlier the basis for the award of any degree or compatible certificate or similar title of this for any other diploma/examining body or university to the best of my knowledge and belief.

(Signature)	(Signature)	(Signature)
[PROF./DR. GUIDE NAME]	**[PROF./DR. HOD NAME]**	**[DR. PRINCIPAL NAME]**
Guide	Head of the Department	Principal

Place: [CITY]

Date:

DECLARATION

We hereby declare that we have formed, completed and written the project report entitled “SELF-MAINTAINING API INTEGRATIONS FOR THIRD-PARTY BREAKING CHANGES”. It has not previously been submitted for the basis of the award of any degree or diploma or other similar title of this for any other diploma/examining body or university.

Place: [CITY]

[STUDENT NAME 1]

[STUDENT NAME 2]

[STUDENT NAME 3]

Date:

ABSTRACT

Modern software depends on third-party web APIs such as payment, messaging and AI services. These APIs change without the consumer's consent: endpoints are removed, parameters become required, response fields disappear. A large empirical study found that 24.6% of APIs removed operations without any prior deprecation, while only 0.6% deprecated them first. The consumer usually learns about such a change when production breaks.

This project designs, implements and evaluates **sma** (Self-Maintaining API integrations), a command-line tool that closes the loop on the consumer side. `sma` watches the OpenAPI specifications that a JavaScript or TypeScript repository depends on. On every run it fetches each specification, inlines its local references, computes a structural difference against a committed baseline, and classifies every change with a deterministic table of 17 rules (11 breaking, 1 warning, 5 safe). Changes no rule explains go to a large language model, whose verdicts below a 0.7 confidence floor are reported but never acted upon. An abstract syntax tree scanner (ts-morph) finds every HTTP call site of the watched API in the consumer's code, plus one hop of callers, and an impact join keeps only the breaking changes that this code actually uses. Those changes are handed to a sandboxed coding agent that may read and edit files but cannot run commands. The tool then verifies the agent's work itself, by running the project's test suite and checking that every changed file was predicted by the impact analysis, and opens a pull request labeled `needs-human` when either check fails. Nothing is merged automatically.

The system was evaluated with three methods. Experimentally, all 51 unit tests pass; on a controlled petstore API upgrade the tool found 6 changes (5 breaking), mapped them to 8 call sites, and produced a patch that turned a 0/4 test result back into 4/4; on a replay of OpenAI's real June 2023 specification change it detected all 5 removed endpoints and correctly reported that only 2 of them affected the sample application. Statistically, 6,700 single-change mutation trials on three specifications were classified with 100% accuracy against 83.0% for a naive structural-diff classifier, and in releases carrying up to 8 simultaneous changes every broken operation was still flagged. The AST scanner reached a precision of 1.00 on a labeled 467-statement corpus against 0.80 and 0.92 for two grep baselines, while missing two-hop callers by design, and a 478-commit longitudinal study of OpenAI's specification exposed one real defect in the rule engine (OpenAPI 3.1 type arrays reported as type changes) and two semantic gaps (schema composition and direction-blind type rules), for which fixes are proposed. Analytically, the detection stages and the AST scan were shown to scale linearly, as the complexity model predicted.

Keywords: API evolution, breaking change detection, OpenAPI, change impact analysis, static analysis, large language models, automated program repair, software maintenance.

INDEX

List of Abbreviations and Symbols . . . . . **i**

List of Figures . . . . . **iv**

List of Graphs . . . . . **v**

List of Tables . . . . . **vi**

List of Listings . . . . . **viii**

1 Introduction **1**

1.1 Introduction 1

1.2 Need, Necessity and Motivation 3

1.2.1 Providers break consumers, often without warning 3

1.2.2 Breaking changes are real and costly in practice 3

1.2.3 Existing tools serve the wrong side of the interface 4

1.2.4 Static evidence is precise; manual work is not 4

1.2.5 Language models can patch code, but cannot be trusted alone 4

1.3 Objectives 5

1.4 Theme 6

1.5 Organization 7

2 Literature Survey **9**

2.1 Introduction and Survey Method 9

2.2 Background: Web APIs, REST and OpenAPI 9

2.2.1 REST and HTTP semantics 9

2.2.2 The OpenAPI Specification 10

2.2.3 JSON Pointer 11

2.2.4 Versioning conventions 11

2.2.5 Software evolution and maintenance 11

2.3 Web API Evolution and Breaking Changes 11

2.3.1 Early qualitative studies 11

2.3.2 Large-scale studies of OpenAPI histories 12

2.3.3 Breaking changes in library ecosystems 12

2.3.4 Consumer-side detection 13

2.4 Library Migration and Dependency Management . . . 14
2.4.1 Refactoring-based migration . . . 14
2.4.2 Learning adaptations from examples . . . 14
2.4.3 Dependency update bots . . . 14
2.4.4 Benchmarks of breaking dependency updates . . . 15
2.5 Specification Diff Tools and Contract Testing . . . 15
2.5.1 Provider-side diff tools . . . 15
2.5.2 Consumer-driven contract testing . . . 15
2.5.3 Specification sources . . . 16
2.6 Static Analysis and Change Impact Analysis . . . 16
2.6.1 Change impact analysis . . . 16
2.6.2 Abstract syntax trees and the TypeScript compiler . . . 16
2.6.3 Call graphs for JavaScript . . . 16
2.7 Automated Program Repair and Large Language Models for Code . . . 17
2.7.1 Classical automated program repair . . . 17
2.7.2 Large language models for code . . . 17
2.7.3 Repository-level coding agents . . . 17
2.8 LLM-Based Library and API Migration . . . 18
2.9 Trust, Verification and Human Oversight . . . 19
2.10 Comparative Analysis of Existing Approaches . . . 19
2.11 Research Gap . . . 19
2.12 Summary of Reviewed Literature . . . 21
2.13 Summary . . . 22

3 System Development . . . 24
3.1 Development Methodology and Requirements . . . 24
3.1.1 Methodology . . . 24
3.1.2 Functional requirements . . . 24
3.1.3 Non-functional requirements . . . 25
3.2 Technology Stack and Design Decisions . . . 26
3.3 System Architecture . . . 26
3.3.1 Module structure . . . 26
3.3.2 Pipeline and data flow . . . 27
3.3.3 Interaction sequence . . . 27
3.4 Mathematical Model of the System . . . 29
3.4.1 Specifications as trees . . . 29
3.4.2 Reference resolution . . . 29
3.4.3 Structural difference . . . 30
3.4.4 Normalized operation model . . . 31

3.4.5 Rules as predicates . . . 31
3.4.6 Classification and the confidence gate . . . 31
3.4.7 Call sites and path shapes . . . 32
3.4.8 The impact join . . . 33
3.4.9 Verification predicate and exit codes . . . 33
3.4.10 Analytical complexity . . . 34
3.5 Watcher: Fetching and Normalizing Specifications . . . 35
3.6 Structural Diff and Rule Engine . . . 36
3.7 Classifier and LLM Fallback . . . 37
3.8 Inventory: AST Call-Site Scanner . . . 39
3.9 Impact Join and Repair Grouping . . . 39
3.10 Repair Agent and Independent Verification . . . 40
3.11 Pull Request Writer . . . 41
3.12 State Store and Data Formats . . . 41
3.13 Command-Line Interface, Report and Degradation . . . 42
3.13.1 Commands and exit codes . . . 42
3.13.2 Graceful degradation . . . 43
3.13.3 HTML report . . . 43
3.14 Demonstration Harness . . . 45
3.14.1 Controlled upstream: the petstore API . . . 45
3.14.2 Real upstream: the OpenAI replay . . . 46
3.15 Automation and Continuous Integration . . . 46
3.16 Security Considerations . . . 46
3.17 Hardening Found by End-to-End Use . . . 47
3.18 Testing Strategy . . . 48
3.19 Summary . . . 48

4 Performance Analysis . . . 49
4.1 Evaluation Design . . . 49
4.1.1 Research questions . . . 49
4.1.2 Methods and the questions they answer . . . 49
4.1.3 Experimental environment . . . 50
4.2 Experimental Method . . . 51
4.2.1 Unit test results . . . 51
4.2.2 Controlled end-to-end upgrade: the petstore API . . . 51
4.2.3 Real-world replay: OpenAI, June 2023 . . . 54
4.3 Statistical Method . . . 56
4.3.1 Mutation benchmark: design . . . 56
4.3.2 Single-change results . . . 57

4.3.3 Compound releases . . . 59
4.3.4 Call-site detection benchmark . . . 60
4.3.5 Longitudinal study of a real specification . . . 62
4.4 Analytical Method . . . 66
4.4.1 Predicted versus measured scaling . . . 66
4.4.2 Scaling of the AST scan . . . 68
4.4.3 Reference inflation . . . 69
4.4.4 End-to-end cost model . . . 70
4.5 Comparison of Results Obtained by Different Methods . . . 70
4.6 Justification for the Differences and Errors . . . 71
4.7 Threats to Validity . . . 72
4.8 Summary . . . 72

5 Conclusions . . . 73
5.1 Conclusions . . . 73
5.2 Future Scope . . . 74
5.3 Applications and Utility . . . 75

Contributions . . . 76

References . . . 77

A Rule Reference . . . 84

B Configuration and Command Reference . . . 85
B.1 Configuration file . . . 85
B.2 Exit codes and environment . . . 85
B.3 Minimal scheduled workflow for any repository . . . 86

C Complete List of Unit Tests . . . 87

D Prompt Given to the Repair Agent . . . 90

E Pull Request Produced by the System . . . 91

F Selected Source Code . . . 93

G Experiment Artifacts . . . 96

Acknowledgement . . . 97

LIST OF ABBREVIATIONS AND SYMBOLS

Abbreviations

Abbreviation	Full form
AI	Artificial Intelligence
API	Application Programming Interface
APR	Automated Program Repair
AST	Abstract Syntax Tree
CI	Continuous Integration
CLI	Command-Line Interface
CRUD	Create, Read, Update, Delete
DAG	Directed Acyclic Graph
ESM	ECMAScript Modules
FN	False Negative
FP	False Positive
GT	Ground Truth
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
LLM	Large Language Model
LOC	Lines of Code
OAS	OpenAPI Specification
PR	Pull Request
REST	Representational State Transfer
RFC	Request for Comments
RSS	Resident Set Size (memory) / Really Simple Syndication (feeds)
SDK	Software Development Kit
SE	Software Engineering

Abbreviation	Full form
sma	Self-Maintaining API integrations (the system developed in this project)
TP	True Positive
TN	True Negative
UI	User Interface
URL	Uniform Resource Locator
YAML	YAML Ain't Markup Language

Symbols

Symbol	Meaning
$S_0,\ S_1$	Baseline (previous) and current versions of an OpenAPI specification
$\rho(S)$	Specification S after inlining of all local `$ref` references
$\Delta(S_0, S_1)$	Set of structural deltas between two specifications, each a triple (operation, pointer, value)
$P(S)$	Set of path templates declared in S
$O(S)$	Set of operations of S, each a pair (path, method)
$F_S(o)$	Field map of operation o: parameters and request-body properties keyed by (location, name)
$R_S(o)$	Response-field map of operation o (lowest 2xx response, JSON content)
$\mathscr{R}$	The rule table; $\lvert\mathscr{R}\rvert = 17$
H	Set of rule hits produced for one pair of specifications
$\sigma(r)$	Severity of rule r: breaking, warning, safe or unknown
L	Leftover deltas that no rule explains
θ	Confidence floor for LLM verdicts; $\theta = 0.7$
c	Confidence value returned by the LLM for one verdict, $0 \leq c \leq 1$
C	Set of call sites found in the consumer repository
$\text{shape}(p)$	Path p with every template variable replaced by `{}`

Symbol	Meaning
$I(h)$	Impact set of hit h: the call sites that h affects
Q	Set of repairable changes (breaking and used)
Φ	Set of files actually changed by the repair agent
$\hat{\Phi}$	Set of files predicted by the impact analysis
T	Boolean outcome of the project's test suite (pass = true)
k	Number of simultaneous changes injected in a compound mutation trial
n	Size of a specification, measured in operations
$t(n)$	Measured execution time for a specification of size n
$\beta_0,\ \beta_1$	Intercept and slope of a fitted linear model $t(n) = \beta_0 + \beta_1 n$
R^2	Coefficient of determination of a fitted model
$P,\ R,\ F_1$	Precision, recall and F1 score

LIST OF FIGURES

Fig. 1.1 How a Breaking Change in a Third-Party API Reaches a Consumer Today . 2
Fig. 1.2 The Closed Maintenance Loop Implemented by `sma` 3

Fig. 2.1 Positioning of Existing Approaches by Side Served and Depth of Automation 20

Fig. 3.1 Module Architecture of `sma` with External Systems and Shared State . . . 26
Fig. 3.2 Data Flow Through One Run of `sma run --fix` 28
Fig. 3.3 Sequence of Interactions During Repair and Verification of One API 29
Fig. 3.4 Flowchart of the Classifier: Deterministic Rules First, LLM Only for Leftovers 38
Fig. 3.5 Impact Join and Grouping on the Petstore Demo: 6 Hits, 8 Sites, 3 Repairs 40
Fig. 3.6 Layout of the On-Disk State Directory 42
Fig. 3.7 Output of `sma doctor` on the Evaluation Machine 44
Fig. 3.8 HTML Report After the Upstream v2 Release, Before Repair (Top Part) . . 44
Fig. 3.9 HTML Report After Repair: Verdict Banner and Repair Run Log 45
Fig. 3.10 Continuous Integration and Scheduled Workflows 46

Fig. 4.1 Terminal Output of the Detection Run After the Petstore v2 Release (Machine A) 52
Fig. 4.2 Terminal Output of the Demo Application Tests (Condensed; Machine A) . 55
Fig. 4.3 Classification of Two Real OpenAI Specification Changes (Machine B) . . 55

LIST OF GRAPHS

Graph 4.1 Sum of Unit Test Durations per Test File (Machine A, Logarithmic Scale) . 51
Graph 4.2 Demo Application Tests Passing Before and After the Upstream Change and the Repair 54
Graph 4.3 Breaking-Change Detection Scores of sma and the Naive Structural Diff . . 58
Graph 4.4 Operators on Which the Naive Diff Errs, with Error Rate (sma Errs on None) 58
Graph 4.5 Label Accuracy and Broken-Operation Recall as Releases Carry More Changes 60
Graph 4.6 Precision, Recall and F1 of the Three Call-Site Detection Methods 61
Graph 4.7 Share of Each Statement Category Reported as a Call Site, by Method . . . 62
Graph 4.8 Growth of OpenAI's Published Specification, January 2022 to October 2026 64
Graph 4.9 Breaking and Safe Findings per Quarter in OpenAI's Specification History 65
Graph 4.10 Distribution of Findings over the Rules Across the Whole History (Artifacts Removed) 65
Graph 4.11 Processing Time per Version Against Specification Size (OpenAI History, Machine B) 67
Graph 4.12 Total Detection Time Against Specification Size, with the Fitted Linear Model 68
Graph 4.13 Per-Stage Detection Time on Logarithmic Axes, with Log-Log Slopes . . . 68
Graph 4.14 AST Scan Time Against Repository Size, with the Fitted Linear Model (Machine A) 69

LIST OF TABLES

Table 1.1 Typical Web API Changes and Their Effect on Existing Consumer Code . . 5
Table 1.2 Scope of the Project 7

Table 2.1 Sources Reviewed, by Theme and Period 10
Table 2.2 Quantitative Findings of Empirical Studies on API Evolution 13
Table 2.3 Comparison of Existing Approaches with sma 19
Table 2.4 Summary of Principal Works Reviewed 21

Table 3.1 Development Phases, Effort and Exit Criteria 25
Table 3.2 Technology Choices and Rejected Alternatives 27
Table 3.3 Source Modules, Responsibilities and Size 28
Table 3.4 Formal Definition of the 17 Classification Rules 32
Table 3.5 Analytical Time Complexity of Each Stage 34
Table 3.6 Commands of the sma Command-Line Interface 43
Table 3.7 Layers, Prerequisites and Behavior When a Prerequisite Is Missing 43
Table 3.8 Designed Differences Between Petstore v1 and v2 45
Table 3.9 Defects Found by End-to-End Use and Their Fixes 47
Table 3.10 Unit Test Files and What They Assert 48

Table 4.1 Evaluation Methods, Research Questions and Pipeline Stages Covered . . . 50
Table 4.2 Experimental Environment 50
Table 4.3 Unit Test Results on Machine A 51
Table 4.4 Petstore Detection and Impact Results Compared with Ground Truth 53
Table 4.5 Recorded Repair Run: What Each Agent Changed 53
Table 4.6 OpenAI Replay: Detected Changes and Their Impact on the Sample Application 56
Table 4.7 Mutation Operators Used in the Benchmark 57
Table 4.8 Breaking-Change Detection on 6,700 Single-Change Trials: sma versus Naive Diff 57
Table 4.9 Compound-Release Results by Number of Simultaneous Changes k 59
Table 4.10 Composition of the Labeled Call-Site Corpus 60
Table 4.11 Call-Site Detection on the Labeled Corpus: AST Scanner versus Text Search 61
Table 4.12 Longitudinal Study of OpenAI's Specification: Data Summary 63

Table 4.13 Kinds of Breaking Findings in the History (492 Findings Recorded in Detail) 65
Table 4.14 Median Stage Times on Synthetic Specifications (Machine B) 67
Table 4.15 Comparison of Results Across Evaluation Methods 70

Table A.1 Classification Rules with Examples 84

Table B.1 Exit Codes and Environment Variables 86

Table C.1 Unit Tests and Durations 87

Table G.1 Experiment Scripts and Result Files 96

LIST OF LISTINGS

3.1 Fetch Loop with Timeout, Retry and Redacted Errors (src/watcher.mjs) 35
3.2 Generic Structural Diff Emitting RFC 6901 Pointers (src/diff.mjs) 36
3.3 Structural Rename Inference in compareFields (src/rules.mjs) 37
3.4 JSON Schema Constraining the LLM Verdicts (src/classify.mjs) 38
3.5 Algorithm of the Call-Site Scan (pseudocode) 39
3.6 Data Contracts Between Modules 42
4.1 Complete Patch Produced by the Agent Runs (git diff main..sma/petstore-20261018093325) 54
B.1 Annotated watch.yaml 85
B.2 GitHub Actions Workflow to Run sma Daily 86
D.1 Prompt for the PARAM_RENAMED Repair (Recorded Run of 18 October 2026) 90
E.1 Pull Request Body of the Recorded Repair Run 91
F.1 Reference Inlining with Cycle Guard (src/watcher.mjs) 93
F.2 Classification: Rules First, LLM for Leftovers, Confidence Floor (src/classify.mjs) 93
F.3 Impact Matching and Repair Grouping (src/impact.mjs) 94
F.4 Independent Verification After the Agent Runs (src/repair.mjs) 95

CHAPTER 1

INTRODUCTION

1.1 Introduction

Very little software is written from scratch today. A typical web or mobile application sends payments through Stripe, text messages through Twilio, source-control events through GitHub and natural-language requests through OpenAI. Each of these services is reached through a *web Application Programming Interface* (API): a set of HTTP endpoints, each with its own parameters, request body and response shape. The application, called the *consumer*, writes code against that interface; the company that runs the service, called the *provider*, owns it and changes it whenever its own business requires.

The consumer and the provider are, in the words of Espinha *et al.*, "loosely coupled yet strongly tied" [1]. They are loosely coupled because they are separate organizations that never share a code base, a release schedule or a test suite. They are strongly tied because a single change on the provider side, such as renaming a query parameter from `limit` to `pageSize`, silently invalidates every line of consumer code that still sends `limit`. Unlike a library, which the consumer can keep at an old version for as long as it likes, a web API exists in only one version on the provider's servers: when the provider moves, every consumer moves with it, ready or not [2].

To make their interfaces machine-readable, most large providers now publish an *OpenAPI specification* [3]: a JSON or YAML document that lists every path, every operation (HTTP method), every parameter with its type and whether it is required, and the schema of every response. OpenAI, Stripe, GitHub, Cloudflare, Twilio, DigitalOcean, Asana and Box all keep their specifications in public Git repositories; OpenAI's, for example, received 478 commits between January 2022 and October 2026 [4], and the project's market research counted more than a hundred commits in thirty days for several of the others [5]. In other words, the information that would warn a consumer about an upcoming break is usually published, in a precise and structured form, before the break reaches production. What is missing is something on the consumer's side that reads it.

Figure 1.1 shows how a breaking change typically travels today. The provider edits its specification and deploys. The consumer's code, its tests (which usually run against mocks) and its continuous integration pipeline all stay green, because nothing in the consumer's repository changed. The first signal is a failure in production, followed by a developer reading changelogs to find out what happened, locating every affected call by hand, patching it and hoping that

nothing was missed.

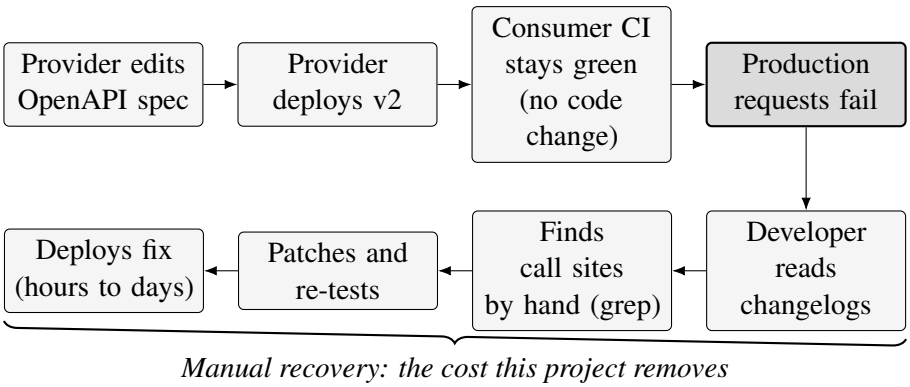


Figure 1.1: How a Breaking Change in a Third-Party API Reaches a Consumer Today

This project builds the missing piece. The system, named `sma` (*Self-Maintaining API integrations*), is a command-line tool written in Node.js that runs on a developer's machine or on a daily schedule in GitHub Actions [5]. Given a short configuration file that names the APIs a repository depends on, `sma` performs the following loop on every run:

1. **Watch.** Fetch the current OpenAPI specification of each API and compare it with the last version seen, which is kept as a committed baseline in the repository.
2. **Classify.** Decide, for every difference, whether it is *breaking* (existing correct client code can now fail), a *warning* (a deprecation) or *safe* (purely additive), using a table of 17 deterministic rules. Only differences that no rule explains are sent to a large language model (LLM), and its verdicts are trusted only above a confidence of 0.7.
3. **Locate.** Parse the consumer's own JavaScript and TypeScript files into abstract syntax trees (ASTs), find every HTTP call that reaches the watched API, and also find the functions that call those wrappers, one hop away.
4. **Join.** Match each breaking change to the call sites it affects. A breaking change that nothing in the repository uses is reported as *not affected* and costs nothing further.
5. **Repair.** Hand each breaking-and-used change to a headless coding agent (Claude Code [6]) that is allowed to read and edit files but is not allowed to run any command.
6. **Verify.** Run the project's tests independently and check that the agent touched only the files the impact analysis predicted. Open a pull request (PR), labeled `needs-human` if either check fails. Nothing is ever merged automatically.

Figure 1.2 summarizes this loop. The rest of this chapter explains why such a tool is needed, what it set out to achieve, the idea that holds the design together, and how this report is organized.

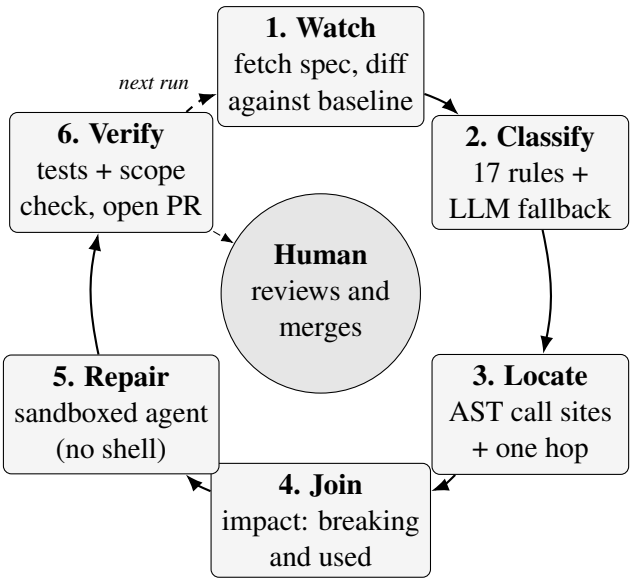


Figure 1.2: The Closed Maintenance Loop Implemented by sma

1.2 Need, Necessity and Motivation

1.2.1 Providers break consumers, often without warning

The first motivation is that breaking changes in web APIs are common and are seldom announced in a way a consumer can act on. Di Lauro, Serbout and Pautasso mined 780,078 commits to the OpenAPI descriptions of 271,111 APIs on GitHub and found that, among the 41,627 APIs with more than one commit, only 263 (0.6%) removed operations that they had first marked as deprecated, while 10,242 (24.6%) removed operations with no deprecation at all [7]. In the same study, 94.8% of the operations that *were* marked deprecated were never removed. A deprecation flag is therefore a poor predictor of removal in both directions: most removals come without one, and most flags never lead to a removal.

Version numbers do not help much either. A large study of 603,293 API specifications found more than 50 different formats for version identifiers, encoding anything from semantic compatibility to release dates [8]. Even in package ecosystems that formally adopt Semantic Versioning [9], the convention leaks: in npm, 44% of the breaking changes that actually broke client builds came from minor or patch releases [10]. A consumer that trusts the version string, or waits for a deprecation notice, will be surprised.

1.2.2 Breaking changes are real and costly in practice

Industry surveys tell the same story from the other side. In Postman's 2024 survey of more than 5,600 developers and API professionals, only 56% of API changes were reported to succeed with minimal issues [11]. The real-world case used later in this report illustrates the point. OpenAI announced on 3 June 2022 that its legacy `/engines`, `/answers`, `/search` and `/classifications`

endpoints would be shut down on 3 December 2022 [12]; the endpoints were removed from the published specification only on 19 June 2023 (commit `d9c3021a`), about six and a half months after the shutdown. Any consumer still calling those endpoints simply stopped working, and the machine-readable record of the change appeared even later than the change itself.

1.2.3 Existing tools serve the wrong side of the interface

Mature tools exist for comparing two versions of a specification: oasdiff [13], openapi-diff [14], Optic [15] and the change-management features of Bump.sh [16]. All of them are built for the *provider*, who runs them in its own pipeline to stop itself from shipping a breaking change by accident. None of them looks at a consumer's source code, so none can answer the consumer's real questions: *which lines of my code does this change break, and how do I fix them?* Dependency bots such as Dependabot and Renovate do work on the consumer side, but they only raise version numbers in package manifests; they do not adapt the code that calls the dependency [17, 18]. Consumer-driven contract testing with Pact needs the provider to take part, which public providers do not [19].

1.2.4 Static evidence is precise; manual work is not

Two observations make automation attractive. First, most breaking changes do not affect a given client at all: across 293,817 Maven clients only 7.9% were affected by breaking changes in the libraries they used, because most changes touched code that no client called [20]; Xavier *et al.* report a similar 2.54% for Java libraries [21]. A tool that joins changes to actual usage can therefore suppress most alarms. Second, developers do not keep up unaided: 81.5% of more than 4,600 studied projects still used outdated dependencies [22]. Searching a code base for affected calls by hand, typically with `grep`, is both slow and error-prone, because a string that mentions `/pets` is not necessarily a call and a call is not always written as a literal string.

1.2.5 Language models can patch code, but cannot be trusted alone

Finally, large language models now resolve real repository issues and migrate library usage [23, 24, 25]. Yet the evidence is clear that their output must be checked by something other than themselves. GitHub Copilot's agent mode migrated 100% of library usage points in a median project but preserved behavior in only 39.75% of the tests [26]. GPT-4o produced correct individual edits 94% of the time, yet only 64% of its migrations passed the developers' tests [27]. Tests themselves can be overfitted by automated repair [28], and people over-trust code written by AI assistants [29]. Any tool that lets a model edit code therefore needs an independent verification step and a human at the end.

Taken together, these points define the need: a consumer-side tool that reads the provider's

published specification, decides deterministically what is breaking, finds precisely where the consumer is affected, proposes a fix, verifies that fix without trusting the model, and leaves the final decision to a person. Table 1.1 lists the kinds of change that motivate the work and their effect on consumer code.

Table 1.1: Typical Web API Changes and Their Effect on Existing Consumer Code

Change in the spec	**Example**	**Effect on existing consumer code**
Endpoint removed	`/engines` deleted (OpenAI, 2023)	Every request returns 404; feature stops working
Required parameter added	`ownerId` required on `POST /pets`	Requests are rejected with 400 until the value is sent
Parameter renamed	`limit` becomes `pageSize`	Old name silently ignored; wrong page size returned
Optional parameter made required	`prompt` required on `POST /completions` (OpenAI, 2023)	Calls that omitted it now fail
Response field removed	`tag` dropped from `Pet`	Code reads `undefined`; output silently corrupted
Field type changed	`id` from integer to string	Comparisons and arithmetic misbehave
Enum value removed	status `pending` no longer accepted	Requests using it are rejected
Authentication changed	API key header replaced by bearer token	Every request is unauthorized
Endpoint deprecated	`deprecated: true`	Still works; signals a future removal
Endpoint or field added	new `/pets/{id}/owner`	No effect on existing code (safe)

1.3 Objectives

The project was carried out with the following objectives. Each is measured in Chapter 4.

O1. Consumer-side change detection. Fetch any OpenAPI 3.x specification (JSON or YAML, local file or URL, optionally behind an authorization header) and detect every structural change since the last committed baseline, without ever losing a change because of a transient failure.

O2. Deterministic, explainable classification. Classify each change as breaking, warning or safe with a fixed rule table, so that the same two specifications always give the same answer and every verdict names the rule that produced it. Avoid false positives from cosmetic edits (descriptions, examples, reordering).

O3. Bounded use of AI for the residue. Use an LLM only for differences that no rule explains, require a structured verdict with a confidence value, and never let a verdict below a confidence floor of 0.7 trigger an automatic repair. Keep a fully offline mode

(`--no-llm`).

O4. Precise impact analysis. Locate every HTTP call site of a watched API in a JavaScript or TypeScript repository through AST analysis, including calls written with template literals, plus one hop of callers; report unresolvable calls instead of dropping them; and keep only the breaking changes that the repository actually uses.

O5. Safe, verified repair. Patch the affected call sites with a coding agent restricted to file tools, then verify the result independently by running the project's tests and comparing the changed files with the predicted impact. Deliver the result as a pull request, never as a merge.

O6. Operability. Provide a CLI with distinct exit codes (0 nothing broken, 1 a breaking change reached this repository, 2 the tool itself failed), a self-contained HTML report, a scheduled GitHub Actions workflow, and graceful degradation when optional components (API key, agent CLI, GitHub CLI) are missing.

O7. Evaluation. Evaluate the system by at least two independent methods: controlled and real-world experiments, statistical benchmarks with ground truth against baseline methods, and an analytical model of its computational cost.

1.4 Theme

The theme of the project is **closing the loop on the consumer side, with determinism wherever possible and trust nowhere it is not earned**. Three ideas follow from it and recur through every chapter.

Consumer-side counterpart of provider tooling. Providers already diff their specifications before release. `sma` moves the same idea to the other side of the interface: it watches *someone else's* specification and reasons about *its own* code. The baseline specification is committed to the consumer's Git history, so the history of everything an upstream API ever changed becomes `git log .sma/specs/<api>.json`.

Determinism first, AI second. Every stage that can be deterministic is deterministic: fetching, reference inlining, structural diffing, the 17 classification rules, the AST scan and the impact join. These stages are unit-tested against exact expected outputs. The non-deterministic components, an LLM for unexplained differences and a coding agent for repairs, are placed at the edges, given narrow inputs, and gated: the LLM by a confidence floor and the agent by a tool sandbox.

Independent verification and a human at the end. The component that produces a patch is never the component that judges it. The agent has no shell, so it cannot run the tests and cannot claim a success it did not earn; the harness runs them itself. The impact analysis that briefs the agent is reused to check it: a file changed outside the predicted set is a signal that

static analysis and the model disagree. Doubtful results are labeled `needs-human`, and no result is merged without a person.

Table 1.2 records the scope that this theme implies, including what was deliberately left out.

Table 1.2: Scope of the Project

In scope	Out of scope (deliberate)
OpenAPI 3.x specifications (JSON or YAML), public or token-protected	APIs that publish no specification (*e.g.*, Twitter/X, Reddit)
Consumer code in JavaScript and TypeScript	Other languages (only the inventory module would change)
HTTP calls via `fetch`, `axios`, `got`, `ky`, `request`; generated SDK method chains	Hand-written SDK wrappers and three or more layers of wrappers
Structural, contract-level breaking changes	Pricing, quota, shutdown or policy changes that no spec shows
A branch and a pull request per API per run	Automatic merging of any change
Daily scheduled runs in CI, or on demand	Real-time push notification from providers

1.5 Organization

The rest of this report is organized as follows.

Chapter 2, Literature Survey, reviews standard books, journal and conference papers, standards and tool documentation on web API evolution, library migration, specification diffing, change impact analysis, automated program repair and the trustworthiness of AI-generated code, with emphasis on work from the last three to five years. It ends with a comparison of fourteen existing approaches and a statement of the research gap that this project addresses.

Chapter 3, System Development, presents the requirements, the architecture and the detailed design of every module of `sma`. It develops a mathematical model of the system: specifications as trees, the structural diff as a set of JSON-pointer deltas, the rules as predicates over normalized operations, the confidence gate, the impact join as a relational match and the verification predicate. It also gives the analytical complexity of each stage, the data formats, the command-line interface, the HTML report and the CI workflows.

Chapter 4, Performance Analysis, evaluates the system by three methods: experimental (unit tests, a controlled end-to-end upgrade with a real agent repair, and the OpenAI real-world replay), statistical (6,700 single-change and 2,700 compound mutation trials against a naive baseline, a labeled call-site corpus against two grep baselines, and a longitudinal study of all 478 commits to OpenAI's specification) and analytical (complexity predictions compared with measured run times). The results of the methods are compared and the differences and errors are justified.

Chapter 5, Conclusions, summarizes the findings, lists the future scope (including the defect found during evaluation), and describes the applications of the tool. A page on the **Contributions** of the work follows.

The **References** are listed in order of first citation. The **Appendices** contain the complete rule specification, the configuration and CLI reference, the full list of unit tests, the prompt given to the repair agent, a pull request produced by the system and selected source code. The report closes with the **Acknowledgement**.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction and Survey Method

This chapter reviews what is already known about the problem stated in Chapter 1 and about the techniques that sma combines. The survey had three aims: (i) to establish, with evidence, how often and how web APIs break their consumers; (ii) to identify which existing tools and research prototypes address any part of the consumer's problem; and (iii) to locate the precise gap that the project fills.

Sources. Standard textbooks, journal articles (*e.g.*, Journal of Systems and Software, Empirical Software Engineering, ACM Transactions on Software Engineering and Methodology, IEEE Transactions on Software Engineering, ACM Computing Surveys), conference proceedings (ICSE, ESEC/FSE, ASE, ISSTA, SANER, ICSME, ICWS, ECSA, ICSA, NeurIPS, ICLR, CCS, IEEE S&P), standards documents (IETF RFCs, the OpenAPI Specification, JSON Schema) and the official documentation of tools were consulted. Every reference was checked against its publisher record, DOI, arXiv page or official web page on 21 October 2026, and summaries report only figures stated in the sources themselves.

Search. Searches combined the terms *web API evolution*, *breaking change*, *OpenAPI diff*, *deprecation*, *library migration*, *dependency update bot*, *change impact analysis*, *call graph JavaScript*, *automated program repair*, *large language model code migration* and *AI code assistant security*. Backward snowballing from recent surveys [30, 31, 32] found older foundational work.

Emphasis on recent work. In line with the report guidelines, preference was given to work from the last three to five years (2021 to 2026), while classic papers were kept where they define a concept that the project relies on. Table 2.1 summarizes the sources reviewed.

2.2 Background: Web APIs, REST and OpenAPI

2.2.1 REST and HTTP semantics

Most public web APIs follow the Representational State Transfer (REST) architectural style defined by Fielding [33]: resources are identified by URLs, manipulated through a small, uniform set of HTTP methods, and represented in a negotiated media type, usually JSON. The meaning of those methods, status codes and authentication headers is standardized in RFC 9110 [34].

Table 2.1: Sources Reviewed, by Theme and Period

Theme	Up to 2020	2021 to 2026	Total
A. Web API evolution and breaking changes	6	9	15
B. Library migration and dependency bots	4	5	9
C. Specifications, standards and diff tools	2	14	16
D. Static analysis and change impact analysis	4	2	6
E. Automated program repair and LLMs for code	2	8	10
F. LLM-based library and API migration	0	6	6
G. Trust and verification of AI-generated code	1	3	4
H. Standard textbooks	5	0	5
Project tools and documentation (Node.js, GitHub)	0	4	4
Total	**24**	**51**	**75**

Living documentation pages (tool manuals, the OpenAI deprecations page) are counted in the period in which they were read.

Practitioner texts such as *RESTful Web APIs* by Richardson, Amundsen and Ruby describe how such APIs are designed to survive change, which they call designing "services for a changing world" [35]. From a consumer's point of view, however, a REST API is a contract: a path template such as `/pets/{id}`, a method, a set of parameters with locations (`query`, `path`, `header`, `cookie`) and types, a request body schema and a response schema. Any edit that narrows what the provider accepts, or widens what the consumer must handle, can break that contract.

2.2.2 The OpenAPI Specification

The OpenAPI Specification (OAS) is the dominant machine-readable format for describing such contracts [3]. A specification document contains a `paths` object whose keys are path templates; each path holds operations keyed by HTTP method; each operation lists `parameters`, an optional `requestBody` and `responses` keyed by status code. Data shapes are described with Schema Objects, which in OAS 3.1 are a superset of JSON Schema Draft 2020-12 [36], with keywords such as `type`, `properties`, `required` and `enum`. Three features of the format matter directly to this project:

- **References.** Reusable schemas live under `components/schemas` and are referenced with `$ref` pointers. A single change to a shared schema therefore appears in every operation that references it. `sma` inlines local references before diffing so that each operation can be compared on its own (Section 3.5).
- **Deprecation.** Operations and parameters may carry a boolean `deprecated` flag, and consumers "SHOULD refrain from usage" of deprecated operations [3]. This feeds the `PATH_DEPRECATED` warning rule.

- **Security schemes.** Authentication methods are declared under `components/securitySchemes`; changing or removing one is breaking for every call (`AUTH_SCHEME_CHANGED`).

OAS 3.1 also allows `type` to be an *array* (for example `["integer", "null"]`) to express nullable values. Section 4.3.5 shows that this small difference from OAS 3.0 matters for any tool that compares types.

2.2.3 JSON Pointer

RFC 6901 defines JSON Pointer, a string syntax for identifying one value inside a JSON document [37]. Because the path keys of an OpenAPI document themselves contain slashes, the escapes `~1` (for `/`) and `~0` (for `~`) are essential: the operation `GET /pets/{id}` is addressed as `/paths/~1pets~1{id}/get`. `sma` reports every structural difference as an RFC 6901 pointer, and the rule engine records which pointers each finding explains.

2.2.4 Versioning conventions

Semantic Versioning 2.0.0 requires the major version to be incremented "when you make incompatible API changes" [9]. It is the convention that dependency bots rely on. Several studies reviewed below show that real projects follow it inconsistently, which is one reason `sma` inspects the specification diff itself rather than the version string.

2.2.5 Software evolution and maintenance

Standard software engineering texts treat evolution and maintenance as the dominant share of a system's lifetime cost and describe adaptive maintenance, changing software to keep it working in a changed environment, as one of its main categories [38, 39]. Adapting to a provider's API change is a textbook case of adaptive maintenance, and refactoring catalogs such as Fowler's [40] describe many of the mechanical edits involved (renaming, threading a new parameter through a call chain). The contribution of `sma` is to automate the detection and much of the editing for this class of maintenance.

2.3 Web API Evolution and Breaking Changes

2.3.1 Early qualitative studies

Li, Xiong, Liu and Zhang studied how changes in web service APIs affect clients [2]. They observed that, unlike clients of local libraries, web API clients usually cannot keep using an old version, which makes migration urgent. Analyzing 5 popular web APIs and 256 changed API

elements, they grouped changes into 16 change patterns and identified 6 migration challenges that do not arise with local APIs. Espinha, Zaidman and Gross interviewed six client developers and analyzed the evolution policies of Twitter, Google Maps, Facebook and Netflix, concluding that clients of web APIs are "loosely coupled yet strongly tied" to providers whose changes they cannot control [1]. Wang, Keivanloo and Zou identified 21 change types in RESTful APIs, 7 of them new, and found that deleted methods generate the most developer discussion on Stack Overflow [41]. Sohan, Anslow and Maurer studied how one evolving web API documented and communicated its changes and made recommendations about versioning and communication [42]. These studies supply the catalogs of change patterns on which a rule table such as sma's is based, but none of them provides tooling that connects a change to client code.

2.3.2 Large-scale studies of OpenAPI histories

The availability of thousands of OpenAPI documents on GitHub enabled quantitative work from 2020 onwards. Yasmin, Tian and Yang built RADA, which reads OpenAPI specifications, finds deprecated elements and the operations they affect, and applied it to 2,224 specifications of 1,368 APIs from APIs.guru [43, 44]; they reported several serious deprecation-related problems and left the migration of client code to future tools. Di Lauro, Serbout and Pautasso first proposed a method for assessing web API evolution at scale from crawled OpenAPI documents, finding that only a subset of APIs change and that histories are more variable than a simple "only grow" model predicts [45]. Their follow-up study, already cited in Chapter 1, is the strongest evidence for this project: of 67,577 removed operations, only 466 had been deprecated first, and 24.6% of APIs removed operations with no deprecation at all [7]. Serbout and Pautasso then analyzed 603,293 specifications created between 2015 and 2023 and found more than 50 formats for version identifiers [8]. Robles *et al.* derived a taxonomy of structural REST API changes classified by their impact on clients from Azure APIs on APIs.guru, and observed that breaking changes tend to decrease in newer versions [46].

2.3.3 Breaking changes in library ecosystems

Library ecosystems have been studied longer and offer two lessons that transfer directly to web APIs. The first is that breaking changes are frequent: across 317 Java libraries, about 9,000 releases and 260,000 client applications, Xavier *et al.* found that 14.78% of API changes break backward compatibility and that the frequency increases over time [21]. The second is that most of them do not affect any particular client: only 2.54% of clients in the same study were actually affected. Ochoa *et al.* replicated and extended an earlier Maven Central study over seven more years with their bytecode tool Maracas; across 119,879 library upgrades, 83.4% complied with semantic versioning, and only 7.9% of 293,817 clients were affected by breaking changes [20]. In the JavaScript ecosystem that sma targets, Venturini *et al.* found that

about 12% of dependent npm packages were broken by a non-major release of a dependency, and that 44% of manifesting breaking changes came from minor or patch releases [10]. Clients recovered in about half the cases, usually by pinning the provider version rather than fixing their own code, an option that web API clients do not have.

2.3.4 Consumer-side detection

Two recent papers come closest to this project on the consumer side. Vidal, Herrera and Casas keep a dictionary of the APIs an application consumes and automatically check it against the current specifications to identify a defined set of change types [47]. Bonorden and van Hoorn use dynamic tracing with OpenTelemetry to record which web API calls a client actually makes, then check those calls for deprecation against an OpenAPI document, an HTTP deprecation header or a knowledge base [48]. They report a precision of 1.00 and recall of 0.95, with all misses caused by an HTTP client style the automatic instrumentation did not cover, and they motivate tracing by the difficulty of resolving call targets statically. Neither work generates or validates a fix, and the second needs live traffic and handles only deprecation. Lamothe, Guéhéneuc and Shang's systematic review of API evolution research names understanding the impact of API evolution on users as one of the three main open challenges in the field [30].

Table 2.2 collects the quantitative findings of this section.

Table 2.2: Quantitative Findings of Empirical Studies on API Evolution

Study	Data set	Key finding used in this project
Li *et al.* 2013 [2]	5 web APIs, 256 changed elements	16 change patterns; 6 client migration challenges specific to web APIs
Xavier *et al.* 2017 [21]	317 Java libraries, 260K clients	14.78% of API changes are breaking; only 2.54% of clients affected
Yasmin *et al.* 2020 [43]	2,224 OpenAPI specs, 1,368 APIs	Deprecation analyzable from specs; client migration left open
Di Lauro *et al.* 2022 [7]	271,111 APIs, 780,078 commits (41,627 with more than one)	24.6% remove operations without deprecation versus 0.6% with it
Ochoa *et al.* 2022 [20]	119,879 upgrades, 293,817 clients	83.4% semver compliant; only 7.9% of clients affected
Venturini *et al.* 2023 [10]	npm packages	44% of manifesting breaks came from minor or patch releases
Serbout, Pautasso 2024 [8]	603,293 API specs	More than 50 version identifier formats in use
Bonorden, van Hoorn 2024 [48]	3 evaluation settings	Runtime tracing: precision 1.00, recall 0.95 for deprecated calls
Postman 2024 [11]	5,600+ survey respondents	Only 56% of API changes succeed with minimal issues

2.4 Library Migration and Dependency Management

2.4.1 Refactoring-based migration

Dig and Johnson studied API changes in four frameworks and one library and found that more than 80% of the changes that broke client applications were refactorings such as renames and moves [49]. They argued that migration tools should replay such refactorings on client code. Henkel and Diwan's CatchUp! did exactly that: it recorded refactorings in the library developer's IDE and replayed them on client code [50]. The approach requires the provider to record its refactorings, which third-party web API providers do not do; a web API consumer sees only the before and after specifications. The finding that breaking changes fall into a small number of recognizable categories is nevertheless what justifies a finite rule table, including a structural rule for renames.

2.4.2 Learning adaptations from examples

Nguyen *et al.*'s LibSync learns API adaptation patterns from clients that have already migrated and from the library's own tests, using graph-based techniques to recommend edit locations and operations; it reported 100% precision and 91% recall on real systems [51]. Dilhara *et al.*'s PyCraft combines static and dynamic analysis with LLMs to generalize code change patterns from examples, reaching an F-measure of 96.6% in identifying correct variants and an 83% acceptance rate for patches submitted to open-source projects [52]. Both depend on examples of the change already made somewhere else. When a third-party API changes overnight, no such examples exist yet; sma substitutes the specification diff and an LLM agent for the missing examples.

2.4.3 Dependency update bots

Developers rarely update dependencies unaided: Kula *et al.* found that 81.5% of more than 4,600 studied GitHub projects still used outdated dependencies, and 69% of surveyed developers were unaware of their vulnerable dependencies [22]. Dependency bots were created to fill this gap. Dependabot opens pull requests that raise versions in manifests and lock files and advises maintainers to check tests and read release notes before merging [53]. He *et al.*'s study of 1,823 projects and 540,665 Dependabot pull requests found that projects reduce technical lag after adopting it, but that its compatibility scores are too sparse to reduce suspicion, developers often throttle its notifications, and 11.3% of projects abandoned it [17]. Renovate offers similar version and replacement pull requests across many platforms [18]. None of these bots adapts the code that calls the dependency, and none applies to a web API, which has no version to bump in a manifest. Their pull-request-based workflow and the distrust it meets, however, directly informed sma's design: its output is also a pull request, but one that carries its own

independent verification and an explicit `needs-human` label when that verification fails.

2.4.4 Benchmarks of breaking dependency updates

Reyes *et al.*'s BUMP benchmark contains 571 reproducible breaking dependency updates from 153 Maven projects, built so that research on repairing breaking updates can be compared on real, executable cases [54]. No comparable benchmark exists for breaking changes in web API specifications paired with consumer repositories, a gap that the evaluation in Chapter 4 partly addresses with a mutation benchmark and a real-world replay.

2.5 Specification Diff Tools and Contract Testing

2.5.1 Provider-side diff tools

Several tools compare two versions of an OpenAPI specification. **oasdiff** is an open-source command-line tool that compares specifications given as files, URLs or Git revisions; its `breaking` command reports "only the changes that break existing API clients", its rules can be listed and customized, and it integrates with GitHub Actions and pre-commit hooks [13]. **openapi-diff** from OpenAPITools is a Java tool that classifies changes as compatible or incompatible and can fail a build [14]. **Optic** provided OpenAPI linting, diffing and traffic-based spec generation; its repository was archived in January 2026, and Optic Labs is now part of Atlassian [15]. **Bump.sh** is a hosted documentation platform that produces a changelog for every uploaded definition, flags breaking changes and notifies teams and subscribed consumers by Slack, webhook, e-mail or RSS [16]. SDK generators such as Stainless list breaking-change detection among their features for the provider, and regenerate the SDK rather than any consumer code [55].

All of these tools answer the provider's question, *is this specification change breaking?*, and answer it well; oasdiff in particular has a far larger rule set than `sma`. None of them reads consumer source code, so none can answer the consumer's question, *which of my call sites break, and how do I fix them?*

2.5.2 Consumer-driven contract testing

Pact is a code-first tool in which consumer tests record the interactions they expect, producing a contract that the provider then verifies; only interactions that consumers actually use are tested [19]. This "test only what is used" idea is close in spirit to `sma`'s impact join. Pact, however, needs the provider to run the verification, which a public provider such as OpenAI will not do for an arbitrary consumer. `sma` obtains a similar usage focus without the provider's cooperation, from static analysis of the consumer's code plus the provider's public specification.

2.5.3 Specification sources

Public specification repositories make consumer-side watching practical. OpenAI's specification repository (`openai-openapi`), used to generate its official SDKs, is the source of the real-world case studied in Chapter 4 [4]. The APIs.guru directory collects public OpenAPI definitions and refreshes them at least weekly [44]. The OpenAI deprecations page provides ground truth for the replayed change and shows that the published specification can lag behind the actual shutdown [12].

2.6 Static Analysis and Change Impact Analysis

2.6.1 Change impact analysis

Change impact analysis (CIA) identifies the potential consequences of a change, or estimates what must be modified to accomplish it [56]. Ren *et al.*'s Chianti decomposes the difference between two versions of a Java program into atomic changes and reports which tests each change affects; on a year of Daikon's history, each affected test depended on only 3.95% of the atomic changes [57]. Li *et al.* surveyed 23 code-based CIA techniques evaluated in 30 papers, proposed a seven-property comparison framework and called for mature tools and common benchmarks [58]. `sma` applies the Chianti model across a network boundary: the "atomic changes" are rule hits on the provider's specification, and the "affected code" is the consumer's call sites. The CIA vocabulary of an *impact set* and its precision is also how `sma`'s verification compares the files the agent changed with the files the analysis predicted.

2.6.2 Abstract syntax trees and the TypeScript compiler

A compiler front end turns source text into an abstract syntax tree in which a call such as `fetch(`${BASE}/pets/${id}`)` is a `CallExpression` node whose first argument is a `TemplateExpression` [59]. Working on the tree rather than on text is what allows a tool to tell a real call from a comment or a log message that merely mentions a path. The TypeScript compiler exposes its parser, type checker and a language service that can find all references to a symbol [60]; ts-morph wraps this API to make navigating and manipulating the tree simpler [61]. `sma` uses ts-morph to parse both JavaScript and TypeScript files with a single parser and to find the callers of each wrapper function.

2.6.3 Call graphs for JavaScript

Feldthaus *et al.* showed that sound static analysis of JavaScript is very hard because of the language's dynamic features, and that a deliberately unsound but fast field-based flow analysis gives highly accurate call graphs for IDE services on large programs in seconds [62]. This

result supports sma's design choice of an approximate, fast analysis: syntactic detection of HTTP calls plus one hop of callers obtained from the language service, rather than a full call graph. It also predicts the analysis's blind spots, dynamically built URLs and deeper wrapper chains, which Chapter 4 measures.

2.7 Automated Program Repair and Large Language Models for Code

2.7.1 Classical automated program repair

Automated program repair (APR) generates patches that make a failing program pass its tests. GenProg used genetic programming to evolve program variants that pass the test suite, repairing defects in 16 C programs with 1.25 million lines of code in 357 seconds on average [63]. Monperrus's bibliography organizes the field into behavioral repair, with oracles such as tests, contracts or crashing inputs, and state repair at run time [64]. The central lesson of APR for this project is the weakness of tests as an oracle: Smith *et al.* showed that patches from GenProg and TrpAutoRepair often overfit the test suite used to produce them, with the median patch passing only 75.0% of held-out tests [28].

2.7.2 Large language models for code

Chen *et al.* introduced Codex and the HumanEval benchmark; Codex solved 28.8% of HumanEval problems with one sample and 70.2% with 100 samples [65]. Xia, Wei and Zhang then showed that pre-trained language models applied directly to repair substantially outperformed all earlier APR techniques across 5 datasets and 3 languages, with larger models doing better [66]. Two surveys map the rapidly growing field. Hou *et al.* reviewed 395 papers published between 2017 and 2024 on LLMs for software engineering [31], and Fan *et al.* argued that hallucination makes filtering of incorrect output essential and that hybrid approaches combining traditional software engineering with LLMs are central to reliability [32]. The rules-first, LLM-second classifier and the independent verifier in sma are direct applications of that hybrid principle.

2.7.3 Repository-level coding agents

SWE-bench turned real GitHub issues from 12 Python repositories into 2,294 tasks in which a model must edit a code base to resolve an issue; at publication the best model solved only 1.96% [23]. SWE-agent showed that a purpose-built agent-computer interface for editing, navigating and testing raised this to 12.5% [24]. AutoCodeRover added AST-aware code search and spectrum-based fault localization and resolved 19% of SWE-bench Lite at an average cost of 0.43 USD per issue [67]. Agentless questioned the need for autonomous agents at all, us-

ing a fixed localize, repair and validate pipeline to solve 32.00% of SWE-bench Lite at 0.70 USD [68]. Two of these findings shape sma. First, how an agent's tools are designed matters (SWE-agent); sma deliberately narrows the tools to file operations only, which the Claude Code headless mode supports through its `--allowedTools` flag [6]. Second, deterministic localization around a single generation step works well (Agentless, AutoCodeRover); sma goes one step further and computes the exact locations itself through impact analysis, instead of asking the model to search for them.

2.8 LLM-Based Library and API Migration

Migration, adapting client code to a changed dependency, is the task closest to sma's repair step, and it has received intense attention since 2024. Almeida, Xavier and Valente used ChatGPT to migrate a client application to a new version of SQLAlchemy and found that one-shot prompting worked best [25]. Their follow-up with GitHub Copilot's agent mode across ten client applications is a cautionary result: the agent achieved a median migration coverage of 100% of API usage points, yet a median test-pass rate of only 39.75% [26]. Islam *et al.* evaluated Llama 3.1, GPT-4o mini and GPT-4o on 321 real-world Python library migrations and found that individual code changes were correct 89%, 89% and 94% of the time, while only 36%, 52% and 64% of whole migrations passed the developers' tests [27]. Reyes *et al.*'s Byam used LLMs to fix client code broken by third-party API changes in the BUMP benchmark; the best model fully fixed 27% of builds and 78% of individual compilation errors when given the error messages and the API differences [69]. MigrationBench extends evaluation to repository-level Java 8 to 17 and 21 upgrades, where an agentic framework reached a pass@1 of 71.67% for minimal migration [70].

Industrial experience points the same way. Nikolov *et al.* describe LLM-assisted migrations inside Google, in which change locations are found with code search and scripts, every LLM edit is validated by building and running unit tests, and changes are never auto-submitted but sent to code owners for review; for an identifier-widening migration, 80% of code modifications in landed changes were fully AI-authored and total time was estimated to drop by about 50% [71].

Three observations follow. All of these systems start from a task that a human has already identified: a target library version, a known migration, or locations found by hand-tuned searches. The compile errors that Byam uses to find breakages do not exist for HTTP calls in JavaScript, where a removed endpoint fails only at run time. And the consistent gap between "edits look right" and "tests pass" is the strongest argument for the independent verification in sma.

2.9 Trust, Verification and Human Oversight

Evidence on human interaction with AI-written code reinforces the need for verification. In a user study by Perry *et al.*, participants using an AI assistant wrote significantly less secure code, yet were more likely to believe their code was secure [29]. Pearce *et al.* generated 1,689 programs from 89 high-risk scenarios with GitHub Copilot and found about 40% of them vulnerable [72]. Combined with test overfitting [28] and the low end-to-end pass rates above, these results support three design rules that sma follows: the agent never judges its own output, a second independent check (scope against the predicted impact) is applied on top of the tests, and a human reviews every pull request before anything is merged.

2.10 Comparative Analysis of Existing Approaches

Table 2.3 compares fourteen existing tools and approaches with sma along the seven capabilities that the consumer's problem requires. Each cell is based on the cited documentation or paper; a capability that a source does not describe is marked as absent rather than assumed. Figure 2.1 places the same approaches on two axes: which side of the interface they serve, and how far along the path from detection to a verified fix they go.

Table 2.3: Comparison of Existing Approaches with sma

Approach	Side	Input	Detects breaking	Locates call sites	Patches	Verifies indep.	Opens PR
oasdiff [13]	Provider	Two specs (file, URL, Git)	Y	N	N	N	N
openapi-diff [14]	Provider	Two specs	Y	N	N	N	N
Optic [15]	Provider	Specs, test traffic	Y	N	N	N	N
Bump.sh [16]	Provider	Uploaded definitions	Y	N	N	N	N
SDK generators [55]	Provider	Spec	P	N	N	N	N
Dependabot [17, 53]	Consumer	Manifests, lock files	N	N	P	N	Y
Renovate [18]	Consumer	Manifests, lock files	N	N	P	N	Y
Pact [19]	Both	Consumer tests	P	N	N	N	N
RADA [43]	Research	Specs	P	N	N	N	N
Vidal *et al.* [47]	Consumer	API dictionary + specs	Y	P	N	N	N
Bonorden, van Hoorn [48]	Consumer	Runtime traces + spec	P	Y	N	N	N
Issue agents [24, 67, 68]	Any repo	Issue text + repo	N	P	Y	P	N
LLM migration [26, 27, 69]	Consumer	Client + target version	N	P	Y	P	N
Google migrations [71]	Internal	Hand-specified migration	N	Y	Y	Y	Y
sma (this work)	**Consumer**	**Third-party spec versions + repo**	**Y**	**Y**	**Y**	**Y**	**Y**

Y = yes, N = no, P = partial (*e.g.*, deprecation only, version bump only, or verification only during research evaluation).

2.11 Research Gap

The survey shows that each step of the consumer's problem has been studied, but never joined into a single pipeline for third-party web APIs.

1. **Detection exists only on the provider side.** Specification diff tools classify breaking

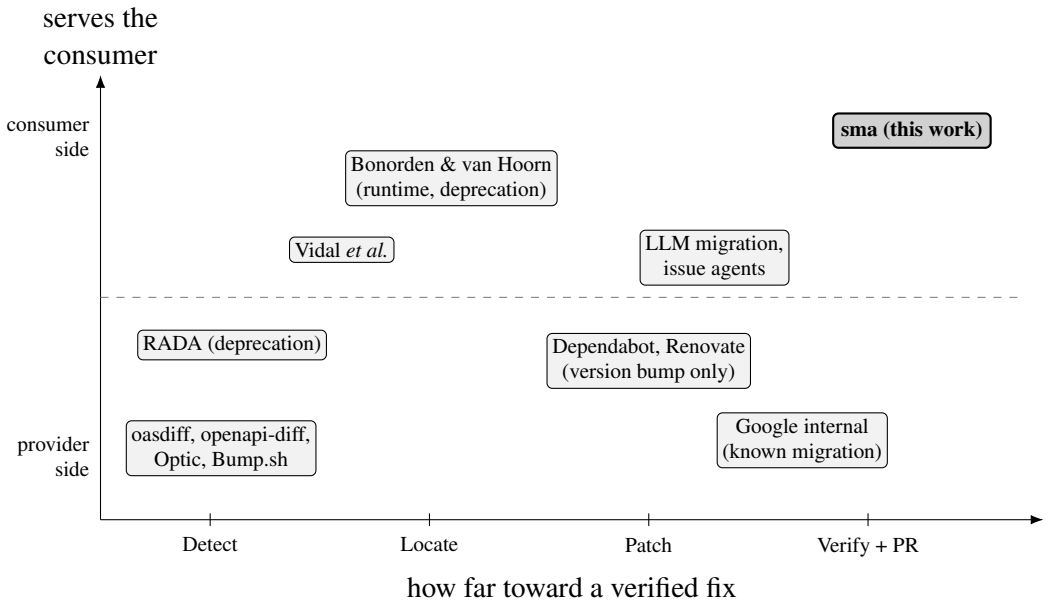


Figure 2.1: Positioning of Existing Approaches by Side Served and Depth of Automation

changes reliably [13, 14, 16, 15], but none reads consumer code. Consumer-side studies stop at detection [47] or address deprecation only, at run time [48].

2. **No join between specification changes and consumer call sites.** Library studies show that most breaking changes affect no given client [21, 20], which argues for joining changes to actual usage, and CIA provides the method [57, 58]. No tool was found that performs this join for OpenAPI-described web APIs.
3. **Repair work starts from a human-identified task.** Coding agents and LLM migration tools need an issue, a target version or hand-written location queries [23, 26, 27, 71]; Byam relies on compile errors that do not exist for HTTP calls in JavaScript [69]. None derives its task from a third-party specification diff.
4. **Agent output is not trustworthy on its own.** Median test-pass rates of 39.75% [26], 64% end-to-end success despite 94% edit-level correctness [27], test overfitting [28] and human over-trust [29] all show that verification must be independent of the generator, yet existing bots that open pull requests do not verify the code change themselves [17].
5. **No benchmark of web API specification changes paired with consumer code.** Benchmarks exist for Java dependency updates [54] and Java version upgrades [70], but not for this setting.

This project addresses gaps 1 to 4 directly: `sma` is a consumer-side pipeline that classifies a third-party specification change deterministically (with an LLM only for the residue), joins each change to statically located call sites, uses that impact set both to brief a sandboxed agent and to check its output independently, and leaves the merge to a human. It addresses gap 5 partially, with a reproducible mutation benchmark, a labeled call-site corpus and a replay of real specification history (Chapter 4). The contribution claimed is the joined-up chain and its

verification design, not any single step; several individual steps (notably oasdiff's rule set) are more complete in existing tools.

2.12 Summary of Reviewed Literature

Table 2.4 lists the principal works reviewed, with the contribution of each and its limitation with respect to the consumer's problem.

Table 2.4: Summary of Principal Works Reviewed

Work	Contribution	Limitation relative to this project
Dig, Johnson 2006 [49]	Over 80% of breaking API changes are refactorings	Library APIs; needs replay of provider refactorings
Nguyen *et al.* 2010 [51]	LibSync learns adaptation patterns from migrated clients	Needs prior migrated clients as examples
Li *et al.* 2013 [2]	16 web API change patterns, 6 client challenges	Catalog only, no tooling
Espinha *et al.* 2015 [1]	Qualitative cost of web API evolution for clients	No automation
Xavier *et al.* 2017 [21]	14.78% breaking changes; 2.54% clients affected	Java libraries, not web APIs
Kula *et al.* 2018 [22]	81.5% of projects keep outdated dependencies	Motivational; no tool
Yasmin *et al.* 2020 [43]	RADA: deprecation analysis of OpenAPI specs	Deprecation only; client migration left open
Lamothe *et al.* 2021 [30]	Systematic review of API evolution research	Survey
Di Lauro *et al.* 2022 [7]	24.6% of APIs remove operations without deprecation	Provider-side measurement only
Ochoa *et al.* 2022 [20]	Maracas joins breaking changes to client usage	JVM bytecode; not web APIs
Vidal *et al.* 2022 [47]	Consumer API dictionary checked against specs	No call-site locations, no repair
Xia *et al.* 2023 [66]	LLMs outperform classical program repair	General bugs; task given
He *et al.* 2023 [17]	Dependabot study: lag reduced, distrust remains	Bumps versions; no code adaptation

Work	Contribution	Limitation relative to this project
Venturini *et al.* 2023 [10]	44% of npm breaks from minor or patch releases	Library ecosystem only
Perry *et al.* 2023 [29]	AI assistant users write less secure code, over-trust it	Motivates human review
Serbout, Pautasso 2024 [8]	Over 50 version identifier formats	Versioning only
Bonorden, van Hoorn 2024 [48]	Runtime tracing of deprecated web API calls	Needs traffic; deprecation only; no fix
Reyes *et al.* 2024 [54]	BUMP benchmark of breaking updates	Java/Maven only
Jimenez *et al.* 2024 [23]	SWE-bench repository-level benchmark	Task from human-written issue
Yang *et al.* 2024 [24]	Agent-computer interface design (SWE-agent)	Agent has a shell and judges itself
Zhang *et al.* 2024 [67]	AST-aware search for issue repair	Generic issues, not API changes
Xia *et al.* 2024 [68]	Fixed localize-repair-validate pipeline	Generic issues; same system validates
Almeida *et al.* 2024 [25]	ChatGPT migrates a SQLAlchemy client	Target version chosen by a human
Dilhara *et al.* 2024 [52]	LLM plus transformation by example (PyCraft)	Needs examples of the change
Islam *et al.* 2025 [27]	94% correct edits, 64% migrations pass tests	Python libraries; task given
Nikolov *et al.* 2025 [71]	Industrial LLM migrations with test validation and review	Internal, known migrations; hand-tuned locations
Almeida *et al.* 2026 [26]	Agent mode: 100% coverage, 39.75% tests pass	Shows unverified agents mislead
Reyes *et al.* 2026 [69]	Byam fixes 27% of broken builds	Relies on compile errors (Java)

2.13 Summary

Web APIs break their consumers often, usually without a deprecation warning or a meaningful version change. Provider-side diff tools detect breaking changes but ignore consumer code;

dependency bots work on the consumer side but do not adapt code and do not apply to web APIs; research on consumer-side detection stops short of repair; and LLM-based repair and migration tools need a human to identify the task and cannot be trusted without independent verification. The next chapter describes how sma was designed to join these pieces into one verified, consumer-side loop.

CHAPTER 3

SYSTEM DEVELOPMENT

This chapter describes how sma was developed: the method and requirements (Section 3.1), the technology choices (Section 3.2), the architecture (Section 3.3), the mathematical model of the system (Section 3.4), the detailed design of each module (Sections 3.5 to 3.13), the data formats, the demonstration harness, automation, security and the testing strategy. The model development follows the *analytical* and *mathematical* methods of the report guidelines: every stage is first defined formally, its computational complexity is derived, and the implementation is then shown to realize the definition.

3.1 Development Methodology and Requirements

3.1.1 Methodology

The system was built by one developer in short, incremental phases, each of which had to produce runnable output before the next started. The plan fixed the data contracts between modules first, so that each module could be built and unit-tested on its own. An unusual but deliberate decision was to build the *demonstration harness first* (Phase 0): a breaking change cannot be studied, tested or demonstrated without an upstream API that the developer controls, because a real provider will not break on demand. Table 3.1 lists the phases with their planned effort and their exit criterion.

After version 1.0.0, the tool was hardened by running it end to end on the demo and on OpenAI's real specification; the defects found that way and their fixes are recorded in the project changelog and summarized in Section 3.17.

3.1.2 Functional requirements

FR1. Read a configuration file (`watch.yaml`) naming the consumer repository, an optional test command, and a list of APIs, each with a name, a specification location, an optional base URL, an optional SDK package and optional request headers.

FR2. Fetch each specification over HTTP(S) or from a local path, in JSON or YAML; retry rate-limited and server errors; time out hung servers.

FR3. Inline local `$ref` references, handling cyclic references without infinite expansion.

FR4. Store the first specification seen as a baseline; on later runs, compute and classify all

Table 3.1: Development Phases, Effort and Exit Criteria

Phase	Work	Effort	Exit criterion
0	Demo harness (petstore v1/v2, demo app)	0.5 day	Demo app tests pass on v1 and fail on v2
1	Store and CLI skeleton	0.5 day	All subcommands dispatch; `.sma/` read and write
2	Watcher, structural diff, rule engine	1.5 days	Exact expected rule hits for v1 to v2, no false positives
3	AST inventory scanner	1.5 days	Exact `file:line` set on a fixture repository
4	Impact mapper	0.5 day	Unused changes reported as not affected
5	Repair agent and independent verification	1.5 days	Branch produced; harness runs tests; scope checked
6	PR writer	0.5 day	PR body with verification block; PR URL logged
7	Automation (GitHub Actions)	0.5 day	Daily scheduled run end to end
	Total	**about 7 days**	Version 1.0.0 released 30 August 2026

differences, and advance the baseline only after classification succeeds.

FR5. Classify differences with a deterministic rule table; send unexplained differences to an LLM when a key is available, otherwise report them as `UNCLASSIFIED`.

FR6. Scan the consumer repository for HTTP and SDK call sites and their one-hop callers; report unresolvable URLs.

FR7. Join changes to call sites; report unused changes as not affected; group related findings into repairs.

FR8. For each API with repairable changes, create a branch, run one sandboxed agent per repair, verify independently, commit, and open a pull request (or write its body to disk when there is no remote).

FR9. Render a self-contained HTML report of the latest findings, call sites and repair runs.

FR10. Expose the commands `init`, `doctor`, `scan`, `check`, `impact`, `fix`, `run`, `diff`, `report`, `reset`, `version` and `help`.

3.1.3 Non-functional requirements

NFR1. Determinism. The same two specifications must always produce the same classification without an LLM.

NFR2. No silent blindness. No failure may cause a change to be recorded as already seen without being reported.

NFR3. Graceful degradation. Detection must need only Node.js and a network; repair and PR creation may degrade but must never block detection.

NFR4. Safety. No change is merged automatically; the agent cannot execute commands; credentials never appear in files, PR bodies or the report.

NFR5. Distinguishable outcomes. Exit code 0 means nothing broken, 1 means a breaking change reached the repository, 2 means the tool failed.

NFR6. Portability. Run on Linux and Windows with Node.js 22 or 24; tolerate Windows text encodings.

NFR7. Inspectability. All state is plain JSON on disk, readable with any editor and diffable with Git.

3.2 Technology Stack and Design Decisions

Table 3.2 records the main technology choices together with the alternatives that were considered and the reason each was rejected. The total runtime dependency set is three npm packages (`ts-morph`, `@anthropic-ai/sdk`, `yaml`) and three optional external command-line tools (`git`, `claude`, `gh`).

3.3 System Architecture

3.3.1 Module structure

sma is organized as one CLI entry point and ten modules, each with a single responsibility. Figure 3.1 shows the modules, the external systems they talk to and the on-disk state they share. Table 3.3 lists the modules with their size in lines of code.

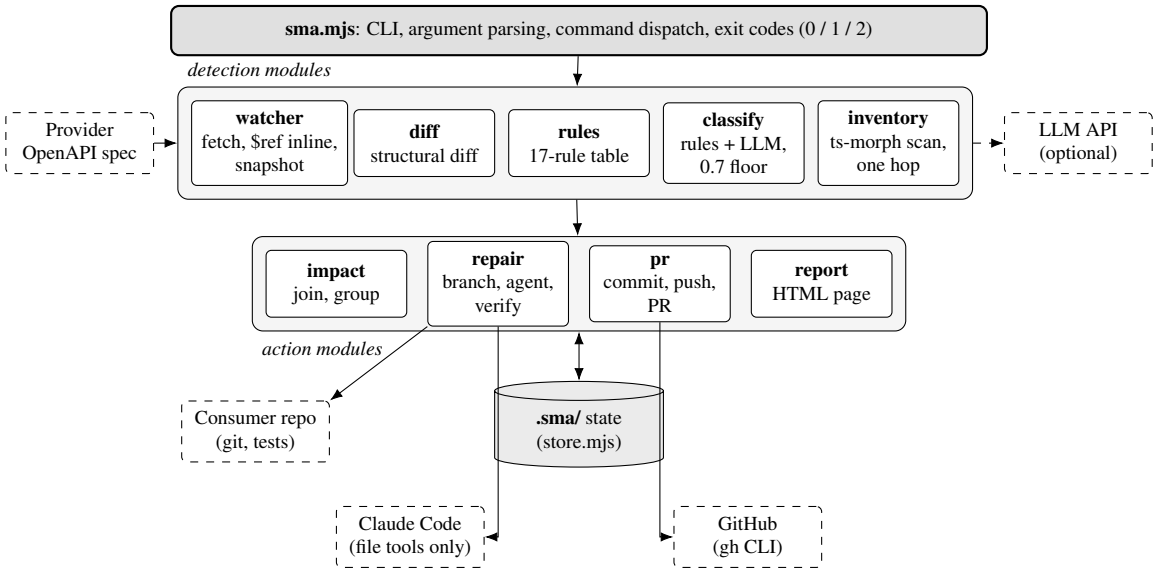


Figure 3.1: Module Architecture of sma with External Systems and Shared State

Table 3.2: Technology Choices and Rejected Alternatives

Need	Choice	Why not the alternative
Implementation language	Plain ECMAScript modules (`.mjs`), Node.js 22+, no build step	Same ecosystem as the target repositories; TypeScript would add a compile step with little gain for a small tool
Finding call sites	ts-morph (TypeScript compiler API)	Regular expressions cannot tell a call from a comment or a log line, and miss template literals (measured in Section 4.3.4)
Specification diff	Hand-written structural diff (41 lines) and rule table	oasdiff is a Go binary; the rules are the intellectual core of the project and are kept fully testable in-process
Classifier	Rules first, LLM second	A pure-LLM classifier is non-deterministic and cannot be unit-tested or reliably demonstrated
State	JSON files under `.sma/`	SQLite adds a dependency and hides state; JSON is diffable and inspectable
Repair	Headless Claude Code (`claude -p`) with an allow-list of file tools	Writing a custom edit loop on the raw model API would take days and duplicate a mature agent
LLM for left-overs	`@anthropic-ai/sdk` with JSON-schema structured output; model `claude-opus-5` by default (`SMA_MODEL` overrides)	Free-text answers would need fragile parsing
Pull requests	GitHub CLI (`gh`)	Octokit would need its own authentication plumbing
Tests	Node.js built-in test runner (`node --test`)	No extra test framework dependency [73]
Automation	GitHub Actions scheduled workflow	No server, queue or database to operate [74]

3.3.2 Pipeline and data flow

Figure 3.2 traces the data through one run of `sma run --fix`. Each arrow is labeled with the artifact that passes between stages; each artifact is also written to `.sma/` so that a run can be inspected after the fact.

3.3.3 Interaction sequence

Figure 3.3 shows the interaction between the harness, the consumer repository, the agent and GitHub for one API with breaking changes. The essential property is visible in the diagram: the agent's only interaction is editing files; every judgment about the result (tests, scope, label) is made by the harness after the agent has finished.

Table 3.3: Source Modules, Responsibilities and Size

File	Responsibility	LOC
`sma.mjs`	CLI entry: commands, flags, exit codes, `.gitignore` handling, `doctor`	349
`src/watcher.mjs`	Fetch spec (retry, timeout, redaction, header expansion), inline `$refs`, snapshots	100
`src/diff.mjs`	Generic structural diff emitting RFC 6901 pointers; `coveredBy`	41
`src/rules.mjs`	The 17-rule breaking-change table and its normalizers	246
`src/classify.mjs`	Rules first; leftover deltas to the LLM; confidence floor	119
`src/inventory.mjs`	ts-morph scan of HTTP and SDK call sites; one-hop callers	207
`src/impact.mjs`	Join changes with call sites; repair grouping; text report	80
`src/repair.mjs`	Branch, prompt, sandboxed agent run, independent verification	150
`src/pr.mjs`	PR body, commit, push, `gh pr create`	85
`src/store.mjs`	Configuration discovery and validation; `.sma/` JSON I/O	112
`src/report.mjs`	Self-contained HTML report	120
Production code total		**1,609**
`test/*.test.mjs`	8 test files, 51 tests	543

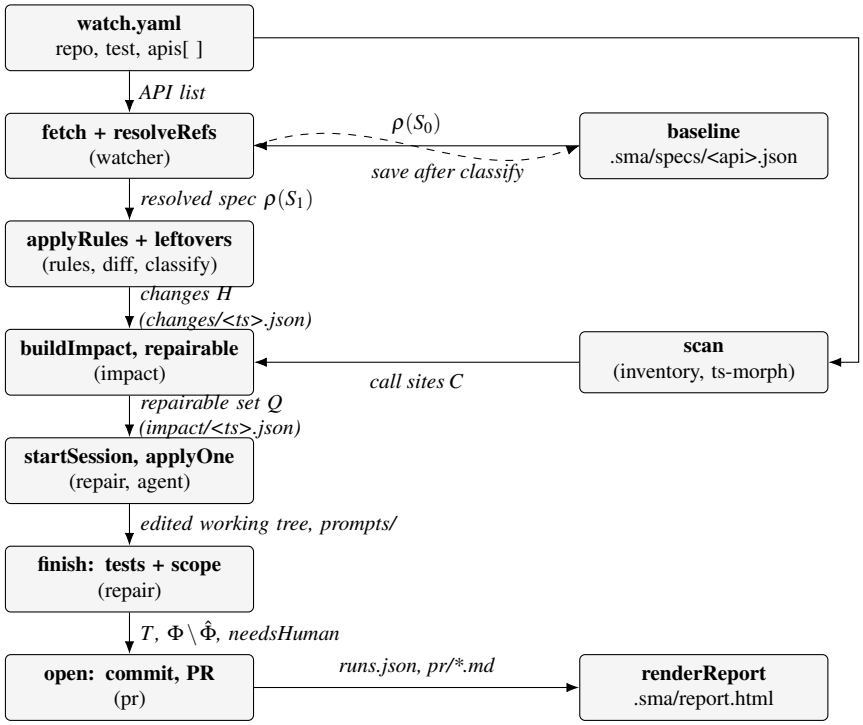


Figure 3.2: Data Flow Through One Run of `sma run --fix`

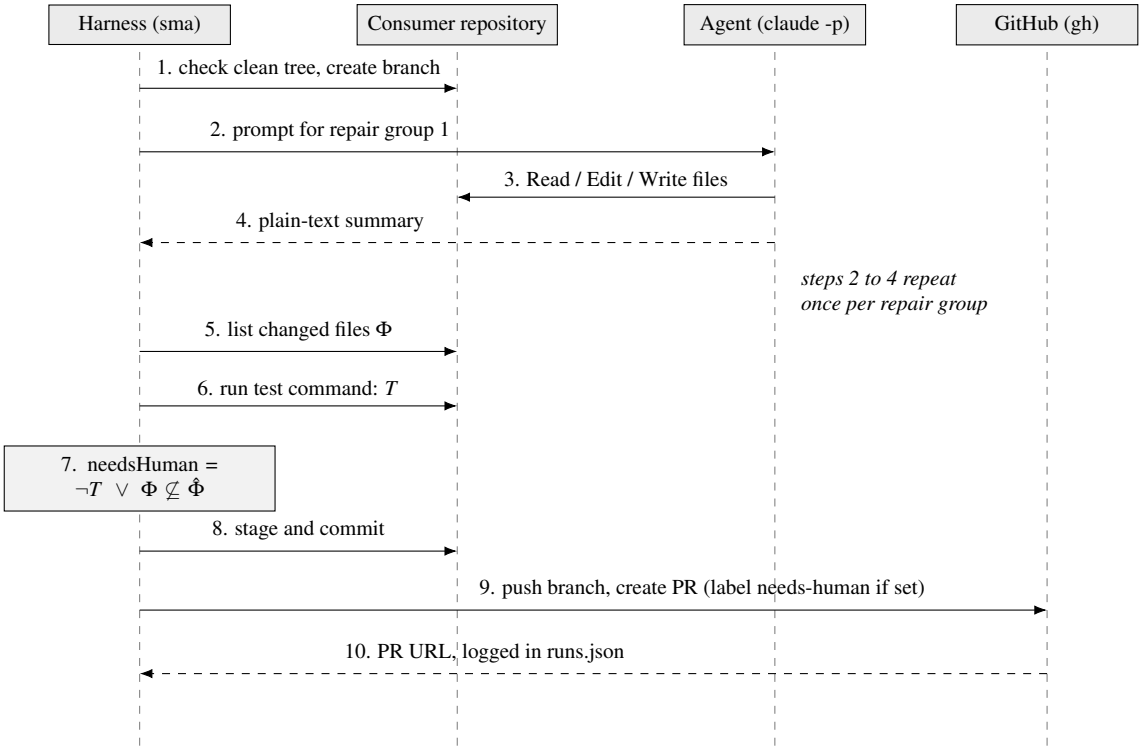


Figure 3.3: Sequence of Interactions During Repair and Verification of One API

3.4 Mathematical Model of the System

This section defines each stage formally. The definitions serve three purposes: they make the behavior precise enough to test, they expose the assumptions behind each heuristic, and they allow the computational cost to be derived analytically and then checked against measurements in Chapter 4.

3.4.1 Specifications as trees

A JSON value is either a scalar (string, number, boolean, null), an *object* (a finite map from string keys to values) or an *array* (a finite sequence of values). An OpenAPI specification S is a JSON value whose root object contains, among others, the keys `paths` and `components`. Let $|S|$ denote the number of nodes in the tree of S. A *JSON pointer* $\pi = /k_1/k_2/\ldots/k_m$ addresses the node reached by following keys $k_1,\ldots,k_m$ from the root, with `/` and `~` in a key escaped as `~1` and `~0` [37].

3.4.2 Reference resolution

Let $\mathrm{ref}(\pi)$ denote a node of the form $\{$`$ref`: `#`$\pi\}$. Reference resolution ρ replaces every such node by the node at π, recursively, while tracking the set V of references already being ex-

panded on the current path:

$$\rho_V(x) = \begin{cases} \{\texttt{\$ref}\colon \pi,\ \texttt{\$circular}\colon \text{true}\} & \text{if } x = \text{ref}(\pi),\ \pi \in V \\ \rho_{V \cup \{\pi\}}(S[\pi]) & \text{if } x = \text{ref}(\pi),\ \pi \notin V,\ S[\pi] \text{ defined} \\ \{k \mapsto \rho_V(x[k])\} & \text{if } x \text{ is an object} \\ [\rho_V(x_1), \ldots, \rho_V(x_n)] & \text{if } x \text{ is an array} \\ x & \text{otherwise} \end{cases} \tag{3.1}$$

and $\rho(S) = \rho_\emptyset(S)$. The cycle guard guarantees termination: each nested expansion adds one element to V, and V is bounded by the number of distinct references. The price of inlining is size. If a schema of size s is referenced r times, it contributes about $r \cdot s$ nodes to $\rho(S)$; with nested references the growth multiplies along each path of the reference graph. Chapter 4 measures an inflation factor of about 19.5 on OpenAI's 2026 specification (3.15 MB raw, 61.4 MB inlined).

3.4.3 Structural difference

The structural diff $\Delta(a, b, \pi)$ of two JSON values at pointer π is a set of deltas, each $(\text{op}, \pi', \text{value})$ with $\text{op} \in \{\text{add}, \text{remove}, \text{change}\}$:

$$\Delta(a, b, \pi) = \begin{cases} \emptyset & \text{if } a = b \\ \{(\text{change}, \pi, (a, b))\} & \text{if } a \text{ or } b \text{ is a scalar, or their kinds differ} \\ \Delta^- \cup \Delta^+ \cup \Delta^= & \text{if } a, b \text{ are both objects or both arrays} \end{cases} \tag{3.2}$$

where

$$\Delta^- = \bigcup_{k \in K_a \setminus K_b} \{(\text{remove}, \pi/k, a_k)\}, \quad \Delta^+ = \bigcup_{k \in K_b \setminus K_a} \{(\text{add}, \pi/k, b_k)\}, \quad \Delta^= = \bigcup_{k \in K_a \cap K_b} \Delta(a_k, b_k, \pi/k). \tag{3.3}$$

For objects, K_a is the key set. For arrays, the key of each element v is $\kappa(v) = v.\text{name}$ if v is an object with a `name`, and the serialization of v otherwise; $K_a = \{\kappa(v) : v \in a\}$. Keying array elements by identity instead of by index makes the diff invariant under reordering, which matters because OpenAPI arrays such as `parameters`, `enum` and `required` are sets in practice:

$$\Delta(a, \text{perm}(a)) = \emptyset \quad \text{for any permutation of a set-like array.} \tag{3.4}$$

A pointer π is *covered by* a set of prefixes Π if $\exists p \in \Pi : \pi = p \vee \pi$ starts with $p/$.

3.4.4 Normalized operation model

Let $M = \{\text{get}, \text{put}, \text{post}, \text{delete}, \text{patch}, \text{head}, \text{options}, \text{trace}\}$. For a resolved specification, $P(S)$ is the set of path keys and the operation set is

$$O(S) = \{(p,m) : p \in P(S),\ m \in M,\ S.\text{paths}[p][m] \text{ defined}\}. \tag{3.5}$$

For an operation $o = (p,m)$ the *field map* $F_S(o)$ merges path-level parameters, operation-level parameters and the properties of the JSON request body (after unwrapping arrays) into one map

$$F_S(o) : (\ell, \nu) \mapsto (\tau, q, E), \tag{3.6}$$

keyed by location $\ell \in \{\text{query}, \text{path}, \text{header}, \text{cookie}, \text{body}\}$ and name ν, with type τ, required flag q and enum set E. The *response map* $R_S(o)$ maps each property name of the lowest-numbered 2xx response's JSON schema (arrays unwrapped) to its type and enum. These two normalizers reduce every parameter, body and response rule to comparisons of finite maps.

3.4.5 Rules as predicates

Write $F_0 = F_{S_0}(o)$, $F_1 = F_{S_1}(o)$, $A = \text{dom}\, F_1 \setminus \text{dom}\, F_0$ (added fields) and $D = \text{dom}\, F_0 \setminus \text{dom}\, F_1$ (removed fields). Each rule $r \in \mathscr{R}$ is a predicate that, when true, produces a hit $h = (r, p, m, \text{detail}, \Pi_h)$ where Π_h is the set of pointers the hit explains. Table 3.4 gives the predicate of every rule together with its severity $\sigma(r)$.

The *rename rule* deserves comment. It is structural rather than lexical: a rename is inferred when exactly one field leaves and exactly one enters at the same location with the same type. This detects `limit` → `pageSize`, two unrelated names that a string-similarity heuristic would miss. Its known limitation follows directly from the predicate: two simultaneous renames on one operation give $|D| = |A| = 2$, so they are reported as two removals and two additions. Both readings are breaking, so the severity is preserved even when the label is not; Chapter 4 quantifies this effect.

3.4.6 Classification and the confidence gate

Let $H = \bigcup_{r \in \mathscr{R}} r(S_0, S_1)$ be the set of rule hits. The *leftover* deltas are the structural differences that no rule explains and that are not known noise:

$$L = \{\delta \in \Delta(\rho(S_0), \rho(S_1)) : \delta.\pi \notin \mathscr{I},\ \neg\text{covered}(\delta.\pi, \textstyle\bigcup_{h \in H} \Pi_h)\}, \tag{3.7}$$

where $\mathscr{I}$ is the ignore set: `/info`, `/servers`, top-level `x-*` vendor extensions, `/components/schemas` (already inlined into every operation, so counting it again would double-report), and

Table 3.4: Formal Definition of the 17 Classification Rules

Rule r	**Predicate (for path** p**, operation** $o=(p,m)$**)**	$\sigma(r)$
`PATH_REMOVED`	$p \in P(S_0) \setminus P(S_1)$	B
`METHOD_REMOVED`	$o \in O(S_0) \setminus O(S_1)$, $p \in P(S_1)$	B
`PARAM_RENAMED`	$\lvert D\rvert = \lvert A\rvert = 1$, $D = \{(\ell,v)\}$, $A = \{(\ell',v')\}$, $\ell = \ell'$, $\tau_0 = \tau_1$	B
`PARAM_REMOVED`	$(\ell,v) \in D$ and not a rename	B
`REQUIRED_PARAM_ADDED`	$(\ell,v) \in A$, $q_1 =$ true, not a rename	B
`OPTIONAL_PARAM_ADDED`	$(\ell,v) \in A$, $q_1 =$ false, not a rename	S
`PARAM_TYPE_CHANGED`	$(\ell,v) \in \operatorname{dom} F_0 \cap \operatorname{dom} F_1$, $\tau_0 \neq \tau_1$	B
`PARAM_MADE_REQUIRED`	common field with $q_0 =$ false, $q_1 =$ true	B
`ENUM_VALUE_REMOVED`	common field or response field with $E_0 \setminus E_1 \neq \emptyset$	B
`ENUM_VALUE_ADDED`	common field or response field with $E_1 \setminus E_0 \neq \emptyset$	S
`RESPONSE_FIELD_REMOVED`	$f \in \operatorname{dom} R_0 \setminus \operatorname{dom} R_1$	B
`RESPONSE_FIELD_TYPE_CHANGED`	$f \in \operatorname{dom} R_0 \cap \operatorname{dom} R_1$, $\tau_0(f) \neq \tau_1(f)$	B
`RESPONSE_FIELD_ADDED`	$f \in \operatorname{dom} R_1 \setminus \operatorname{dom} R_0$	S
`PATH_DEPRECATED`	$o \in O(S_0) \cap O(S_1)$, deprecated in S_1 but not in S_0	W
`PATH_ADDED`	$p \in P(S_1) \setminus P(S_0)$	S
`METHOD_ADDED`	$o \in O(S_1) \setminus O(S_0)$, $p \in P(S_0)$	S
`AUTH_SCHEME_CHANGED`	scheme n with $n \in \operatorname{dom} \Sigma_0$ and $\Sigma_0(n) \neq \Sigma_1(n)$ (removal or modification; additions excluded)	B

B = breaking, W = warning, S = safe; Σ = map of security schemes. 11 rules are breaking, 1 is a warning and 5 are safe.

any segment named `description`, `summary`, `example`, `examples`, `title`, `operationId` or `tags`.

If $L = \emptyset$, classification is complete and no LLM call is made. Otherwise, if an API key is present and `--no-llm` is not set, the deltas in L are sent in one request and the model returns, for each pointer, a verdict $(\pi, s, \text{reason}, c)$ with claimed severity s and confidence $c \in [0,1]$, constrained by a JSON schema. The effective severity is

$$\sigma_{\text{eff}}(v) = \begin{cases} s & c \geq \theta \\ \text{unknown} & c < \theta \end{cases}, \qquad \theta = 0.7. \tag{3.8}$$

Without a key, every $\delta \in L$ becomes an `UNCLASSIFIED` finding with severity unknown. In both cases nothing is dropped and nothing is invented: a finding of unknown severity is reported but, as shown next, can never reach the repair agent.

3.4.7 Call sites and path shapes

A call site is a tuple $c = (\text{api}, \text{kind}, m, p, \text{file}, \text{line})$ with kind $\in \{\text{http}, \text{caller}, \text{sdk}\}$. The *shape* function maps a path template to a canonical form by replacing every variable segment with `{}`:

$$\text{shape}(\texttt{/pets/\{id\}}) = \text{shape}(\texttt{/pets/\$\{petId\}}) = \texttt{/pets/\{\}}. \tag{3.9}$$

For a request whose URL argument is a template literal, the scanner replaces each `${...}` substitution with `{}`, strips the configured base URL (or a leading `{}` that stands for an interpolated origin), and drops the query string and fragment, so the code's URL and the specification's path template meet in the same shape space. A URL that is not a literal, template or object with a literal `url` is placed in a separate *unresolved* list instead of being guessed.

The one-hop caller set is defined through the enclosing function $\mathrm{fn}(c)$ of each HTTP site:

$$C_{\text{caller}} = \{(\text{api}, \text{caller}, m_c, p_c, f, \ell) : c \in C_{\text{http}},\ (f,\ell) \in \text{refs}(\text{fn}(c)) \setminus \text{imports}\}, \tag{3.10}$$

where each caller inherits the method and path of the request it reaches.

3.4.8 The impact join

For a hit h and a call site c the match predicate is

$$\text{match}(h,c) \iff c.\text{api} = h.\text{api} \ \wedge\ \mu(h,c), \tag{3.11}$$

$$\mu(h,c) = \begin{cases} h.\text{rule} \in W & \text{if } h.p = \text{null} \\ \text{shape}(c.p) = \text{shape}(h.p) \ \wedge\ (h.m = \text{null} \vee c.m = h.m) & \text{if } c.\text{kind} \in \{\text{http}, \text{caller}\} \\ \text{lower}(\text{opId}(h)) \sqsubseteq \text{lower}(c.\text{symbol}) & \text{if } c.\text{kind} = \text{sdk} \end{cases} \tag{3.12}$$

where $W = \{\texttt{AUTH_SCHEME_CHANGED}\}$ is the set of whole-specification rules and $\sqsubseteq$ denotes substring. The impact set is $I(h) = \{c \in C : \text{match}(h,c)\}$, and h is *affected* iff $I(h) \neq \emptyset$. The restriction to W for path-less findings is a guard against a specific false-positive class: an `UNCLASSIFIED` delta also has no path, and letting "no path" mean "everywhere" made every such delta claim every call site.

The repairable set keeps only hits that are breaking and affected, grouped by the key $g(h) = (\text{api}, r, \text{subject})$, where the subject is the first defined of the field, parameter name, renamed-from name, scheme name or path:

$$Q = \left\{ \left(g,\ \textstyle\bigcup_{h:\, g(h)=g} I(h),\ \{(h.m, h.p)\}\right) : \sigma(h) = \text{breaking} \wedge I(h) \neq \emptyset \right\}. \tag{3.13}$$

One field removed from a shared schema surfaces as one hit per operation; the grouping collapses them into a single repair carrying all endpoints and the union of their call sites.

3.4.9 Verification predicate and exit codes

Let $\hat{\Phi} = \{c.\text{file} : c \in \bigcup_{q \in Q} I(q)\}$ be the files the analysis predicts, Φ the files actually changed in the working tree after all agent runs (tracked modifications, deletions, renames by new name

and newly created files), and T the outcome of the test command. Then

$$\text{applied} = (\Phi \neq \emptyset), \qquad \text{needsHuman} = \neg T \ \vee \ (\Phi \setminus \hat{\Phi} \neq \emptyset). \tag{3.14}$$

If nothing was changed the branch is deleted. The process exit code is

$$\text{exit} = \begin{cases} 2 & \text{the tool failed (configuration, network, prerequisite)} \\ 1 & \exists h \in H : \sigma(h) = \text{breaking (after } \texttt{impact}\text{: } Q \neq \emptyset) \\ 0 & \text{otherwise.} \end{cases} \tag{3.15}$$

After `fix` the exit code stays 1 even when a PR was opened, because an unmerged PR is still an unmerged break.

3.4.10 Analytical complexity

Let n_0, n_1 be the sizes of the two resolved specifications, $|O|$ the number of operations, $\bar{f}$ the mean number of fields and response properties per operation, N the number of AST nodes in the consumer repository, $|C|$ the number of call sites and $|H|$ the number of hits. Table 3.5 derives the cost of each stage.

Table 3.5: Analytical Time Complexity of Each Stage

Stage	Complexity	Reasoning
Parse (YAML/JSON)	$O(\text{bytes})$	Single pass of the parser
Reference resolution ρ	$O(\lvert\rho(S)\rvert)$	Every output node is produced once; output may exceed input by the inflation factor
Structural diff Δ	$O(n_0 + n_1)$	Each node visited once; array keying uses hash maps, $O(1)$ expected per element (serialization of unnamed elements adds their size)
Rule engine	$O(\lvert O\rvert \cdot \bar{f})$	Per operation, constant work per field and response property; no pairwise search
Leftover computation	$O((n_0 + n_1) \cdot \lvert H\rvert)$ worst case	Each delta is tested against the explained pointer prefixes; $\lvert H\rvert$ is small in practice
AST scan	$O(N)$ + reference queries	One traversal per file; the language service answers one reference query per HTTP site
Impact join	$O(\lvert H\rvert \cdot \lvert C\rvert)$	Nested match over hits and sites; both are small (tens to hundreds)
Repair	dominated by agent and test time	External processes; bounded by 15 min (agent) and 10 min (tests) timeouts

The model predicts that detection, the deterministic stages up to and including the impact join, should scale *linearly* in specification size for a fixed number of changes. Chapter 4 tests this prediction with a least-squares fit of $t(n) = \beta_0 + \beta_1 n$ to measured times.

3.5 Watcher: Fetching and Normalizing Specifications

The watcher (`src/watcher.mjs`) implements requirement FR2 to FR4. A location that matches `^https?://` is fetched over HTTP; anything else is read as a file path resolved against the directory of `watch.yaml`, not against the current directory, so that `--config ../watch.yaml` behaves the same from anywhere. Both go through one YAML parser, because YAML is a superset of JSON.

Robust fetching. Listing 3.1 shows the fetch loop. Node's `fetch` reports every network problem as the same two words, "fetch failed", so the loop unwraps the underlying error code and names the URL; for a refused connection to localhost it adds a hint to start the demo server. A hung server fails after 60 seconds instead of hanging a CI job. Responses with status 429 or 5xx are retried up to three attempts, waiting for the server's `Retry-After` value when given, otherwise 2^{attempt} seconds, capped at 30 seconds.

Listing 3.1: Fetch Loop with Timeout, Retry and Redacted Errors (src/watcher.mjs)

```
async function fetchOverHttp(url, headers) {
  const shown = redact(url);
  for (let attempt = 1; ; attempt++) {
    let res;
    try {
      res = await fetch(url, { headers, signal: AbortSignal.timeout(TIMEOUT_MS) });
    } catch (err) {
      const code = err.name === 'TimeoutError' ? `no response after ${TIMEOUT_MS / 1000}s` : (err.cause?.code ?? err.message);
      throw new Error(`could not fetch ${shown}: ${code}`);
    }
    if (res.ok) return res.text();
    const retryable = res.status === 429 || res.status >= 500;
    if (!retryable || attempt === ATTEMPTS) throw new Error(`could not fetch ${shown}: HTTP ${res.status}`);
    const wait = Math.min(Number(res.headers.get('retry-after')) || 2 ** attempt, 30);
    await new Promise((r) => setTimeout(r, wait * 1000));
  }
}
```

Credentials. Private specifications are supported through `headers:` in the configuration. A value such as `Bearer ${SPEC_TOKEN}` is expanded from the environment at fetch time, so the token is never written into `watch.yaml` or `.sma/`; an unset variable is an error rather than an empty header. Any query parameter whose name contains `key`, `token`, `secret`, `sig`, `auth` or `password` is masked as `***` wherever a URL is printed, in errors, PR bodies and the report.

Reference inlining implements ρ from Section 3.4 (Listing F.1 in Appendix F). Only local references beginning with `#/` are followed; remote references are left untouched.

Snapshots and the ordering rule. The resolved specification is saved as `.sma/specs/`

<name>.json. The order of operations in check enforces NFR2: on the first run the baseline is written immediately (nothing can be lost), but on later runs the new snapshot is saved only *after* classify() has returned. If the snapshot advanced first, a network failure during LLM classification would record the new specification as already seen, and the change would never be reported again. The baseline is committed to Git rather than cached, because a cache eviction would be silent: loadSnapshot would return nothing, the run would take the first-run branch, print "baseline captured" and exit 0, reporting all clear indefinitely while detecting nothing.

3.6 Structural Diff and Rule Engine

The diff (src/diff.mjs, 41 lines) implements Δ exactly as defined, including set-like comparison of arrays (Listing 3.2). It is generic: it knows nothing about OpenAPI, which is why it can also compute the leftovers that no rule explains.

Listing 3.2: Generic Structural Diff Emitting RFC 6901 Pointers (src/diff.mjs)

```
export function diff(before, after, pointer = '') {
  if (before === after) return [];
  if (isObj(before) && isObj(after)) {
    const keys = new Set([...Object.keys(before), ...Object.keys(after)]);
    return [...keys].flatMap((k) => {
      const p = `${pointer}/${esc(k)}`;
      if (!(k in after)) return [{ op: 'remove', pointer: p, before: before[k] }];
      if (!(k in before)) return [{ op: 'add', pointer: p, after: after[k] }];
      return diff(before[k], after[k], p);
    });
  }
  if (Array.isArray(before) && Array.isArray(after)) {
    // Spec arrays (parameters, enums, required) are sets: compare by value, not
    index.
    const key = (v) => (isObj(v) && v.name ? v.name : JSON.stringify(v));
    const b = new Map(before.map((v) => [key(v), v]));
    const a = new Map(after.map((v) => [key(v), v]));
    const out = [];
    for (const [k, v] of b) {
      const p = `${pointer}/${esc(k)}`;
      if (!a.has(k)) out.push({ op: 'remove', pointer: p, before: v });
      else out.push(...diff(v, a.get(k), p));
    }
    for (const [k, v] of a) if (!b.has(k)) out.push({ op: 'add', pointer: `${
    pointer}/${esc(k)}`, after: v });
    return out;
  }
  return [{ op: 'change', pointer, before, after }];
}
```

The rule engine (src/rules.mjs) is the core of the project. applyRules(before, after) walks the old paths: a missing path yields PATH_REMOVED; for each method present before, a miss-

ing method yields `METHOD_REMOVED`; otherwise it checks deprecation and compares fields and responses through the normalizers F and R. A second walk over the new paths finds additions, and `compareAuth` compares security schemes. Listing 3.3 shows the structural rename rule. Every hit records the JSON pointers of the operation it explains (*e.g.*, `/paths/~1pets/get`); those pointers are what the classifier subtracts when computing L. Appendix A specifies every rule with an example.

Listing 3.3: Structural Rename Inference in compareFields (src/rules.mjs)

```
const removed = [...oldF.keys()].filter((k) => !newF.has(k));
const added = [...newF.keys()].filter((k) => !oldF.has(k));
// A rename is "exactly one out, exactly one in, same location and same type".
// Structural, not lexical: `limit` and `pageSize` share no letters.
if (removed.length === 1 && added.length === 1) {
  const from = oldF.get(removed[0]);
  const to = newF.get(added[0]);
  if (from.in === to.in && from.type === to.type) {
    at2('PARAM_RENAMED', { from: from.name, to: to.name, in: from.in },
        `${from.in} parameter '${from.name}' was renamed to '${to.name}'`);
    return compareCommonFields(oldF, newF, at2);
  }
}
```

Figure 3.4 shows how the rule engine and the LLM fallback combine into the classifier.

3.7 Classifier and LLM Fallback

`src/classify.mjs` implements Figure 3.4. The prompt sent to the model states that the deterministic rule engine has already classified everything it recognizes, defines the three severities (*breaking*: existing correct client code can now fail or silently misbehave; *warning*: still works but signals removal; *safe*: purely additive or cosmetic), and explicitly asks for a low confidence rather than a guessed "breaking" when the model is unsure. The response is constrained by the JSON schema in Listing 3.4, so it never needs free-text parsing.

The confidence floor $\theta = 0.7$ separates "worth telling a human about" from "worth letting a machine edit code over". A model that is 0.55 sure a change breaks something is useful in a report and dangerous in a codemod. Below the floor the finding is kept with severity *unknown* and the suffix "(low confidence c, not auto-patched)"; because `repairable()` accepts only `severity === 'breaking'`, such a finding can never reach the agent. For the demo's v1 to v2 change the rules explain the entire diff, and a unit test asserts that $L = \emptyset$, so the default path costs nothing.

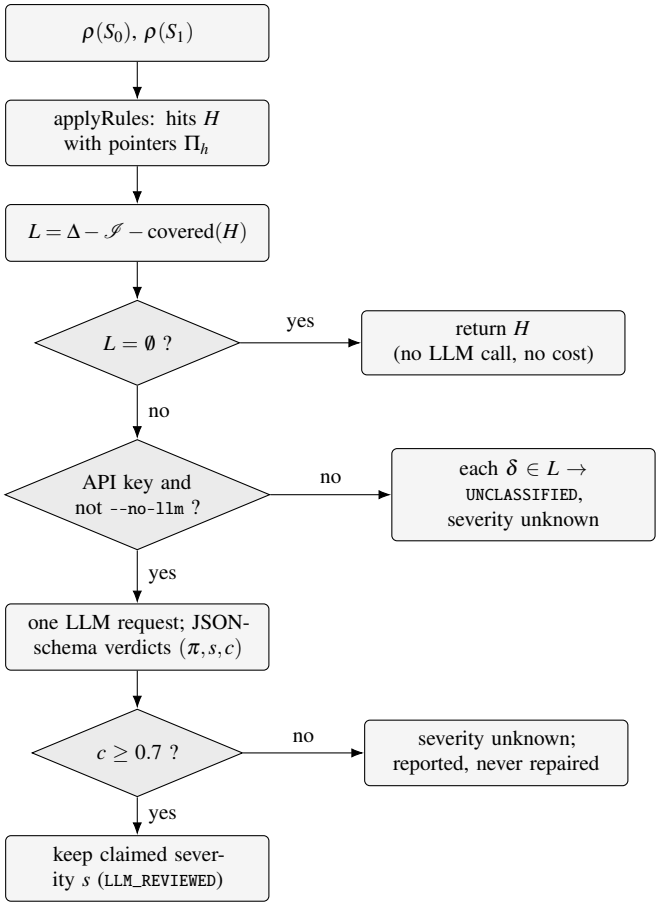


Figure 3.4: Flowchart of the Classifier: Deterministic Rules First, LLM Only for Leftovers

Listing 3.4: JSON Schema Constraining the LLM Verdicts (src/classify.mjs)

```
const VERDICT_SCHEMA = {
  type: 'object',
  properties: { verdicts: { type: 'array', items: {
    type: 'object',
    properties: {
      pointer:    { type: 'string' },
      severity:   { type: 'string', enum: ['breaking', 'warning', 'safe'] },
      reason:     { type: 'string' },
      confidence: { type: 'number' },
    },
    required: ['pointer', 'severity', 'reason', 'confidence'],
    additionalProperties: false } } },
  required: ['verdicts'], additionalProperties: false,
};
```

3.8 Inventory: AST Call-Site Scanner

`src/inventory.mjs` builds a ts-morph project over every `.js`, `.jsx`, `.mjs`, `.cjs`, `.ts`, `.tsx`, `.mts` and `.cts` file in the repository, excluding `node_modules`, with `allowJs` so that JavaScript and TypeScript share one parser. Listing 3.5 summarizes the scan as pseudocode.

Listing 3.5: Algorithm of the Call-Site Scan (pseudocode)

```
for each source file F in repo (excluding node_modules):
  for each watched api A:
    for each CallExpression E in F:
      root := first segment of callee text            # fetch, axios.get, got.post, ...
      if root not in {fetch, axios, got, ky, request}: continue
      u := urlOf(first argument)                      # literal | template | {url: ...}
      if u is null: record E as UNRESOLVED; continue  # never silently dropped
      p := toPath(u, A.baseUrl)                       # strip origin/query, ${...} -> {}
      if p is null: continue                          # another host: not this API
      m := verb from callee (axios.post) | options.method | GET
      add site (A, http, m, p, file, line, snippet)
    if A.sdk is set: add every call chain rooted at an import (or `new` instance) of A.sdk
for each http site s:                                 # one hop out
  f := name of the function enclosing s
  for each reference r to f (language service), not in import/export, not already known:
    add site (s.api, caller, s.method, s.path, file(r), line(r), via f)
```

Three design decisions in the scanner are worth stating. First, **AST rather than text**: a comment such as `// fetch('/pets') later` or a log line that prints a URL is not a call, and a template literal `` `${BASE}/pets/${id}` `` is. Second, **unresolved, not dropped**: when the URL is a variable or a computed expression, the call is reported in its own block by `sma scan`, because a silently dropped call site is an undetected break. Third, **one hop, not a call graph**: the caller pass catches the case that recurs in practice, a wrapper function and the tests or helpers that call it. In the demo, four of the eight sites found are such callers in `test/api.test.mjs`; without them, a removed response field would look as if it affected only the file holding the `fetch`, while the assertions that depend on the old shape, which mention no URL and import no HTTP client, would quietly break. Deeper wrapper chains need a real call graph and are a documented limitation.

3.9 Impact Join and Repair Grouping

`src/impact.mjs` implements the match predicate and the repairable set Q of Section 3.4. The SDK branch looks up the `operationId` of the changed operation in the stored snapshot and

checks whether it appears, case-insensitively, in the SDK method chain; this works for SDKs generated from the specification and is a heuristic for hand-written ones. Changes with no matching sites are kept with `affected: false` and rendered as "not affected (no call sites in this repo)". Reporting them is how a reader can tell that the mapper is matching rather than guessing. Figure 3.5 illustrates the join and the grouping on the demo data.

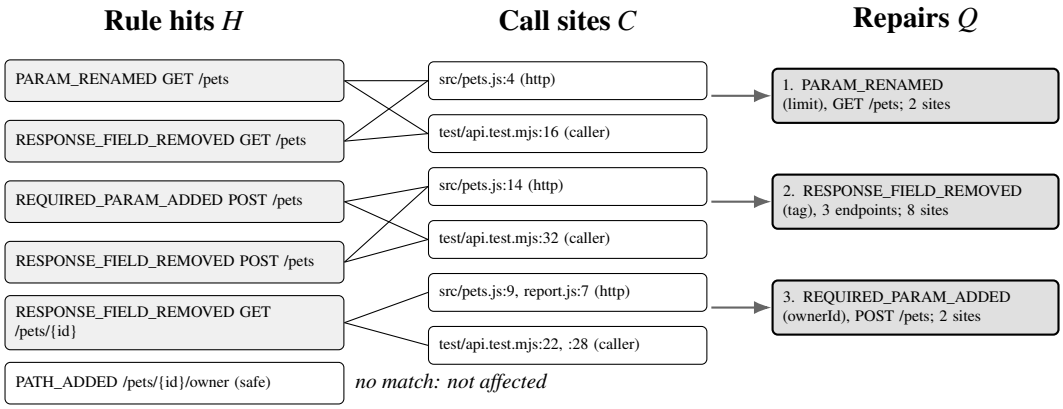


Figure 3.5: Impact Join and Grouping on the Petstore Demo: 6 Hits, 8 Sites, 3 Repairs

3.10 Repair Agent and Independent Verification

`src/repair.mjs` realizes the verification design of Section 3.4.

Session. `startSession` refuses to proceed unless the repository is a Git repository with a clean working tree, records the current branch as the base, and creates `sma/<api>-<stamp>`. One specification change is one branch: each repair group gets its own focused agent run, but all runs land on the same branch, because a pull request that fixed only part of what an upgrade broke would fail its own tests.

Prompt. `buildPrompt` gives the agent the API, rule, severity, endpoints, reason, raw detail and every affected call site as `file:line snippet`, followed by six rules: change only what is needed and touch only the listed files (and their direct callers if forced); do not refactor, rename, reformat or "improve" anything; keep the existing style; thread new information such as a newly required parameter through function signatures rather than hard-coding it, and name any invented placeholder; leave fixes for other changes from the same update alone; and do not run any commands or tests. The prompt is written to `.sma/prompts/` before the run, so what was asked is auditable afterwards. Appendix D reproduces a real prompt.

Sandbox. The agent is started as

```
claude -p --permission-mode acceptEdits --allowedTools Read,Edit,Write,Glob,Grep
```

with the prompt on standard input, the repository as working directory and a 15-minute timeout [6]. The allowed tool list contains no shell. The agent therefore cannot run the test suite, and

its report of the result can never become the evidence that the repair worked. Removing the capability removes both the temptation and the ambiguity: the agent edits, and someone else decides whether the edit was right.

Verification. `finish` performs three steps in which the agent has no part. It lists the touched files with `git status --porcelain --untracked-files=all`, so that files the agent *created* are counted (an early version used `git diff --name-only`, which lists only tracked modifications, and silently discarded a repair written as a new module). If nothing changed, the branch is deleted and the group is skipped. Otherwise it runs the configured test command (default `npm test`, 10-minute timeout) and compares the touched files with the predicted set $\hat{\Phi}$. `needsHuman` is set if the tests fail or any file lies outside the prediction; either is a signal that the model's understanding and the static analysis disagree.

3.11 Pull Request Writer

`src/pr.mjs` builds the PR body from the impacts and the agents' own summaries: one numbered section per change with its reason, whether it was detected by the rule engine or by LLM review, its call sites and the agent's quoted summary; then the diff statistics; then a *verification* block stating the test result and the number of files outside the predicted impact ("run by the harness, not the agent"); then a reviewer checklist; and, if needed, a bold **needs-human** warning not to merge without reading. The body is always written to `.sma/pr/<branch>.md` before anything is pushed. A single commit titled `fix(<api>): patch N breaking API change(s)` is created. If the repository has no Git remote (as in the local demo) the process stops there and points to the Markdown file; otherwise the branch is pushed and `gh pr create` is called [75], adding `--label needs-human` when verification failed. A missing or failing `gh` is reported with the reason and the location of the body, never as a crash. Appendix E reproduces the PR body produced in the evaluation.

3.12 State Store and Data Formats

`src/store.mjs` locates `watch.yaml` the way Git locates `.git`: the `--config` flag wins, then the `SMA_CONFIG` environment variable, then a walk upward from the current directory. Without the walk, running `sma check` from a subdirectory would silently start a new project against an empty `.sma/` and report "baseline captured" forever. The configuration is validated on load: `repo` and a non-empty `apis` list are required, every API needs `name` and `spec`, names must match `[a-zA-Z0-9._-]+` (the name becomes a file name, so `../../x` must not decide where the tool writes) and must be unique. Text files are read in whatever encoding they arrived in; Windows PowerShell's `echo x > file` writes UTF-16LE with a byte-order mark, which a plain UTF-8 read turns into null bytes.

All state is plain JSON under `.sma/` (Figure 3.6). Everything is a per-run artifact and is ignored by Git, except `.sma/specs/`, the baseline, which `sma init` deliberately leaves tracked. Listing 3.6 shows the data contracts that connect the modules.

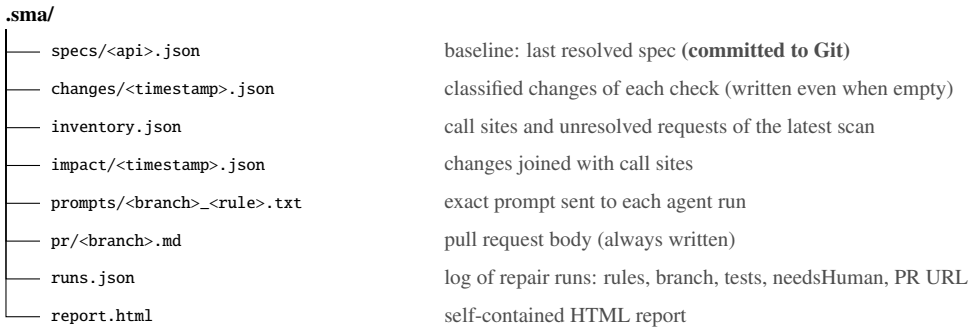


Figure 3.6: Layout of the On-Disk State Directory

Listing 3.6: Data Contracts Between Modules

```
// Call site (inventory.json)
{ api: "petstore", kind: "http" | "caller" | "sdk", method: "GET", path: "/pets/{}",
  symbol: "octokit.repos.get" /* sdk only */, via: "getPet" /* caller only */,
  file: "src/pets.js", line: 9, snippet: "fetch(`${BASE}/pets/${id}`)" }
// Change (changes/<ts>.json)
{ api: "petstore", rule: "REQUIRED_PARAM_ADDED", severity: "breaking" | "warning" |
    "safe" | "unknown",
  path: "/pets", method: "POST", detail: { name: "ownerId", in: "body", type: "
    integer" },
  reason: "new required body parameter 'ownerId'", pointers: ["/paths/~1pets/post"],
    source: "rule" | "llm" | "none" }
// Impact (impact/<ts>.json) = change + { operationId, sites: [CallSite], affected:
    boolean }
// Run (runs.json)
{ at: "2026-10-18T09:34:45.403Z", api: "petstore", rules: ["PARAM_RENAMED", ...],
  branch: "sma/petstore-20261018093325", testsPass: true, needsHuman: false, pr:
    null }
```

3.13 Command-Line Interface, Report and Degradation

3.13.1 Commands and exit codes

Table 3.6 lists the commands. `run` chains `scan`, `check`, `impact`, optionally `fix`, and always `report`. `diff` needs no configuration, baseline or repository and answers “would upgrading from this version to that one break anyone?”. The flags are `--no-llm` (rules only, fully offline and deterministic), `--json` (machine-readable `diff` output) and `--config <path>`.

Table 3.6: Commands of the sma Command-Line Interface

Command	Function
`sma init`	Write `watch.yaml` and `.gitignore` lines; if a configuration exists, capture the baseline
`sma doctor`	Report installed prerequisites and what each missing one costs (Figure 3.7)
`sma scan`	Find every call site for the watched APIs; list unresolved requests
`sma check`	Fetch specs, diff against the baseline, classify; exit 1 if any change is breaking
`sma impact`	Join the latest changes with the call sites; exit 1 if any breaking change is used
`sma fix`	Repair breaking-and-used changes: branch, agent, verify, PR
`sma run [--fix]`	All of the above, then write the HTML report
`sma diff <a> <b>`	Classify the changes between two spec files or URLs, no configuration needed
`sma report`	Render `.sma/report.html` from what is on disk (never fetches)
`sma reset`	Forget every snapshot and start over
`sma version, help`	Print the version or usage

3.13.2 Graceful degradation

Requirement NFR3 is realized as a layered design (Table 3.7); `sma doctor` prints this table for the current machine. Detection must never fail for lack of an optional component, and Figure 3.7 shows the output on the machine used for the evaluation, on which only the detection layers were available.

Table 3.7: Layers, Prerequisites and Behavior When a Prerequisite Is Missing

Layer	Needs	Behavior without it
Detect (fetch, diff, rules)	Node.js 22+	Must never fail
Impact (AST scan)	bundled	An unparsable file is reported, not fatal
Explain ambiguous deltas	`ANTHROPIC_API_KEY`	Deltas become `UNCLASSIFIED`; rules still apply
Repair	`claude` CLI, `git`	`--fix` refuses; the report is still produced
Open the PR	`gh`, `GH_TOKEN`	PR body written to `.sma/pr/` and printed

3.13.3 HTML report

`src/report.mjs` renders a single self-contained HTML page with no external assets, suitable for opening locally or attaching as a CI artifact. It shows a verdict banner, six summary tiles (APIs watched, call sites found, specification changes, breaking, breaking and used here, repair runs), the pipeline, the watched APIs, the findings sorted by severity with their call sites (callers marked "via *function*()"), and the repair runs. Every string derived from a specification or from

Terminal: sma doctor on the evaluation machine

```
$ node sma.mjs doctor
  sma 1.0.0
  ok    node 22.23.2
  ok    config .../Self-MaintainingAI/watch.yaml
  ok    git
  MISS  ANTHROPIC_API_KEY
          without it: unmodelled deltas are reported as UNCLASSIFIED instead of judged
  MISS  claude CLI
          without it: `--fix` is unavailable - sma reports what broke, you patch it
  MISS  gh CLI
          without it: the PR body is written to .sma/pr/ instead of opened
  MISS  GH_TOKEN
          without it: `gh` cannot open the PR

Detection works. Anything missing above only limits repair, never detection.
```

Figure 3.7: Output of `sma doctor` on the Evaluation Machine

code is HTML-escaped. Figure 3.8 shows the report produced after the petstore v2 release, before repair; Figure 3.9 shows the verdict and the repair log after the fix.

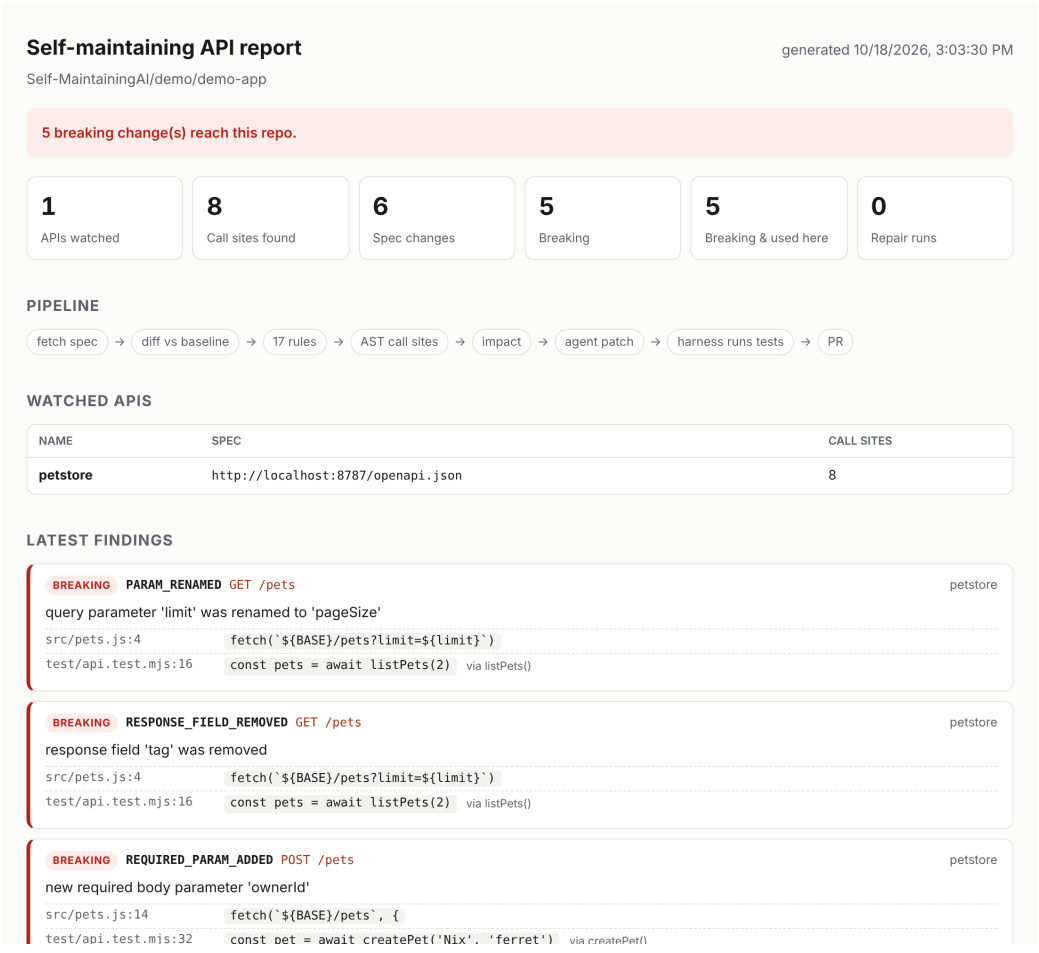


Figure 3.8: HTML Report After the Upstream v2 Release, Before Repair (Top Part)

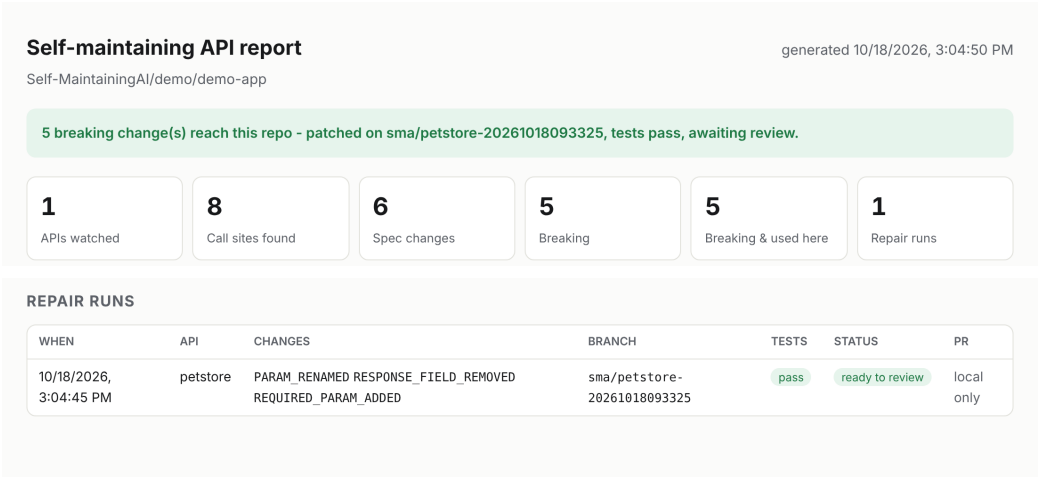


Figure 3.9: HTML Report After Repair: Verdict Banner and Repair Run Log

3.14 Demonstration Harness

3.14.1 Controlled upstream: the petstore API

`demo/petstore/server.mjs` is a small `node:http` server that serves `openapi.json` from whichever version the file `ACTIVE` names (`v1` or `v2`) and implements the corresponding data endpoints. Version 1 has three operations: `GET /pets` (query parameter `limit`), `POST /pets` (body `name`, `tag`) and `GET /pets/{id}`; the shared `Pet` schema has `id`, `name`, `tag` and `status`. Version 2 deliberately exercises four different rule classes (three breaking, one safe): `limit` is renamed `pageSize`, `ownerId` becomes a required body field, `tag` is removed from the `Pet` schema (and therefore from three responses), and a new `GET /pets/{id}/owner` endpoint is added (Table 3.8).

Table 3.8: Designed Differences Between Petstore v1 and v2

Edit in v2	Operation(s)	Expected rule	Severity
`limit` renamed `pageSize`	GET /pets	PARAM_RENAMED	breaking
`ownerId` required in body	POST /pets	REQUIRED_PARAM_ADDED	breaking
`tag` removed from `Pet`	GET /pets, POST /pets, GET /pets/{id}	RESPONSE_FIELD_REMOVED (3 hits)	breaking
New owner endpoint	GET /pets/{id}/owner	PATH_ADDED	safe

`demo/app/` is a small consumer: `src/pets.js` wraps the three endpoints, `src/report.js` calls `GET /pets/{id}` a second time, and `test/api.test.mjs` runs four tests against the live server. The tests pass on v1 and must fail on v2; that failure is the proof that a later repair did something. `demo/setup.mjs` copies `demo/app` to `demo/demo-app` and initializes it as its own Git repository, because `sma fix` branches whatever `repo:` points at and that must not be the tool's own checkout.

3.14.2 Real upstream: the OpenAI replay

`demo/real-world/run.mjs` replays a real change: OpenAI's published specification at commit `a27cad2f` is used as the baseline and commit `d9c3021a` of 19 June 2023 ("remove disabled/deprecated endpoints") as the new version. Both URLs are pinned to commit hashes, so the result never drifts. The consumer, `demo/real-world/app/`, is a small bot written against the early-2023 API: `openai.js` calls `/engines`, `/answers`, `/completions` and `/models`, and `bot.js` calls the first two through helper functions.

3.15 Automation and Continuous Integration

Four GitHub Actions workflows support the project (Figure 3.10). `ci.yml` runs the test suite on every push to, and pull request against, the main branch, across a matrix of Ubuntu and Windows with Node.js 22 and 24. `sma.yml` is the product workflow: every day at 06:00 UTC (and on manual dispatch) it first runs the tool's own tests ("if the detector is broken, its verdicts mean nothing"), then runs `sma run --fix` with `ANTHROPIC_API_KEY` and `GITHUB_TOKEN` as secrets, uploads the HTML report as an artifact, commits the new baseline back to the repository (even, and especially, when a breaking change was found, so the same change is not reported again the next day), and keeps the real exit code so that "an API broke you" (1) is distinguishable from "sma failed" (2). Because the baseline is committed, no cache step is needed and nothing can be evicted. Without the secrets, the same workflow still runs and simply fails the job when an API breaks, instead of writing the fix. `release.yml` publishes to npm with provenance when a version tag is pushed, after running the tests ("a tag is not a promise; never publish a red build"), and `pages.yml` deploys the project web page.

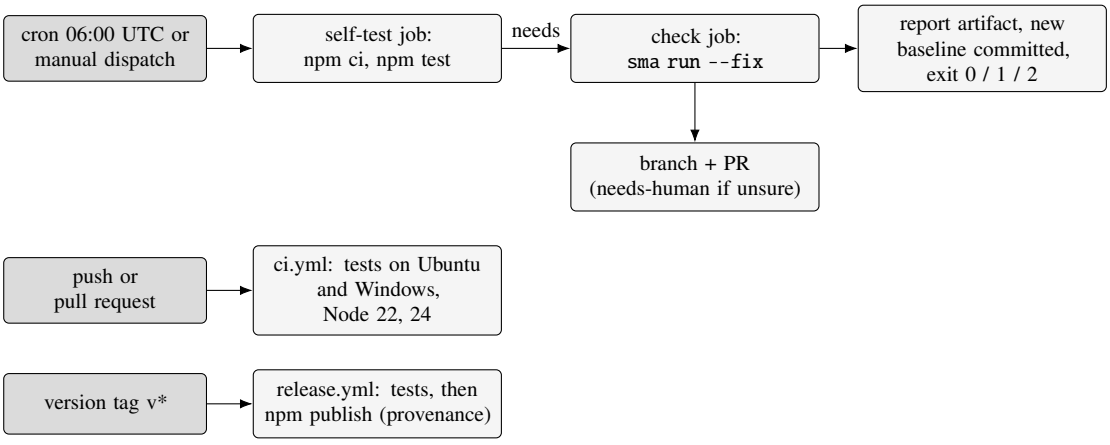


Figure 3.10: Continuous Integration and Scheduled Workflows

3.16 Security Considerations

- **No automatic merge.** The pull request is the deliverable; a human always decides.

- **Least privilege for the agent.** File tools only; no shell, no network tools, no ability to run tests or install packages. Edits outside the predicted files are flagged.
- **Secrets.** Header values are expanded from environment variables at run time and never written to disk; credential-like URL parameters are masked in every output.
- **Path safety.** API names are validated before being used as file names, preventing writes outside `.sma/specs/`.
- **Output escaping.** The HTML report escapes every specification- or code-derived string, so a malicious specification cannot inject script into the report (covered by a unit test).
- **Prompt injection surface.** Specification text reaches the LLM classifier only as JSON-pointer deltas, and the classifier's output is a schema-constrained severity that is advisory below the confidence floor; the agent's prompt contains code snippets from the consumer's own repository. Any residual risk is bounded by the agent's sandbox and the human review.

3.17 Hardening Found by End-to-End Use

Running the tool end to end on the demo and on OpenAI's real specification after version 1.0.0 exposed defects that no unit test had caught. Each was fixed and, where possible, a regression test was added (Table 3.9).

Table 3.9: Defects Found by End-to-End Use and Their Fixes

Defect	Fix
Baseline advanced before classification; a failure lost the change forever	Snapshot saved only after `classify()` returns
Files created by the agent were neither counted nor scope-checked	`git status --porcelain --untracked-files=all`
Every modified file after the first was parsed as `M path`, flagging correct repairs as needs-human	Porcelain parsing tolerant of the trimmed first line
An *added* auth scheme was breaking and claimed every call site	Additions skipped; removal and modification still breaking
Top-level `x-*` vendor extensions were 4 of 9 findings on OpenAI's spec	Added to the ignore set
Relative local `spec:` resolved against the current directory	Resolved against the configuration file
Missing `gh` reported as "failed: undefined"	Clear message naming the cause and the PR body location

3.18 Testing Strategy

The deterministic core is covered by 51 unit tests in 8 files, run with the Node.js built-in test runner (Table 3.10; the full list is in Appendix C). Two tests are treated as load-bearing: `diff.test.mjs` asserts the *exact* set of rule hits for petstore v1 to v2, checking both that nothing is missed and that nothing is invented, and `inventory.test.mjs` asserts the exact call sites found in a fixture repository, including one deliberately unresolvable `fetch(path)`.

Table 3.10: Unit Test Files and What They Assert

File	Tests	What is asserted
`diff.test.mjs`	10	Exact hit set v1 to v2; no false positives on identical specs; reorder invariance; each rule class
`auth.test.mjs`	5	Added scheme not breaking; removed or changed scheme breaking
`impact.test.mjs`	8	Join by shape and method; other APIs ignored; not-affected kept; grouping and de-duplication
`inventory.test.mjs`	7	Exact sites; template literals; method from options; other hosts ignored; unresolved reported; one-hop callers
`repair.test.mjs`	6	Created, modified, renamed and nested files counted correctly
`report.test.mjs`	3	HTML escaping; verdict text
`store.test.mjs`	5	Configuration discovery; path-traversal and duplicate names rejected
`watcher.test.mjs`	7	Local path resolution; credential masking; header expansion; retry on 503
Total	**51**	

3.19 Summary

This chapter developed sma from requirements to implementation. The system was first modeled mathematically, as reference resolution, a set-valued structural diff, 17 predicates over normalized operations, a confidence gate, a relational impact join and a verification predicate, and its computational cost was derived analytically, predicting linear scaling of detection. Each module was then shown to implement its part of the model, with deliberate, documented limits (one hop of callers, a structural rename heuristic, operationId matching for SDKs). The next chapter measures how well this design performs.

CHAPTER 4

PERFORMANCE ANALYSIS

This chapter evaluates sma by three methods, as the report guidelines require: an **experimental** method (Section 4.2), in which the real tool is run on a controlled API upgrade and on a real provider change; a **statistical** method (Section 4.3), in which large numbers of labeled trials are scored against ground truth and against baseline methods; and an **analytical** (mathematical) method (Section 4.4), in which the complexity model of Chapter 3 is compared with measured times. Section 4.5 compares the results of the methods with each other and Section 4.6 justifies the differences and errors.

4.1 Evaluation Design

4.1.1 Research questions

RQ1. Correctness of detection. Does the classifier report exactly the changes that occurred, with the right severity, and nothing for cosmetic edits?

RQ2. Comparison with a baseline classifier. How much better is the rule engine than a generic structural diff that treats every removal or modification as breaking?

RQ3. Precision of impact analysis. Does the AST scanner find the call sites that matter, and avoid false ones, compared with text search?

RQ4. End-to-end effectiveness. On a real breaking upgrade, does the full loop (detect, map, repair, verify) return the consumer to a passing state?

RQ5. Behavior on real specifications. What does the tool report across the real history of a fast-moving public API, and does that history expose defects that synthetic tests miss?

RQ6. Scalability. How does detection time grow with specification size, and does it match the analytical model?

4.1.2 Methods and the questions they answer

Table 4.1 maps each evaluation method to the research questions and to the stages of the pipeline it exercises. Four of the six research questions (RQ1, RQ3, RQ5, RQ6) are answered by at least two methods; RQ2 is answered statistically and RQ4 experimentally, and Section 4.5 sets both against the analytical model or the literature.

Table 4.1: Evaluation Methods, Research Questions and Pipeline Stages Covered

Method	Experiment	RQs	Stages
Experimental	Unit test suite (51 tests)	RQ1, RQ3	all deterministic stages
Experimental	Petstore v1 to v2 end to end, including a real agent repair	RQ1, RQ3, RQ4	all stages
Experimental	OpenAI June 2023 replay	RQ1, RQ5	detect, scan, impact
Statistical	6,700 single-change mutation trials versus naive diff	RQ1, RQ2	detect
Statistical	2,700 compound-release trials (k = 1 to 8)	RQ1	detect
Statistical	Labeled 467-statement call-site corpus versus two grep baselines	RQ3	scan
Statistical	478-commit longitudinal study of OpenAI's specification	RQ5, RQ6	detect
Analytical	Complexity model versus measured times; linear regression	RQ6	detect

4.1.3 Experimental environment

Two machines were used (Table 4.2). Experiments that need the consumer-side scanner (ts-morph) or the project's own dependencies were run on the developer's computer; specification-only experiments that need large amounts of real specification history were run on a cloud workstation with a full clone of OpenAI's specification repository. All timing results report the machine used.

Table 4.2: Experimental Environment

	Machine A (developer computer)	Machine B (cloud workstation)
Processor	AMD Ryzen 7 7730U host; Linux VM with 2 vCPU	Intel Xeon @ 2.10 GHz, 2 vCPU
Memory	3.8 GiB	7.8 GiB
Operating system	Ubuntu 22.04, kernel 6.8	Ubuntu 24.04.5, kernel 6.18
Node.js	22.23.2	22.22.0
`sma` version	1.0.0 + unreleased fixes (commit `3cf434e`)	same source tree
Used for	Unit tests, petstore end to end, call-site benchmark	Mutation trials, compound trials, OpenAI history, scaling

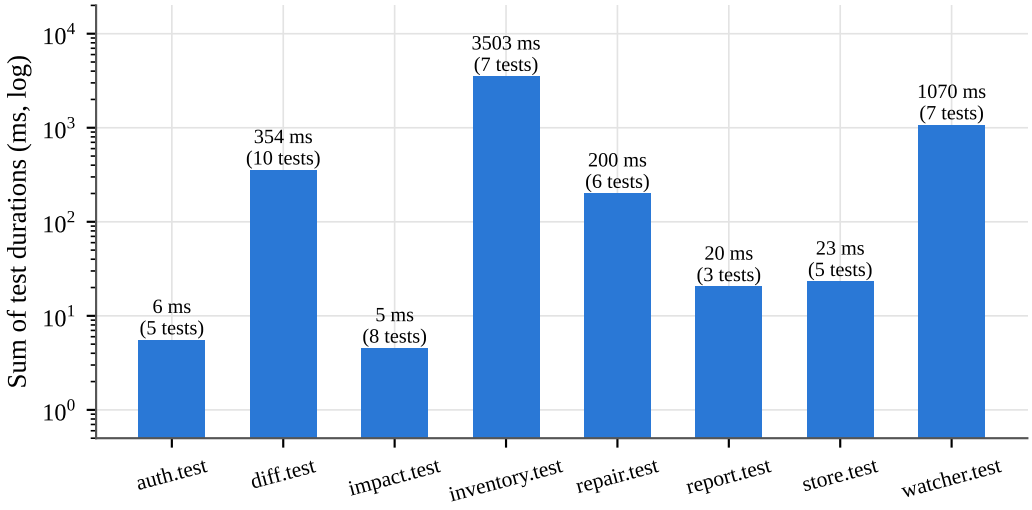
4.2 Experimental Method

4.2.1 Unit test results

All 51 unit tests passed on Machine A (Table 4.3). The total suite took 31.5 s by the test runner's clock, of which the individual tests account for about 5.2 s; the rest is process start-up and module loading for each test file. Graph 4.1 shows how the 5.2 s of test time is distributed: two test files account for most of it. The seven inventory tests build a ts-morph project and query the TypeScript language service (0.3 to 0.9 s each, 3.5 s in total), and one watcher test deliberately waits for a retry after a simulated HTTP 503 (1.05 s). The diff and rule tests, which load and resolve both petstore specifications, take 10 to 98 ms each; the impact tests take under 3 ms.

Table 4.3: Unit Test Results on Machine A

Measure	Value
Test files	8
Tests	51
Passed	51
Failed, cancelled, skipped	0, 0, 0
Suite duration (runner clock)	31.5 s
Same suite, CI matrix	Ubuntu and Windows, Node.js 22 and 24



Graph 4.1: Sum of Unit Test Durations per Test File (Machine A, Logarithmic Scale)

4.2.2 Controlled end-to-end upgrade: the petstore API

Procedure. The demo repository was rebuilt at API v1, the petstore server was started at v1, the baseline was recorded with `sma run --no-llm`, the server was switched to v2, and `sma run`

`--no-llm` was executed again. The repair step was taken from the project's recorded run of 18 October 2026, in which `sma fix` invoked the real Claude Code agent; its branch, prompts, PR body and run log are preserved in the project folder. For this report the agent's patch was re-applied to a fresh copy of the demo repository and the demo tests were re-run independently against both API versions.

Detection and impact. Figure 4.1 shows the actual terminal output of the detection run. The first run captured a baseline with 2 endpoints (3 operations) and exited with code 0. The second run reported 6 changes, 5 of them breaking, mapped them to 8 call sites across 3 files, reported the safe `PATH_ADDED` change as not affecting the repository, and exited with code 1. Table 4.4 compares the result with the designed ground truth of Table 3.8: every expected hit was found and nothing else was reported.

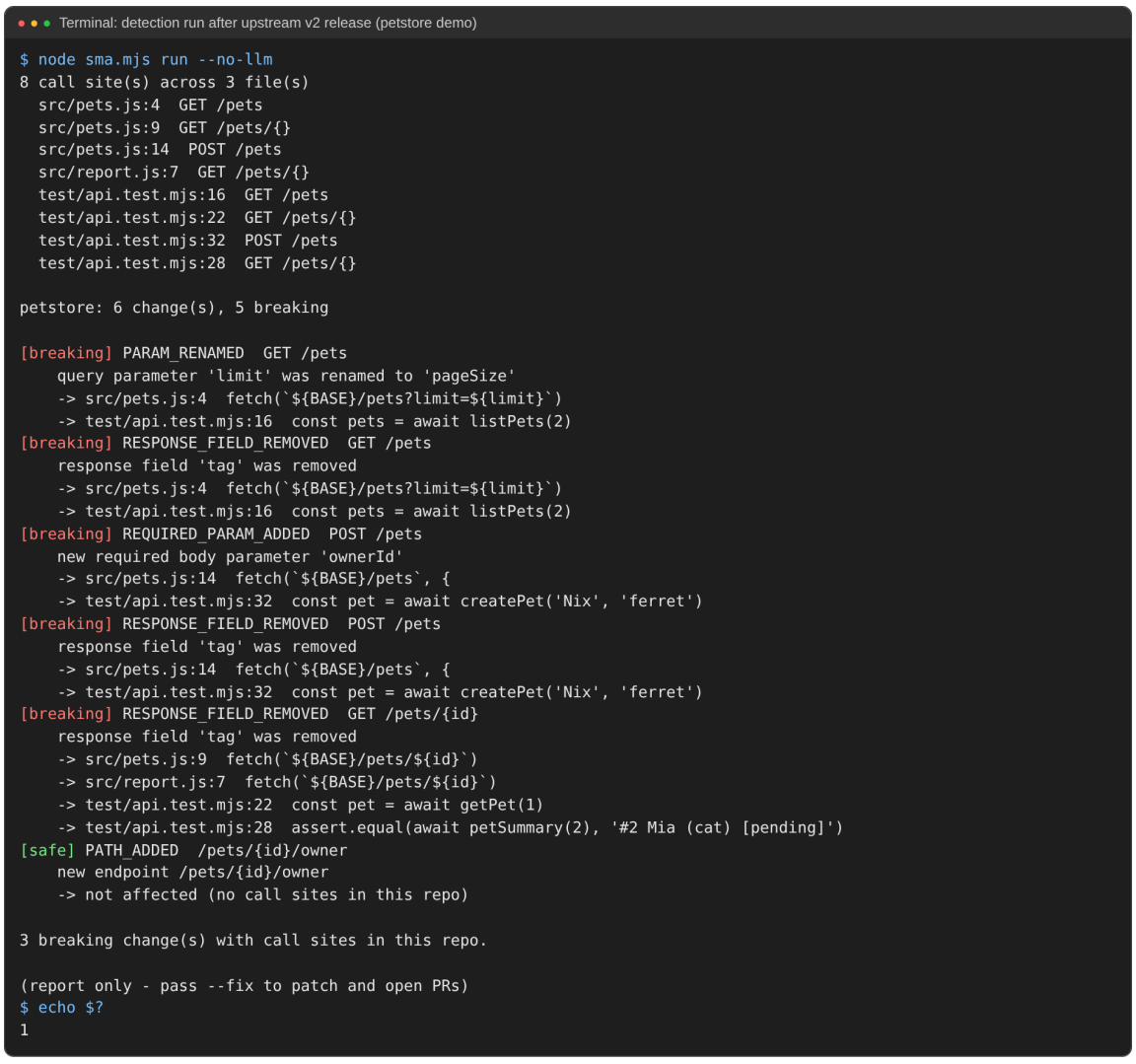


Figure 4.1: Terminal Output of the Detection Run After the Petstore v2 Release (Machine A)

Repair and verification. The recorded repair run created the branch `sma/petstore-20261018093325`, executed three agent runs (one per repair group) and then verified the result itself. The whole `fix` stage, from the run's start stamp (09:33:25 UTC) to its log entry

Table 4.4: Petstore Detection and Impact Results Compared with Ground Truth

Measure	Expected	Observed	Match
Changes detected	6	6	yes
Breaking / warning / safe	5 / 0 / 1	5 / 0 / 1	yes
Leftover deltas sent to the LLM	0	0	yes
Call sites found (direct HTTP + one-hop callers)	8 (4 + 4)	8 (4 + 4)	yes
Files containing call sites	3	3	yes
Breaking changes with call sites	5	5	yes
Repair groups after grouping	3	3	yes
Unaffected changes reported as such	1 (`PATH_ADDED`)	1	yes
Exit code (baseline run / detection run)	0 / 1	0 / 1	yes
Wall-clock time (baseline run / detection run)	-	3.46 s / 3.16 s	-
Peak memory (resident set)	-	211 MB / 210 MB	-

(09:34:45 UTC), took about 80 seconds, consistent with the approximately 75 seconds stated in the project's demonstration guide. Table 4.5 summarizes what each agent run did, taken from the PR body; Listing 4.1 shows the complete resulting diff.

Table 4.5: Recorded Repair Run: What Each Agent Changed

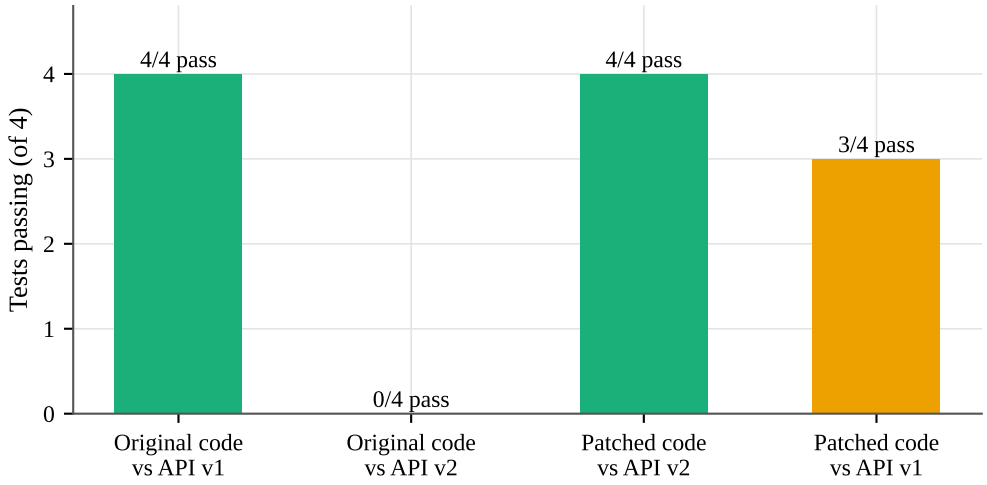
Repair group	Change made by the agent	Files
PARAM_RENAMED (limit)	Request now sends `?pageSize=${limit}`; function signature unchanged so callers are unaffected	`src/pets.js`
RESPONSE_FIELD_REMOVED (tag)	`describePet` returns the name only; two expected strings in the tests updated; `createPet` still sends `tag` because v2 still accepts it in requests	`src/pets.js`, `test/api.test.mjs`
REQUIRED_PARAM_ADDED (ownerId)	`createPet(name, tag, ownerId)` threads the new value into the body; the test passes owner 10, which exists in the server's seed data	`src/pets.js`, `test/api.test.mjs`
Harness verification	Tests: **PASS**. Files changed: 2, both predicted (outside scope: none). `needsHuman`: false. Diff: 7 insertions, 7 deletions.	PR body written to `.sma/pr/` (no remote)

Independent re-verification. Graph 4.2 and Figure 4.2 show the demo application's own tests in four configurations. The original code passed 4 of 4 tests against v1 and *0 of 4* against v2: the upstream release really did break the consumer, in the ways predicted (for example `'Rex (undefined)'` instead of `'Rex (dog)'`, which is the removed `tag` field surfacing as `undefined`). With the agent's patch applied, the tests passed 4 of 4 against v2. Against v1, the patched code passed 3 of 4: `listPets` now sends `pageSize`, which v1 ignores, so the page-size assertion fails. This is the expected consequence of migrating to a new contract, not a defect, and it confirms that the repair targets v2 specifically.

The agents' own summaries, quoted in the PR body (Appendix E), show the kind of judgment

Listing 4.1: Complete Patch Produced by the Agent Runs (git diff main..sma/petstore-20261018093325)

```
--- a/src/pets.js
+++ b/src/pets.js
 export async function listPets(limit = 10) {
-  const res = await fetch(`${BASE}/pets?limit=${limit}`);
+  const res = await fetch(`${BASE}/pets?pageSize=${limit}`);
-export async function createPet(name, tag) {
+export async function createPet(name, tag, ownerId) {
-    body: JSON.stringify({ name, tag }),
+    body: JSON.stringify({ name, tag, ownerId }),
 export function describePet(pet) {
-  return `${pet.name} (${pet.tag})`;
+  return `${pet.name}`;
--- a/test/api.test.mjs
+++ b/test/api.test.mjs
-  assert.equal(describePet(pet), 'Rex (dog)');
+  assert.equal(describePet(pet), 'Rex');
-  assert.equal(await petSummary(2), '#2 Mia (cat) [pending]');
+  assert.equal(await petSummary(2), '#2 Mia [pending]');
-  const pet = await createPet('Nix', 'ferret');
+  const pet = await createPet('Nix', 'ferret', 10);
```



Graph 4.2: Demo Application Tests Passing Before and After the Upstream Change and the Repair

the reviewer checklist is meant to catch. The `tag` repair changed user-visible text ("Rex" instead of "Rex (dog)"), and the agent flagged this explicitly as a product decision; the `ownerId` repair noted that a caller omitting the new argument would send `undefined` and receive HTTP 400. Neither concern is visible to a test suite, which is why a human remains the final step.

4.2.3 Real-world replay: OpenAI, June 2023

Procedure. OpenAI's specification at commit `a27cad2f` was the baseline, and commit `d9c3021a` (19 June 2023, "remove disabled/deprecated endpoints") the new version. The consumer was

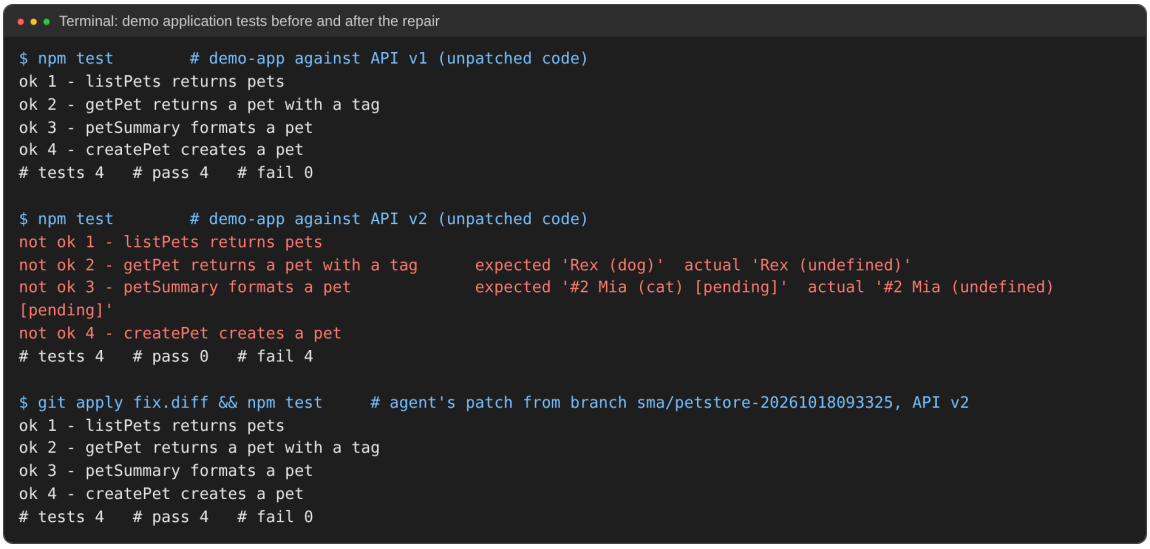


Figure 4.2: Terminal Output of the Demo Application Tests (Condensed; Machine A)

the sample bot in `demo/real-world/app`. The run was executed with `--no-llm`.

Results. Figure 4.3 shows the classifier output and Table 4.6 the impact. All five removed endpoints were detected as `PATH_REMOVED`. The scanner found 6 call sites across 2 files (4 direct requests in `openai.js`, 2 one-hop callers in `bot.js`). Exactly the two endpoints the bot uses, `/engines` and `/answers`, were marked affected, each with its direct request and the helper call one hop away; the other three were correctly reported as not affecting the repository. The calls to `/completions` and `/models`, which still exist, produced no finding. A second real change was also reproduced: between commits `9ce93317` and `c012b5cc` (13 June 2023), OpenAI made `prompt` required on `POST /completions`, and `sma` reported exactly one finding, `PARAM_MADE_REQUIRED`. The `diff` of the first pair took 0.35 s on Machine B, including parsing two YAML files of 102 and 132 kB.

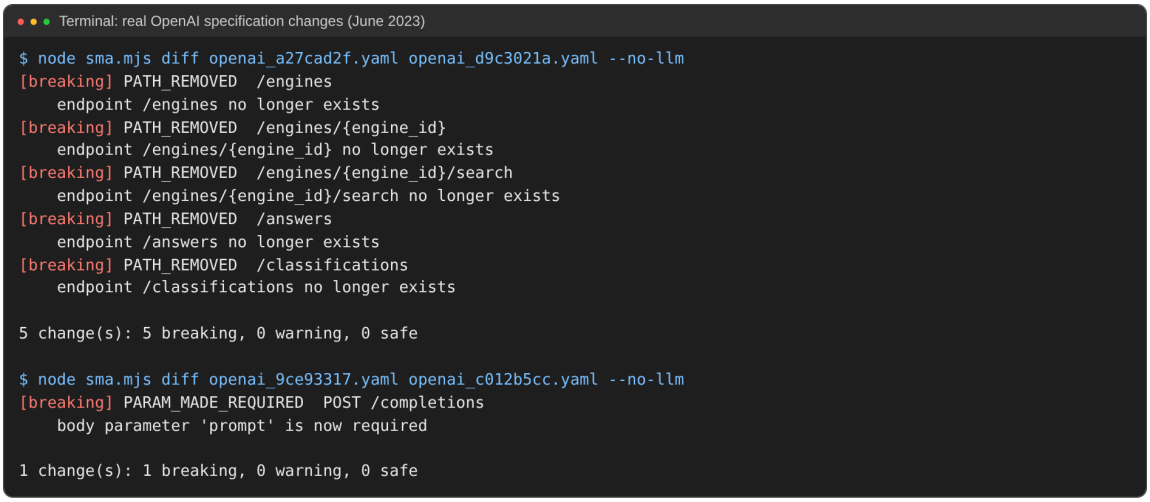


Figure 4.3: Classification of Two Real OpenAI Specification Changes (Machine B)

Observation: the specification lagged production. OpenAI's deprecations page records that

Table 4.6: OpenAI Replay: Detected Changes and Their Impact on the Sample Application

Finding	Severity	Impact (call sites)
PATH_REMOVED `/engines`	breaking	`openai.js:6` (http), `bot.js:4` (caller via `listEngines`)
PATH_REMOVED `/engines/{engine_id}`	breaking	not affected
PATH_REMOVED `/engines/{engine_id}/search`	breaking	not affected
PATH_REMOVED `/answers`	breaking	`openai.js:11` (http), `bot.js:6` (caller via `answer`)
PATH_REMOVED `/classifications`	breaking	not affected
PARAM_MADE_REQUIRED `POST /completions` (`prompt`), commit `c012b5cc`	breaking	found by `sma diff` (no repository join)

these endpoints were shut down on 3 December 2022 [12]; the commit that removed them from the published specification is dated 19 June 2023, 198 days later. A consumer relying on `sma` alone would have learned of this removal about six and a half months after its calls began to fail. This is an honest limit of any specification-driven approach (Section 4.6) and is not specific to `sma`.

4.3 Statistical Method

4.3.1 Mutation benchmark: design

To measure classification accuracy with known ground truth, a mutation benchmark was built. A *mutation operator* injects one precisely known change into a real resolved specification and records the rule hit(s) that a correct classifier must report. Twenty-four operators were defined (Table 4.7): eighteen that correspond to real API changes, covering every rule, and six *noise* operators that change only prose, examples, order or the version string and must produce no finding. Each operator was applied 100 times to each of three base specifications, choosing the target operation, parameter or field uniformly at random from the eligible ones (seeded pseudo-random generator, seed 42): the petstore v1 specification (3 operations), OpenAI's specification at commit `a27cad2f` (2023) and OpenAI's specification as of 28 May 2024 (commit `d376fedb`). An operator was skipped on a base where no target was eligible (for example, the 2023 OpenAI specification has no response enum). This gave **6,700 trials**.

Each trial was scored by two classifiers:

- **Method A, sma:** `applyRules` plus the leftover computation. A trial is *exact* if the multiset of reported (rule, method, path) equals the expected one and no leftover delta remains.

- **Method B, naive structural diff:** the same generic diff, classifying every `remove` or `change` delta as breaking and every `add` as safe. This is the behavior of a generic JSON comparison without API semantics, and the natural baseline for the question "does the rule table add value?".

Table 4.7: Mutation Operators Used in the Benchmark

Operator	Injected change	True severity
PATH_REMOVED, METHOD_REMOVED	Delete a path, or one method of a multi-method path	breaking
REQUIRED_PARAM_ADDED	Add a required query parameter	breaking
REQUIRED_BODY_FIELD_ADDED	Add a required property to the JSON request body	breaking
PARAM_RENAMED, PARAM_REMOVED	Rename or delete a query parameter	breaking
PARAM_TYPE_CHANGED, PARAM_MADE_REQUIRED	Swap a parameter's type; make an optional one required	breaking
RESPONSE_FIELD_REMOVED, RESPONSE_FIELD_TYPE_CHANGED	Delete or retype a property of the 2xx response	breaking
ENUM_VALUE_REMOVED	Drop the first value of a response enum	breaking
AUTH_SCHEME_CHANGED	Replace an existing security scheme	breaking
PATH_DEPRECATED	Set `deprecated: true` on an operation	warning
OPTIONAL_PARAM_ADDED, RESPONSE_FIELD_ADDED, ENUM_VALUE_ADDED, PATH_ADDED, METHOD_ADDED	Purely additive changes	safe
NOISE: description, summary, example, info.version, reorder parameters, reorder enum	Cosmetic edits	none

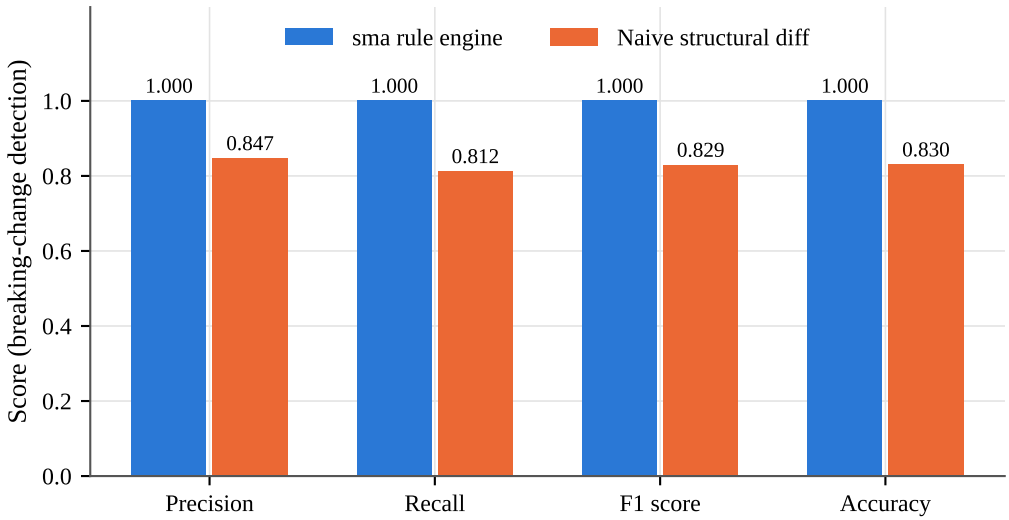
4.3.2 Single-change results

All 6,700 trials were classified exactly by `sma`: every expected hit was reported with the right rule, method, path and severity, no extra hit was reported, and no leftover delta remained, including for all 1,600 noise trials. The rename rule found every one of the 300 renames, although each new name was a random string unrelated to the old one. Table 4.8 and Graph 4.3 compare the two methods on the binary question *was a breaking change reported?*

Table 4.8: Breaking-Change Detection on 6,700 Single-Change Trials: `sma` versus Naive Diff

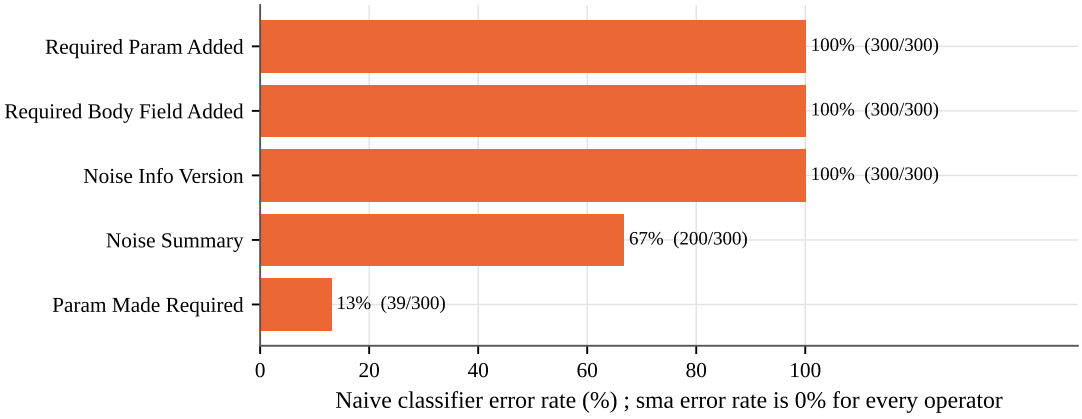
Method	TP	FN	FP	TN	Precision	Recall	F1	Accuracy
A: `sma` rule engine	3,400	0	0	3,300	1.000	1.000	1.000	1.000
B: naive structural diff	2,761	639	500	2,800	0.847	0.812	0.829	0.830

The naive classifier's errors are not random; they are systematic and occur exactly where API semantics matter (Graph 4.4):



Graph 4.3: Breaking-Change Detection Scores of sma and the Naive Structural Diff

- **Missed breaking changes (639).** Adding a required parameter or a required body field is an `add` delta, which the naive method calls safe; it missed all 600 such trials. It also missed 39 of 300 parameters made required, those where the `required` key was newly added rather than flipped from `false`.
- **False alarms (500).** Bumping `info.version` and editing an operation's `summary` are `change` deltas that the naive method calls breaking (500 false alarms); sma's ignore set and rules report nothing for them.
- **Noise volume.** For true breaking changes the naive diff emitted 1.35 deltas per change on average (a rename appears as one removal plus one addition; a removed response field appears in both `properties` and `required`), whereas sma emitted exactly 1.00 finding per change.



Graph 4.4: Operators on Which the Naive Diff Errs, with Error Rate (sma Errs on None)

Classification time. The mean time to classify one mutated pair (rules plus leftovers, specifications already resolved) was 0.20 ms for petstore, 3.1 ms for OpenAI 2023 and 44.4 ms

for OpenAI 2024 (median 0.09, 2.8 and 40.4 ms; 95th percentile 0.17, 4.6 and 62.7 ms) on Machine B.

4.3.3 Compound releases

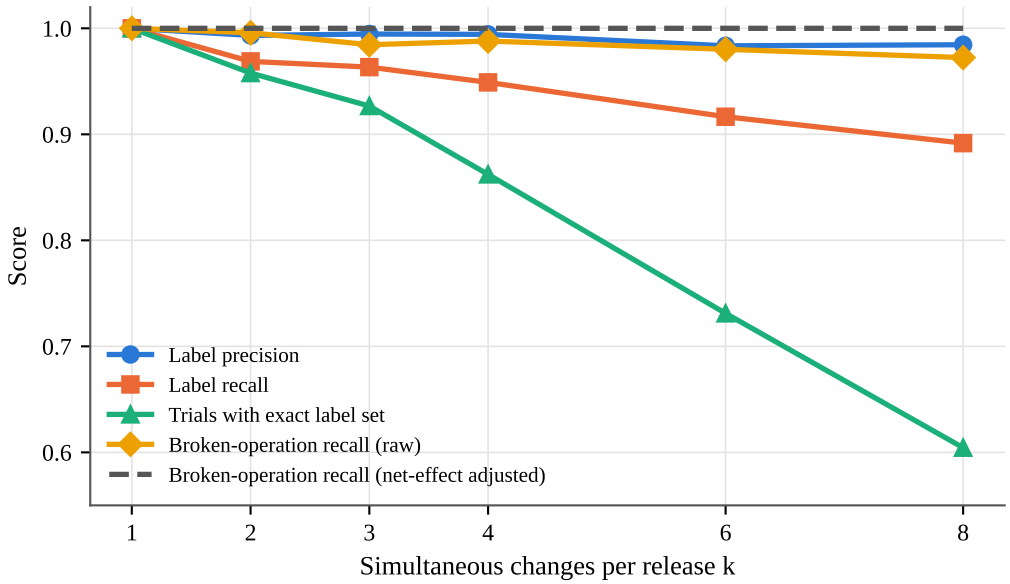
Real releases often contain several changes, sometimes to the same operation, and the rule predicates (notably the rename rule) interact. A second benchmark therefore applied $k \in \{1, 2, 3, 4, 6, 8\}$ randomly chosen operators in one release (85% real changes, 15% noise), allowing them to land on the same operation, with 150 trials per k per base specification (**2,700 trials**). Expected labels were accumulated as the operators were applied. Two measures were computed: *label* precision and recall (does each expected (rule, method, path) appear?) and *broken-operation recall* (is every operation that a breaking operator touched reported with at least one breaking finding?). The second is the safety-critical measure, because the repair stage acts on breaking findings per operation, not on the exact label. Table 4.9 and Graph 4.5 give the results.

Table 4.9: Compound-Release Results by Number of Simultaneous Changes k

k	Label P	Label R	Exact trials	Broken ops	Op recall (raw)	Op recall (adj.)
1	1.0000	1.0000	100.0%	235	1.0000	1.0000
2	0.9933	0.9688	95.8%	477	0.9958	1.0000
3	0.9946	0.9634	92.7%	710	0.9845	1.0000
4	0.9943	0.9490	86.2%	913	0.9880	1.0000
6	0.9835	0.9166	73.1%	1,262	0.9802	1.0000
8	0.9845	0.8917	60.4%	1,597	0.9724	1.0000

Adjusted recall excludes operations whose net change was zero (38 cases, *e.g.*, a parameter type swapped twice) or purely additive (55 cases, *e.g.*, a breaking edit applied to a parameter that was itself added in the same release), which are correctly not breaking for an existing client.

Label recall falls from 1.00 to 0.89 as k grows to 8, and the fraction of trials with an exactly correct label set falls to 60.4%. Inspection of every mismatch showed that they are *label* disagreements caused by interacting edits, not missed breakage: the most frequent were parameters made required after being added in the same release (correctly reported once, as `REQUIRED_PARAM_ADDED`), renames combined with another parameter change on the same operation (the rename predicate no longer holds, so the change appears as a removal plus an addition, both breaking), and repeated application of the same edit. At the operation level, raw recall stays at or above 0.972, and every one of the 93 apparent misses was traced to a net-zero or purely additive change that a correct classifier should *not* call breaking. After this adjustment, **no broken operation was missed in any of the 2,700 trials**. The practical consequence is that the label shown to the reviewer can be imprecise in busy releases, while the decision that matters, whether an operation needs attention, is preserved.



Graph 4.5: Label Accuracy and Broken-Operation Recall as Releases Carry More Changes

4.3.4 Call-site detection benchmark

Corpus. A generator produced a labeled JavaScript repository of 80 files (40 modules and 40 consumer files) containing 467 labeled statements (Table 4.10). Statements were drawn from 21 templates with weights favoring real calls: 12 forms of direct HTTP call (`fetch` with literal, template and query-string URLs and with a `method` option; `axios.get/post/put`; the `axios({url, method})` configuration form; `got.delete`; `ky.patch`; `request`), 2 forms whose URL is computed at run time, 1 form using an HTTP client the scanner does not know (`this.http.get`), and 6 forms of *non-call* that merely mention an API path (a comment, a log line, a request to another host, a string concatenation, an object literal). Every module function that made a real call was invoked from a consumer file (a one-hop caller) and that consumer function was invoked again from `main()` (a two-hop caller).

Table 4.10: Composition of the Labeled Call-Site Corpus

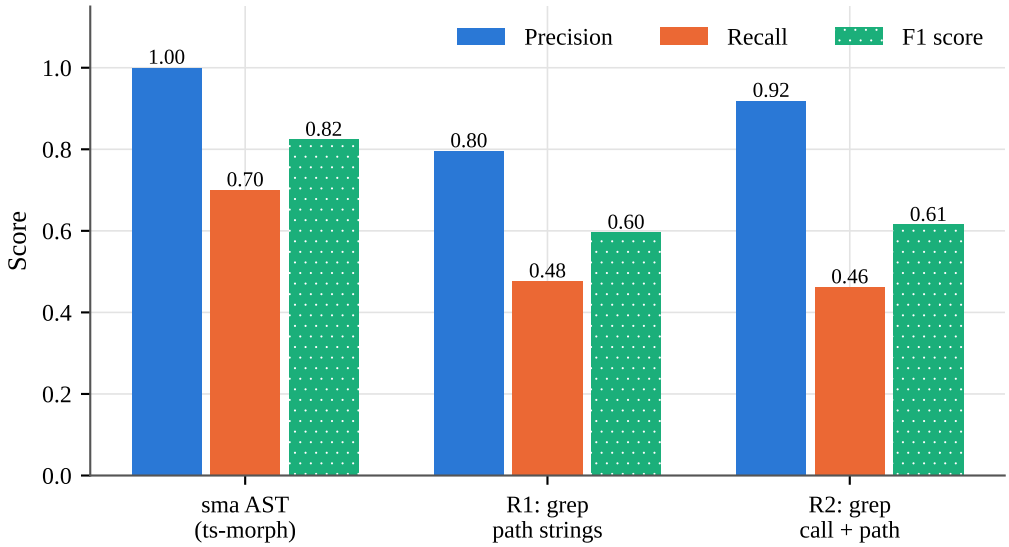
Category	Statements	Ground truth
Direct HTTP calls (supported clients, literal or template URL)	182	call site
One-hop callers of a wrapper function	109	call site (affected)
Two-hop callers	109	call site (affected; outside sma's design)
Calls with a URL computed at run time	10	call site (cannot be resolved statically)
Calls through an unsupported client (`this.http.get`)	6	call site
Non-calls that mention an API path	51	not a call site
Total	**467**	416 positives, 51 negatives

Methods compared. Method A is `sma`'s ts-morph scanner. Two text-search baselines represent what a developer would do by hand: **R1** reports every line containing a quoted string with an API path ("grep for the endpoint"); **R2** additionally requires an HTTP client name followed by `.` or `(` on the same line ("grep for calls to the endpoint"). Both baselines extract the method from `.get(`, `.post(` or `method: '...'` on the line.

Results. Table 4.11 and Graphs 4.6 and 4.7 give the results.

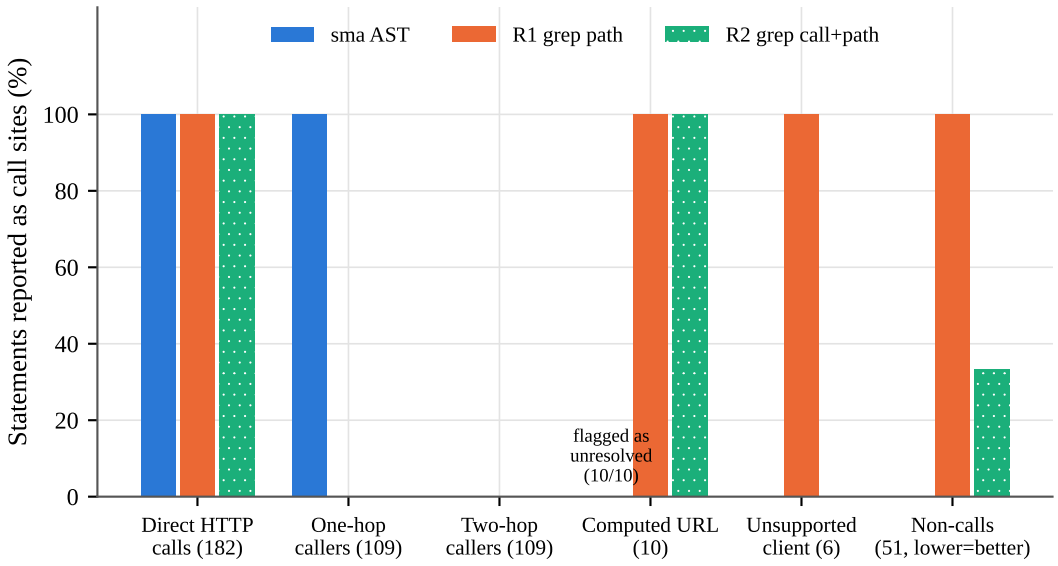
Table 4.11: Call-Site Detection on the Labeled Corpus: AST Scanner versus Text Search

Method	Reported	TP	FP	FN	P	R	F1	Time
A: `sma` AST scanner	291	291	0	125	1.000	0.700	0.823	1,028 ms
R1: grep path strings	249	198	51	218	0.795	0.476	0.595	2.3 ms
R2: grep call + path	209	192	17	224	0.919	0.462	0.614	2.3 ms



Graph 4.6: Precision, Recall and F1 of the Three Call-Site Detection Methods

- **Precision.** The AST scanner reported *no* false call site. R1 reported all 51 non-calls (comments, logs, other hosts, strings), and even the stricter R2 reported 17 of them (requests to another host and URLs printed by a logger that happen to share a line with `fetch`).
- **Recall within the design.** Every one of the 182 direct calls and every one of the 109 one-hop callers was found. Neither text baseline found a single caller, because a caller line contains no URL. Excluding the 109 two-hop callers, which are outside the one-hop design, the scanner's recall is $291/307 = 0.948$; counting the 10 computed-URL calls that it explicitly *flagged* as unresolved for human review, $301/307 = 0.980$. The only silent misses were the 6 calls through an unsupported client.



Graph 4.7: Share of Each Statement Category Reported as a Call Site, by Method

- **By-design misses.** All 109 two-hop callers were missed, exactly as the one-hop limit predicts.
- **Endpoint attribution.** Of the 291 sites found, 260 (89.3%) were attributed to the correct method and path. All 31 errors came from one construct: the 19 calls written as `axios({ url, method: 'post' })` were recorded as `GET`, because the scanner reads the method from the callee name or from a *second* options argument, not from the configuration object itself; the remaining 12 errors are the callers of those functions, which inherit the wrong method. Every other form (82 `fetch`, 37 `axios.verb`, 19 `got`, 9 `ky`, 16 `request`) was attributed correctly. This is a real, easily fixed defect (Section 5.2).
- **Cost.** The AST scan took about 1.0 s for 80 files against 2.3 ms for the text search, a price of roughly one second for zero false positives and the caller pass.

4.3.5 Longitudinal study of a real specification

Purpose and procedure. Synthetic benchmarks can only test what their author thought of. To observe the classifier on input that no one involved in the project controlled, OpenAI's public specification repository [4] was cloned in full and *every* commit that changed `openapi.yaml` was replayed in order, from the first (19 January 2022) to the latest (21 October 2026). For each consecutive pair of versions the script parsed and resolved both specifications and ran the rule engine and the leftover computation exactly as `sma check --no-llm` would, recording every finding and the time of each stage on Machine B. Table 4.12 summarizes the data.

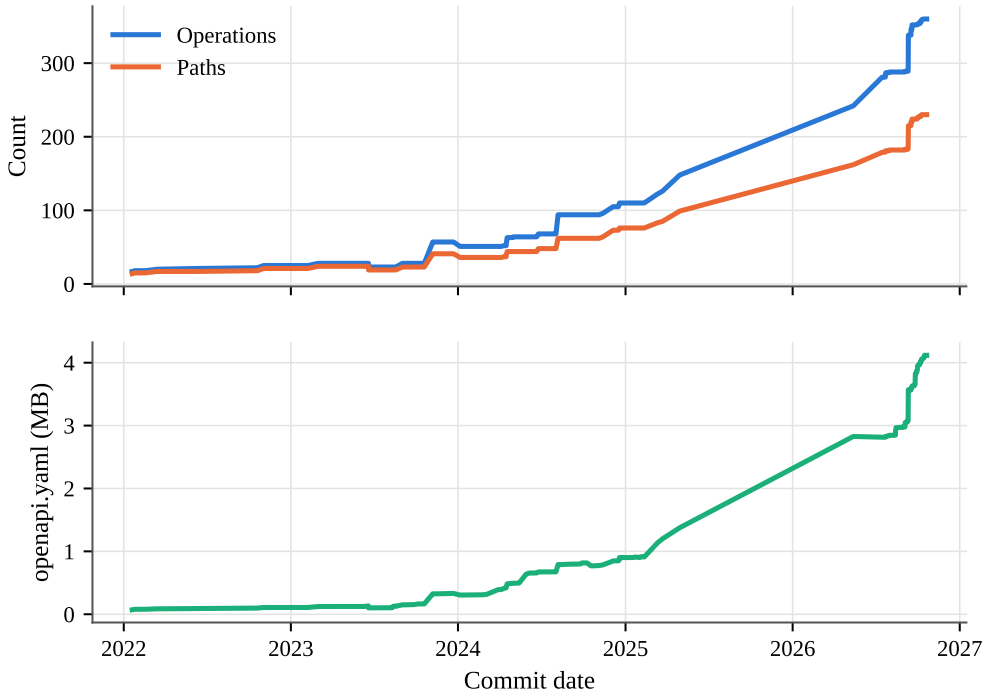
Growth of the specification. Graph 4.8 shows how the target grew: the number of operations rose more than twenty-fold and the file more than fifty-fold in under five years, with a steep rise after the specification was replaced by a complete, generated version in May 2026. A consumer-

Table 4.12: Longitudinal Study of OpenAI's Specification: Data Summary

Measure	Value
Commits that changed `openapi.yaml` (Jan 2022 to Oct 2026)	478
Versions retrievable / parsed successfully	477 / 473
Versions that were not valid YAML as published by the provider	4
Consecutive version pairs classified	472
Specification size, first to last version	14 to 230 paths; 17 to 360 operations; 71 kB to 4.1 MB
OpenAPI version	3.0.0 until 13 May 2026, then 3.1.0
Pairs with no detected change at all	41
Pairs with no leftover delta (including the 41 unchanged)	141
Changed pairs fully explained by the rules	100 of 431 (23.2%)
Pairs with leftover deltas for the LLM / `UNCLASSIFIED`	331 (14,384 deltas)
Raw rule findings	37,451
of which identity artifacts of OAS 3.1 type arrays (defect)	35,900 (95.9%)
Findings after removing the artifacts	1,551
breaking / warning / safe	739 / 27 / 785
Pairs with at least one breaking finding	69
Processing time per version (parse + resolve + rules + leftovers)	median 3.8 s, maximum 10.2 s

side tool therefore has to cope with an input that changes size by orders of magnitude over its lifetime.

Finding 1: a real defect with OpenAPI 3.1 type arrays. Starting on 10 September 2026, almost every commit produced between 83 and 273 (median 215) `RESPONSE_FIELD_TYPE_CHANGED` and `PARAM_TYPE_CHANGED` findings, even for commits that only edited a description. Inspection showed findings such as *field `failed_at` changed type from `["integer","null"]` to `["integer","null"]`*: the type had *not* changed. OAS 3.1 allows `type` to be an array to express nullability, and OpenAI began to use this form in September 2026 (163 occurrences in the file by 23 September, 296 by 8 October). The rule engine compares types with JavaScript's `!==`, which for two arrays compares object identity, not content, so every array-typed field is reported as changed on every run. In total 35,900 of the 37,451 raw findings (95.9%) were such artifacts, spread over 191 version pairs. None of the synthetic tests could expose this, because the petstore specification and the mutation operators use OAS 3.0 scalar types. In practice the defect would have made every daily run report hundreds of breaking changes and, wherever a repository uses the affected endpoints, trigger repairs for changes that never happened. The



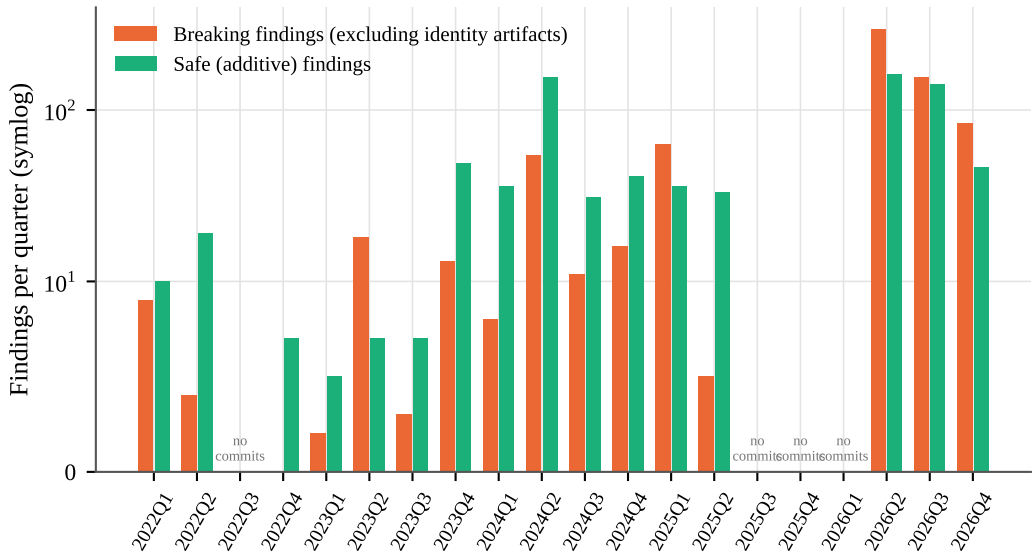
Graph 4.8: Growth of OpenAI's Published Specification, January 2022 to October 2026

fix is a two-line change (the two type comparisons in `rules.mjs`) to compare types by value (Section 5.2); all figures below exclude the artifacts.

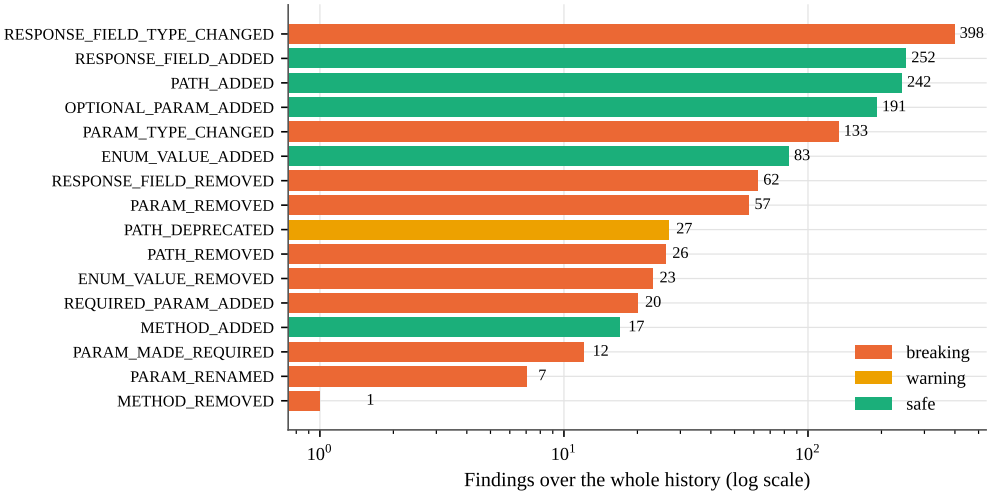
Finding 2: the real breaking changes match the commit history. After removing the artifacts, 739 breaking findings remained in 69 of the 472 pairs (Graph 4.9). The large ones line up with events documented in the commit messages: required parameters fixed in February 2022 ("fix some required params"), the removal of `/engines`, `/answers` and three other endpoints in June 2023, the migration of the Assistants API in April 2024 (`file_ids` removed from requests and responses), the removal of a project's `app_use_case` and `business_website` fields in October 2024 ("Revert app use case fields"), the move of the eval-run cancel endpoint in September 2026 (`METHOD_REMOVED`), `model` becoming required for image generation when the DALL·E models were deprecated (`PARAM_MADE_REQUIRED`), and a body field renamed from `geography` to `residency` in August 2026, which the structural rename rule reported as `PARAM_RENAMED`. Graph 4.10 gives the distribution over the rules.

Finding 3: precision on real data, and two semantic gaps. Because the history has no ground truth, a manual check was carried out. The 492 breaking findings recorded in detail (up to 60 per version pair) were first grouped by kind (Table 4.13); then a random sample of 30 of the structural findings (removals, required parameters, renames, enum removals) was checked against the specification text and the commit message.

Of the 30 sampled structural findings, **22 were genuine contract changes** and **8 were false**, giving a sample precision of 73%. All 8 false findings came from a single commit of 11 March



Graph 4.9: Breaking and Safe Findings per Quarter in OpenAI's Specification History



Graph 4.10: Distribution of Findings over the Rules Across the Whole History (Artifacts Removed)

Table 4.13: Kinds of Breaking Findings in the History (492 Findings Recorded in Detail)

Kind	Count	Interpretation
Structural (removal, required, rename, enum)	157 (31.9%)	Mostly genuine contract changes; see the manual check below
Nullability change (`string` ↔ `["string","null"]`)	177 (36.0%)	Breaking for a *response* field (clients may now receive `null`); not breaking for a *request* parameter that merely accepts `null`
Composition change (a scalar `type` replaced by `anyOf`/`oneOf`, or the reverse)	129 (26.2%)	Ambiguous: may widen or narrow the contract; needs schema-level comparison
Plain type change (*e.g.*, `string` → `integer`)	29 (5.9%)	Genuine, *e.g.*, batch timestamps changed from string to integer in April 2024

2025 in which OpenAI restructured `CreateChatCompletionRequest` as an `allOf` of a shared base schema and an object: the field normalizer F reads only the top-level `properties` of the request body, so every field that moved inside `allOf` appeared to be removed. Together with the composition and nullability rows of Table 4.13, this identifies two semantic gaps that the synthetic benchmark could not reveal: schema composition keywords (`allOf`, `anyOf`, `oneOf`) are not merged before comparison, and type changes are judged without regard to direction (request versus response, widening versus narrowing).

Finding 4: the residue is large on real specifications. On the demo, the rules explain the entire diff and the LLM is never called. On OpenAI's history, of the 431 version pairs that changed anything, the rules fully explained only 100 (23.2%); the other 331 left 14,384 deltas, mostly inside nested schemas and composition keywords that no rule models. With an API key these would be sent to the LLM and gated by the confidence floor; without one they appear as `UNCLASSIFIED`. This shows that the LLM fallback is not a corner case on a real, fast-moving API, and that extending the rules to nested properties and composition would reduce both cost and noise.

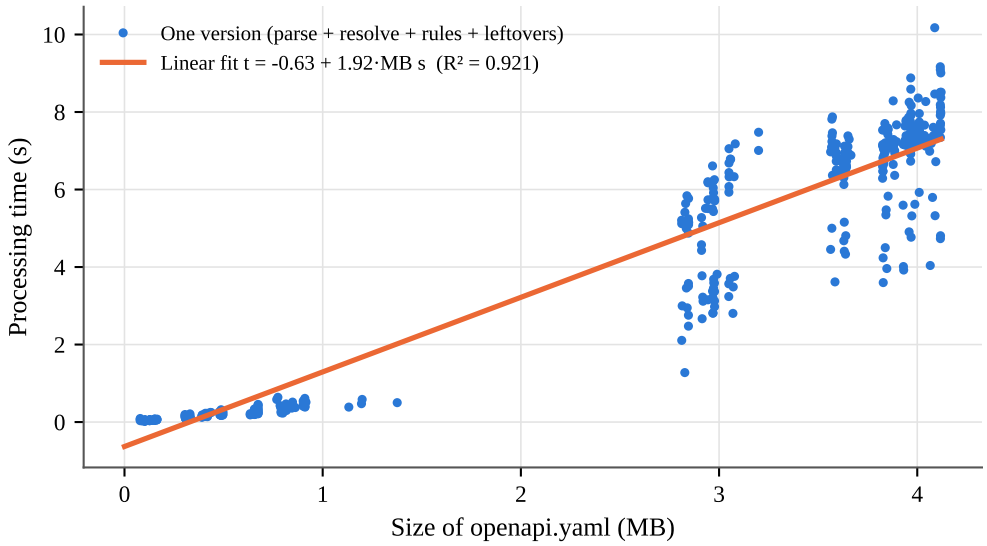
Finding 5: robustness to broken input. Four published versions (two on 28 June 2023, two on 17 June 2024) were not valid YAML. On those days `sma` would have exited with code 2 (tool failure) and, by the ordering rule of Section 3.5, kept the previous baseline, so the changes would have been reported on the next valid version instead of being lost.

Processing time on real input. Graph 4.11 plots the processing time of each version against the size of its YAML file. A linear fit gives $t \approx -0.63 + 1.92 \cdot \text{MB}$ s ($R^2 = 0.921$): even the 4.1 MB specification of October 2026, which inflates to 61.4 MB when resolved, is processed in under about 10 s. Over the whole history, 23% of the time was spent parsing YAML, 14% resolving references and 64% computing leftovers; the rule engine itself took well under 1%. The vertical spread at a given size therefore reflects mainly how much the two versions differ, because more deltas mean more leftover work.

4.4 Analytical Method

4.4.1 Predicted versus measured scaling

Section 3.4.10 predicted that each detection stage is linear in the size of the specification for a fixed number of changes. To test the prediction, synthetic specifications with $N \in \{10, 25, 50, 100, 250, 500, 1000, 2000, 4000\}$ resources were generated, each resource having three operations (list, create, get), two query parameters, a body and a 10-property response schema referenced through `$ref`, so that the specifications ranged from 30 to 12,000 operations (32 kB to 12.8 MB resolved). A second copy of each had one resource in every 100 changed, and at least one (a rename, a removed field and a new required parameter on each), so that



Graph 4.11: Processing Time per Version Against Specification Size (OpenAI History, Machine B)

about 1% of resources changed in the larger specifications. Each stage was timed five times on Machine B and the median taken (Table 4.14).

Table 4.14: Median Stage Times on Synthetic Specifications (Machine B)

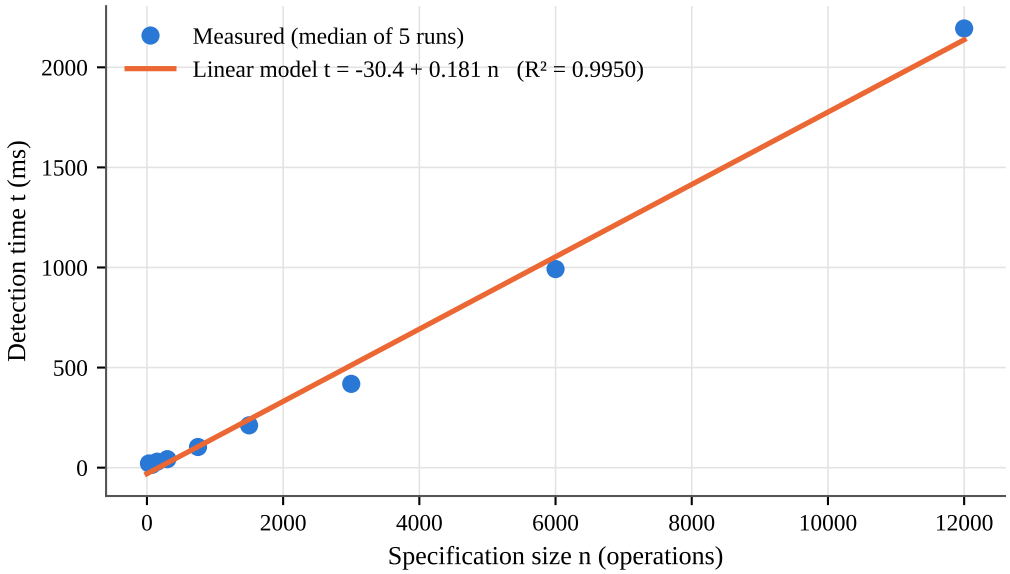
Operations n	**Resolved size (kB)**	**Resolve (ms)**	**Diff (ms)**	**Rules (ms)**	**Leftovers (ms)**	**Hits**	**Heap (MB)**
30	32	3.7	6.1	0.7	6.8	6	11
75	80	2.9	3.2	2.2	3.2	6	15
150	159	4.3	12.0	3.4	5.9	6	17
300	319	6.9	12.5	6.6	9.6	6	36
750	798	23.2	23.3	11.0	22.6	18	43
1,500	1,596	43.5	55.7	21.0	47.8	30	65
3,000	3,192	90.4	108.9	34.5	94.5	60	128
6,000	6,391	231.4	231.1	69.0	229.2	120	380
12,000	12,789	531.4	468.3	144.6	518.4	240	632

A least-squares fit of the linear model $t(n) = \beta_0 + \beta_1 n$ to the total detection time (resolving both specifications, diff, rules and leftovers) gives

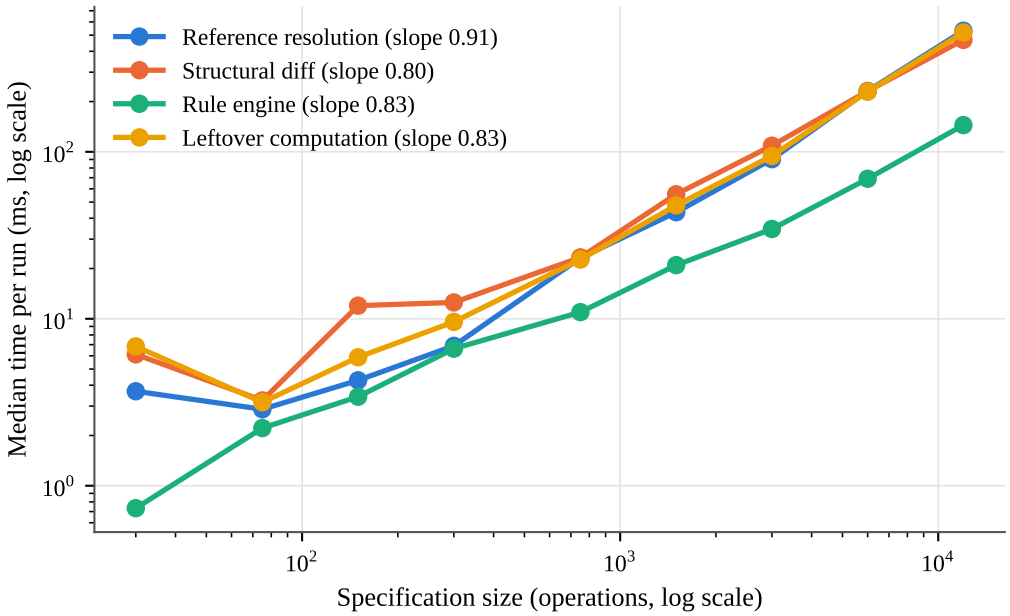
$$t(n) \approx -30.4 + 0.181\, n \text{ ms}, \qquad R^2 = 0.995, \tag{4.1}$$

that is, about 0.18 ms per operation, or about 2.1 s predicted for a 12,000-operation specification (2.19 s measured) (Graph 4.12). The per-stage fits have R^2 between 0.992 and 0.999. On logarithmic axes (Graph 4.13) the slopes over the whole range are 0.80 to 0.91, below the predicted 1, because fixed costs (function compilation, small allocations) dominate the smallest specifications; fitted over $n \geq 750$ only, the slopes are 1.15 (resolve), 1.07 (diff), 0.92 (rules) and

1.13 (leftovers), all close to 1. The slightly super-linear tail of the stages that allocate the most (resolve and leftovers produce a new object per node) is consistent with garbage-collection cost growing with heap size, which rose from 11 to 632 MB. The analytical prediction of linear scaling is therefore confirmed within the measured range.



Graph 4.12: Total Detection Time Against Specification Size, with the Fitted Linear Model



Graph 4.13: Per-Stage Detection Time on Logarithmic Axes, with Log-Log Slopes

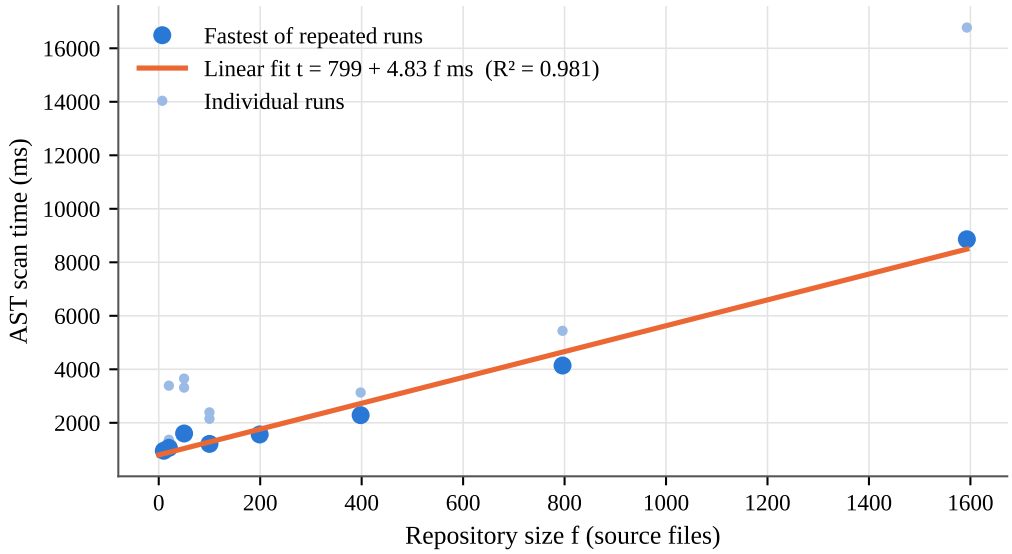
4.4.2 Scaling of the AST scan

The analytical model predicted that the scan costs $O(N)$ in the number of AST nodes plus one language-service query per HTTP site. The labeled corpus generator was run at eight sizes

from 10 to 1,593 files (57 to 8,891 labeled statements) on Machine A, two or three times each. Because Machine A is a virtual machine whose timings varied by up to a factor of three between identical runs, the fastest run at each size is used as the estimate of cost (the usual practice for noisy benchmarks), with all runs shown (Graph 4.14). A linear fit gives

$$t_{\text{scan}}(f) \approx 799 + 4.83\,f \text{ ms}, \qquad R^2 = 0.981, \tag{4.2}$$

where f is the number of files: a fixed cost of about 0.8 s to create the TypeScript project and language service, plus about 5 ms per file. A 1,593-file repository was scanned in 8.9 s. Precision stayed at 1.00 and recall between 0.696 and 0.721 at every size, so accuracy does not degrade with scale. The prediction of linear growth is confirmed; the large intercept explains why the per-file cost is invisible on small repositories such as the demo (3 files).



Graph 4.14: AST Scan Time Against Repository Size, with the Fitted Linear Model (Machine A)

4.4.3 Reference inflation

The model also predicted that reference resolution could inflate a specification far beyond its raw size. On OpenAI's specification of 21 October 2026 (4.1 MB of YAML, 230 paths), parsing took 1.49 s and resolution 1.02 s on Machine B, and the resolved object was 61.4 MB as JSON against 3.15 MB before resolution: an **inflation factor of 19.5**. Because the baseline stored in `.sma/specs/` is the *resolved* specification, this factor also determines the size of the committed file. The project's own pilot plan records 111.5 MB for GitHub's resolved specification. This is the main scalability risk identified by the analysis, and Section 5.2 proposes storing the raw specification and resolving on load.

4.4.4 End-to-end cost model

Combining the measurements gives a simple cost model for one scheduled run on a repository with one watched API:

$$t_{\text{run}} \approx t_{\text{start}} + t_{\text{fetch}} + t_{\text{parse}} + t_{\text{detect}}(n) + t_{\text{scan}}(N) + \mathbb{1}[Q \neq \emptyset]\,(|Q|\,\bar{t}_{\text{agent}} + t_{\text{tests}}), \tag{4.3}$$

where on the petstore demo the deterministic part (process start, fetch, scan of 3 files, detection, report) measured 3.2 s in total, and the repair part measured about 80 s for $|Q| = 3$ agent runs and one test run. Detection is therefore cheap enough to run on every push, while the repair cost is paid only when a breaking change actually reaches the repository, which the impact join keeps rare.

4.5 Comparison of Results Obtained by Different Methods

Table 4.15 sets the results of the three methods side by side for each research question.

Table 4.15: Comparison of Results Across Evaluation Methods

Question	Experimental	Statistical	Analytical / literature
RQ1 detection	Petstore 6/6 changes, OpenAI 5/5 removals and 1/1 required parameter, 0 false findings	6,700/6,700 exact; compound op-level recall 1.00 (adjusted), label recall 0.89 at $k = 8$	Predicates (Table 3.4) predict rename ambiguity under compound edits; observed
RQ2 versus baseline	Rules explain the whole petstore diff; LLM never called	Accuracy 1.000 versus 0.830; naive misses all required additions	Naive diff has no notion of requiredness or cosmetics, so these errors are predictable
RQ3 impact	8/8 sites (4 callers) on petstore; 6/6 and 2/5 affected on OpenAI	P 1.00 versus 0.80 and 0.92; R 0.95 within design; 2-hop recall 0	One-hop design predicts 2-hop recall 0; observed exactly
RQ4 end to end	Tests 0/4 to 4/4; 2 files, all predicted; needs-human false	-	Literature: 36% to 64% of LLM migrations pass the tests [27]; median test-pass rate 39.75% for an agent [26]; motivates the independent check
RQ5 real specs	Spec lagged shutdown by 198 days	478-commit study: type-array defect (95.9% of raw findings); sample precision 73% on structural findings (`allOf` gap)	Synthetic mutations used OAS 3.0 scalar types and flat schemas, so could not reveal either gap
RQ6 scalability	3.2 s per run on petstore; scan 799 + 4.83 ms per file	Real history: $t \approx -0.63 + 1.92 \cdot$MB s ($R^2 = 0.921$)	Linear model $R^2 = 0.995$; log-log slopes near 1 for $n \geq 750$

4.6 Justification for the Differences and Errors

1. **Perfect single-change accuracy versus imperfect compound labels.** The single-change operators were derived from the same definitions as the rules, so a 100% score shows that the implementation matches its specification (construct validity), not that the rule table is complete. Compound releases break the independence assumption behind the rename predicate ($|D| = |A| = 1$), which explains every label disagreement. Severity at the operation level is unaffected because every reading of an ambiguous combination is itself breaking.
2. **Naive diff errors.** The naive classifier's 639 misses and 500 false alarms are structural: it cannot know that an *added* required field breaks clients or that a version string does not. These are not tuning errors; any classifier without API semantics shares them.
3. **Call-site recall of 0.70.** The headline recall includes 109 two-hop callers that the design deliberately excludes and 10 computed URLs that the scanner flags instead of guessing. Within its stated design the scanner's recall is 0.95 (0.98 with flags). The remaining misses (6 calls through `this.http`) and the 31 attribution errors (`axios(config)`) are genuine gaps in the list of recognized forms; both are local fixes.
4. **Synthetic versus real results.** The mutation benchmark reported no false positives, while the history study (Section 4.3.5) found many spurious type-change findings. The difference is the input: the synthetic specifications use OAS 3.0 scalar types, while OpenAI's later specification uses OAS 3.1 type arrays, which the rule engine compares by reference identity instead of by value. This is why real data was included in the evaluation, and it is the most important error found. The same real data showed the 73% sample precision of structural findings, against 100% in the benchmark; all of the difference came from one schema restructured with `allOf`, a composition keyword that the synthetic specifications never used.
5. **Scaling slope below 1 at small sizes and slightly above 1 at large sizes.** Fixed overheads dominate small inputs; garbage-collection cost grows with heap size for large ones. Neither changes the conclusion of linear behavior within the tested range.
6. **Timing variability.** Some measurements on Machine B were taken while the long history study ran on the other core, and Machine A is a virtual machine. Medians of repeated runs (synthetic scaling) or the fastest of repeated runs (AST scan) were used to reduce this effect; absolute times should be read as indicative and the trends as robust.
7. **Specification lag.** The 198-day gap between OpenAI's shutdown and the specification change is an error of the input, not of the tool. Specification-driven detection can be no fresher than the published specification; providers whose specifications lag production, or who publish none, are outside what any specification diff can see.
8. **Single recorded repair.** The end-to-end repair was observed on one controlled upgrade

with three changes. It demonstrates that the loop works and that verification behaves as designed; it does not estimate the agent's success rate, which the literature shows can be far lower on harder migrations. The `needs-human` path exists precisely for those cases.

4.7 Threats to Validity

Internal validity. The mutation operators and the call-site corpus were written by the same developer who wrote the tool, which risks testing what the tool already does. This was mitigated by including forms the tool is known not to handle (unsupported client, computed URLs, two-hop callers), by reporting the naive baseline, and above all by the real-world history study, whose input no one involved in the project controlled. **External validity.** Real-world evidence comes from one provider (OpenAI) and one sample application; providers differ in specification style. **Construct validity.** Label accuracy and operation-level recall measure different things; both are reported, and the safety-critical one is identified. **Conclusion validity.** Trials are seeded and every script and result file is kept, so all numbers in this chapter can be regenerated.

4.8 Summary

Measured by three methods, sma meets its objectives: its deterministic classifier is exact on single changes and never missed a broken operation in compound releases, clearly outperforming a naive diff; its AST scanner reports no false call sites and finds callers that text search cannot; on a real upgrade the full loop restored a broken consumer to passing tests with a patch confined to the predicted files; and detection scales linearly, as the analytical model predicted. Real specification history exposed one implementation defect (OAS 3.1 type arrays compared by identity) and two semantic gaps (schema composition with `allOf`/`anyOf`/`oneOf`, and direction-blind type rules), and the call-site benchmark exposed a further defect (`axios(config)` methods); all are carried into the future scope.

CHAPTER 5

CONCLUSIONS

5.1 Conclusions

This project set out to close the loop between a third-party API's published specification and the consumer code that depends on it. The system built for this purpose, `sma`, watches OpenAPI specifications, classifies every change with 17 deterministic rules and a confidence-gated LLM fallback, locates affected call sites through AST analysis with one hop of callers, hands only breaking-and-used changes to a sandboxed coding agent, and verifies the result independently before opening a pull request for a human to review. The main conclusions are:

1. **The consumer-side gap is real and can be closed.** The literature shows that providers frequently remove operations without warning (24.6% of APIs versus 0.6% that deprecate first) and that existing diff tools serve only the provider. `sma` demonstrates a complete consumer-side pipeline from specification change to verified patch, the combination that the survey of fourteen existing approaches did not find.
2. **Deterministic classification is both accurate and cheap.** On 6,700 single-change mutation trials the rule engine was exact in every case, against an accuracy of 0.830 for a naive structural diff that misses every newly required field and raises false alarms on cosmetic edits. In releases with up to eight simultaneous changes, labels became less precise (recall 0.89) but no broken operation was missed. On the demo and the two replayed OpenAI changes the rules explained every difference, so the LLM was not needed; across OpenAI's full history, however, 331 of 472 version pairs left deltas that only the LLM fallback could judge.
3. **AST-based impact analysis is precise and finds what text search cannot.** On a labeled corpus the scanner reported no false call site (precision 1.00 versus 0.80 and 0.92 for grep baselines), found every direct call and every one-hop caller (text search found no callers at all), and flagged computed URLs for review instead of guessing. Joining changes to usage reduced the OpenAI case from five breaking removals to the two that actually mattered.
4. **Independent verification makes agent repair usable.** On the controlled upgrade, the consumer's tests went from 4/4 to 0/4 after the upstream release and back to 4/4 with the agent's patch, which touched only the two predicted files. Because the agent had no shell, this verdict came from the harness, not from the agent.
5. **Detection scales linearly**, as the analytical model predicted ($R^2 = 0.995$, about 0.18 ms

per operation), so the tool is cheap enough to run on every push or daily in CI.

6. **Real data finds what synthetic data cannot.** The 478-commit study of OpenAI's specification exposed a defect that no synthetic test revealed (OAS 3.1 type arrays compared by identity, 95.9% of raw findings) and two semantic gaps (schema composition keywords and direction-blind type rules, which brought the precision of a sample of real structural findings to 73%); the call-site benchmark exposed another defect (methods inside `axios(config)`). Each has a well-defined fix. The study also showed that a published specification can lag the real shutdown of an endpoint by more than six months, a limit of every specification-driven approach.

Overall, the objectives O1 to O7 of Section 1.3 were met, with the limitations stated above made explicit rather than hidden.

5.2 Future Scope

The following improvements follow directly from the evaluation, in order of priority.

1. **Compare types by value.** Replace the identity comparison of `type` in `PARAM_TYPE_CHANGED` and `RESPONSE_FIELD_TYPE_CHANGED` with a comparison of the serialized (or sorted) value, so that OAS 3.1 type arrays such as `["integer","null"]` stop producing spurious findings. This is a two-line change (one per rule) with a regression test drawn from the OpenAI history.
2. **Normalize schema composition and make type rules direction-aware.** Merge `allOf` members before building the field and response maps, compare `anyOf`/`oneOf` alternatives instead of treating them as a missing type, and judge a type change by direction: a response field that becomes nullable is breaking, a request parameter that starts accepting `null` is not. This removes the false removals and most ambiguous type findings seen in the OpenAI history.
3. **Read the method from the axios configuration object.** When the first argument is an object literal, take its `method` property, which fixes the only attribution errors observed in the call-site benchmark.
4. **Store the raw specification and resolve on load.** This removes the 19.5-fold inflation of the committed baseline on large specifications (61.4 MB for OpenAI, 111.5 MB for GitHub).
5. **Deeper and wider impact analysis.** Follow callers transitively up to a configurable depth, recognize more HTTP clients (`this.http`, `fetch` wrappers, `undici`) and trace constant URL variables, which would recover the two-hop and computed-URL cases measured in Chapter 4.
6. **Broader rule coverage.** Add rules for removed response status codes, changed con-

tent types, parameter location changes and tightened constraints (`maxLength`, `minimum`, `pattern`), moving toward the coverage of provider-side tools such as oasdiff, and handle two simultaneous renames on one operation with name and position evidence.

7. **Other languages.** Only the inventory module is language-specific; adding Python (with its AST module) or Java would extend the tool to the other major API-consuming ecosystems.
8. **A public benchmark.** Package the mutation operators, the labeled call-site corpus and replayed real specification histories as a reproducible benchmark of web API breaking changes paired with consumer code, which the literature survey found to be missing.
9. **Measure the agent.** Run the repair stage on a larger set of real upgrades to estimate its success rate and the rate at which the `needs-human` label is correctly raised, and calibrate the 0.7 confidence floor empirically.
10. **Fresher signals.** Combine specification diffs with provider deprecation pages, changelog feeds and HTTP `Deprecation`/`Sunset` headers observed at run time, to narrow the lag between a real shutdown and a specification change.

5.3 Applications and Utility

- **Teams that integrate fast-moving APIs.** Any JavaScript or TypeScript project calling OpenAI, Stripe, GitHub, Cloudflare, Twilio, DigitalOcean, Asana, Box or another provider that publishes an OpenAPI specification can add `sma` as a daily CI job and learn about breaking changes from a red build and a ready-made pull request, instead of from production errors.
- **Upgrade planning.** `sma diff <old> <new>` answers, with no configuration, whether moving from one specification version to another would break existing clients, which is useful before adopting a new API version.
- **Audit trail of upstream changes.** Because the baseline is committed, `git log .sma/specs/<api>.json` becomes a permanent history of every change the provider shipped, useful for incident analysis and vendor management.
- **Internal platform teams.** An organization whose services consume each other's APIs can use `sma` on the consumer side of every internal API that publishes a specification, complementing provider-side gates.
- **Education and research.** The rule table, the mutation benchmark and the replay of real specification history form a compact, reproducible testbed for teaching API evolution and for studying LLM-assisted maintenance with independent verification.

CONTRIBUTIONS

The innovative work and new ideas generated by this project, as supported by the analysis in Chapter 4, are:

1. **A consumer-side, closed-loop pipeline for third-party API changes.** To the best of the literature survey (Chapter 2), sma is the first tool that takes a third-party OpenAPI change from detection, through impact analysis on the consumer's own code, to an independently verified pull request. Existing tools stop at detection, at version bumps, or start from a task chosen by a human.
2. **A formally specified rule table with a structural rename rule.** Seventeen rules are defined as predicates over normalized operations (Table 3.4); renames are inferred from structure rather than name similarity, and all 300 random renames in the benchmark were found.
3. **Using the impact set twice: to brief the agent and to audit it.** The call-site analysis that tells the agent where to edit is reused to check which files it changed. With a shell-less agent and harness-run tests, this gives a verification predicate, $\neg T \vee \Phi \not\subseteq \hat{\Phi} \Rightarrow$ `needs-human`, that relies on nothing the agent reports about itself.
4. **Confidence-gated, residue-only use of an LLM.** The model sees only deltas no rule explains, answers in a fixed schema, and below a 0.7 confidence floor can never trigger a repair.
5. **An evaluation methodology for this setting**: 24 mutation operators (9,400 trials), a labeled call-site corpus with deliberate out-of-design cases, and a replay of all 478 commits to a public specification, which found a defect and two semantic gaps that synthetic tests could not.

Patent possibility. Item 3, a statically computed impact set used both to constrain a code-modifying agent and to verify its output before human review, may be examined for novelty with the guide. Because the code is already public under the MIT license, any filing would have to be assessed against that prior disclosure.

REFERENCES

[1] T. Espinha, A. Zaidman and H. Gross, "Web API growing pains: Loosely coupled yet strongly tied", *Journal of Systems and Software*, Vol. 100, 2015, pp. 27-43. DOI: https://doi.org/10.1016/j.jss.2014.10.014.

[2] J. Li, Y. Xiong, X. Liu and L. Zhang, "How Does Web Service API Evolution Affect Clients?", *Proceedings of the 2013 IEEE 20th International Conference on Web Services (ICWS)*, IEEE Computer Society, 2013, pp. 300-307. DOI: https://doi.org/10.1109/ICWS.2013.48.

[3] OpenAPI Initiative, "OpenAPI Specification v3.1.0", 2021. Available: https://spec.openapis.org/oas/v3.1.0 (accessed 21 October 2026).

[4] OpenAI, "openai-openapi: OpenAPI specification for the OpenAI API". Available: https://github.com/openai/openai-openapi (accessed 21 October 2026).

[5] Project team, "self-maintaining-api: Watch external API specs, find what they break in your repo, patch it, open a PR". Available: https://github.com/GlaDrancE/self-maintaining-api (accessed 21 October 2026).

[6] Anthropic, "Run Claude Code programmatically". Available: https://code.claude.com/docs/en/headless (accessed 21 October 2026).

[7] F. Di Lauro, S. Serbout and C. Pautasso, "To Deprecate or to Simply Drop Operations? An Empirical Study on the Evolution of a Large OpenAPI Collection", *Software Architecture: 16th European Conference, ECSA 2022*, Lecture Notes in Computer Science, Vol. 13444, Springer, 2022, pp. 38-46. DOI: https://doi.org/10.1007/978-3-031-16697-6_3.

[8] S. Serbout and C. Pautasso, "How Are Web APIs Versioned in Practice? A Large-Scale Empirical Study", *Journal of Web Engineering*, Vol. 23, No. 4, 2024, pp. 465-506. DOI: https://doi.org/10.13052/jwe1540-9589.2341.

[9] T. Preston-Werner, "Semantic Versioning 2.0.0". Available: https://semver.org/spec/v2.0.0.html (accessed 21 October 2026).

[10] D. Venturini, F. R. Cogo, I. Polato, M. A. Gerosa and I. S. Wiese, "I Depended on You and You Broke Me: An Empirical Study of Manifesting Breaking Changes in Client Packages", *ACM Transactions on Software Engineering and Methodology*, Vol. 32, No. 4, 2023, pp. 1-26. DOI: https://doi.org/10.1145/3576037.

[11] Postman, "2024 State of the API Report", *Technical Report*, Postman, Inc., 2024. Available: https://voyager.postman.com/doc/postman-state-of-the-api-report-2024.pdf (accessed 21 October 2026).

[12] OpenAI, “Deprecations”. Available: https://developers.openai.com/api/docs/deprecations (accessed 21 October 2026).

[13] oasdiff contributors, “oasdiff: OpenAPI Diff and Breaking Changes”. Available: https://github.com/oasdiff/oasdiff (accessed 21 October 2026).

[14] OpenAPITools, “openapi-diff: Utility for comparing two OpenAPI specifications”. Available: https://github.com/OpenAPITools/openapi-diff (accessed 21 October 2026).

[15] Optic Labs, “Optic: OpenAPI linting, diffing and testing”. Available: https://github.com/opticdev/optic (accessed 21 October 2026).

[16] Bump.sh, “API Change Management”. Available: https://docs.bump.sh/help/api-change-management/ (accessed 21 October 2026).

[17] R. He, H. He, Y. Zhang and M. Zhou, “Automating Dependency Updates in Practice: An Exploratory Study on GitHub Dependabot”, *IEEE Transactions on Software Engineering*, Vol. 49, No. 8, 2023, pp. 4004-4022. DOI: https://doi.org/10.1109/TSE.2023.3278129.

[18] Mend.io, “Renovate Documentation”. Available: https://docs.renovatebot.com/ (accessed 21 October 2026).

[19] Pact Foundation, “Pact Documentation: Introduction to Contract Testing”. Available: https://docs.pact.io/ (accessed 21 October 2026).

[20] L. Ochoa, T. Degueule, J. Falleri and J. Vinju, “Breaking bad? Semantic versioning and impact of breaking changes in Maven Central”, *Empirical Software Engineering*, Vol. 27, No. 3, 2022. DOI: https://doi.org/10.1007/s10664-021-10052-y.

[21] L. Xavier, A. Brito, A. Hora and M. T. Valente, “Historical and impact analysis of API breaking changes: A large-scale study”, *Proceedings of the 2017 IEEE 24th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2017, pp. 138-147. DOI: https://doi.org/10.1109/SANER.2017.7884616.

[22] R. G. Kula, D. M. German, A. Ouni, T. Ishio and K. Inoue, “Do developers update their library dependencies? An empirical study on the impact of security advisories on library migration”, *Empirical Software Engineering*, Vol. 23, No. 1, 2018, pp. 384-417. DOI: https://doi.org/10.1007/s10664-017-9521-5.

[23] C. E. Jimenez, J. Yang, A. Wettig *et al.*, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?”, *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. Available: https://arxiv.org/abs/2310.06770 (accessed 21 October 2026).

[24] J. Yang, C. E. Jimenez, A. Wettig *et al.*, “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering”, *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024, pp. 50528-50652. DOI: https://doi.org/10.52202/079017-1601.

[25] A. Almeida, L. Xavier and M. T. Valente, “Automatic Library Migration Using Large Language Models: First Results”, *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2024, pp. 427-433. DOI: https://doi.org/10.1145/3674805.3690746.

[26] A. Almeida, M. T. Valente and L. Xavier, “Using Copilot Agent Mode to Automate Library Migration: A Quantitative Assessment”, *Proceedings of the 2026 International Workshop on Agentic Engineering*, 2026, pp. 135-139. DOI: https://doi.org/10.1145/3786167.3788411.

[27] M. M. Islam, A. K. Jha, M. Mahmoud, I. Akhmetov and S. Nadi, “An Empirical Study of Python Library Migration Using Large Language Models”, *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025, pp. 867-879. Available: https://arxiv.org/abs/2504.13272 (accessed 21 October 2026).

[28] E. K. Smith, E. T. Barr, C. Le Goues and Y. Brun, “Is the Cure Worse Than the Disease? Overfitting in Automated Program Repair”, *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, 2015, pp. 532-543. DOI: https://doi.org/10.1145/2786805.2786825.

[29] N. Perry, M. Srivastava, D. Kumar and D. Boneh, “Do Users Write More Insecure Code with AI Assistants?”, *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023, pp. 2785-2799. DOI: https://doi.org/10.1145/3576915.3623157.

[30] M. Lamothe, Y. Guéhéneuc and W. Shang, “A Systematic Review of API Evolution Literature”, *ACM Computing Surveys*, Vol. 54, No. 8, 2021, pp. 1-36. DOI: https://doi.org/10.1145/3470133.

[31] X. Hou, Y. Zhao, Y. Liu *et al.*, “Large Language Models for Software Engineering: A Systematic Literature Review”, *ACM Transactions on Software Engineering and Methodology*, Vol. 33, No. 8, 2024, pp. 1-79. DOI: https://doi.org/10.1145/3695988.

[32] A. Fan, B. Gokkaya, M. Harman *et al.*, “Large Language Models for Software Engineering: Survey and Open Problems”, *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, 2023, pp. 31-53. DOI: https://doi.org/10.1109/ICSE-FoSE59343.2023.00008.

[33] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures”, Ph.D. Dissertation, University of California, Irvine, 2000. Available: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm (accessed 21 October 2026).

[34] R. T. Fielding, M. Nottingham and J. Reschke, “HTTP Semantics”, *Request for Comments 9110*, Internet Engineering Task Force (IETF), 2022. Available: https://www.rfc-editor.org/rfc/rfc9110 (accessed 21 October 2026).

[35] L. Richardson, M. Amundsen and S. Ruby, *RESTful Web APIs: Services for a Changing World*, O’Reilly Media, 2013, ISBN 978-1-4493-5806-8.

[36] A. Wright, H. Andrews, B. Hutton and G. Dennis, “JSON Schema: Draft 2020-12”. Available: https://json-schema.org/draft/2020-12 (accessed 21 October 2026).

[37] P. Bryan, K. Zyp and M. Nottingham, “JavaScript Object Notation (JSON) Pointer”, *Request for Comments 6901*, Internet Engineering Task Force (IETF), 2013. DOI: https://doi.org/10.17487/RFC6901.

[38] I. Sommerville, *Software Engineering*, 10th Edition, Pearson, 2016, ISBN 978-0-13-394303-0.

[39] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 9th Edition, McGraw-Hill Education, 2020, ISBN 978-1-259-87297-6.

[40] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd Edition, Addison-Wesley, 2018, ISBN 978-0-13-475759-9.

[41] S. Wang, I. Keivanloo and Y. Zou, “How Do Developers React to RESTful API Evolution?”, *Service-Oriented Computing: 12th International Conference, ICSOC 2014*, Lecture Notes in Computer Science, Vol. 8831, Springer, 2014, pp. 245-259. DOI: https://doi.org/10.1007/978-3-662-45391-9_17.

[42] S. M. Sohan, C. Anslow and F. Maurer, “A Case Study of Web API Evolution”, *2015 IEEE World Congress on Services (SERVICES)*, 2015, pp. 245-252. DOI: https://doi.org/10.1109/SERVICES.2015.43.

[43] J. Yasmin, Y. Tian and J. Yang, “A First Look at the Deprecation of RESTful APIs: An Empirical Study”, *Proceedings of the 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2020, pp. 151-161. DOI: https://doi.org/10.1109/ICSME46990.2020.00024.

[44] APIs.guru, “OpenAPI Directory”. Available: https://github.com/APIs-guru/openapi-directory (accessed 21 October 2026).

[45] F. Di Lauro, S. Serbout and C. Pautasso, “Towards Large-Scale Empirical Assessment of Web APIs Evolution”, *Web Engineering: 21st International Conference, ICWE 2021*, Lecture Notes in Computer Science, Springer, 2021, pp. 124-138. DOI: https://doi.org/10.1007/978-3-030-74296-6_10.

[46] N. Robles, N. Potes, K. Garcés, J. L. Cánovas Izquierdo and J. Cabot, “Exploratory Analysis of the Structural Evolution of public REST APIs”, *Anais do XXVI Congresso Ibero-Americano em Engenharia de Software (CIbSE 2023)*, Sociedade Brasileira de Computação, 2023, pp. 92-106. DOI: https://doi.org/10.5753/cibse.2023.24695.

[47] G. Vidal, F. Herrera and S. Casas, “Automatic identification of web API change types in consumer applications”, *Anais do XXV Congresso Ibero-Americano em Engenharia de Software (CIbSE 2022)*, Sociedade Brasileira de Computação, 2022, pp. 91-104. DOI: https://doi.org/10.5753/cibse.2022.20965.

[48] L. Bonorden and A. van Hoorn, “Detecting Usage of Deprecated Web APIs via Tracing”, *Proceedings of the 2024 IEEE 21st International Conference on Software Architecture (ICSA)*, 2024, pp. 23-33. DOI: https://doi.org/10.1109/ICSA59870.2024.00011.

[49] D. Dig and R. E. Johnson, “How do APIs evolve? A story of refactoring”, *Journal of Software Maintenance and Evolution: Research and Practice*, Vol. 18, No. 2, 2006, pp. 83-107. DOI: https://doi.org/10.1002/smr.328.

[50] J. Henkel and A. Diwan, “CatchUp!: capturing and replaying refactorings to support API evolution”, *Proceedings of the 27th International Conference on Software Engineering (ICSE)*, 2005, pp. 274-283. DOI: https://doi.org/10.1145/1062455.1062512.

[51] H. A. Nguyen, T. T. Nguyen, G. Wilson, A. T. Nguyen, M. Kim and T. N. Nguyen, “A graph-based approach to API usage adaptation”, *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, 2010, pp. 302-321. DOI: https://doi.org/10.1145/1869459.1869486.

[52] M. Dilhara, A. Bellur, T. Bryksin and D. Dig, “Unprecedented Code Change Automation: The Fusion of LLMs and Transformation by Example”, *Proceedings of the ACM on Software Engineering*, Vol. 1, No. FSE, 2024, pp. 631-653. DOI: https://doi.org/10.1145/3643755.

[53] GitHub, “About Dependabot version updates”. Available: https://docs.github.com/en/code-security/dependabot/dependabot-version-updates/about-dependabot-version-updates (accessed 21 October 2026).

[54] F. Reyes, Y. Gamage, G. Skoglund, B. Baudry and M. Monperrus, “BUMP: A Benchmark of Reproducible Breaking Dependency Updates”, *Proceedings of the 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2024, pp. 159-170. DOI: https://doi.org/10.1109/SANER60148.2024.00024.

[55] Stainless, “Stainless Documentation”. Available: https://www.stainless.com/docs/ (accessed 21 October 2026).

[56] S. A. Bohner and R. S. Arnold (Eds.), *Software Change Impact Analysis*, IEEE Computer Society Press, 1996, ISBN 978-0-8186-7384-9.

[57] X. Ren, F. Shah, F. Tip, B. G. Ryder and O. Chesley, “Chianti: a tool for change impact analysis of Java programs”, *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2004, pp. 432-448. DOI: https://doi.org/10.1145/1028976.1029012.

[58] B. Li, X. Sun, H. Leung and S. Zhang, “A survey of code-based change impact analysis techniques”, *Software Testing, Verification and Reliability*, Vol. 23, No. 8, 2013, pp. 613-646. DOI: https://doi.org/10.1002/stvr.1475.

[59] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd Edition, Addison-Wesley, 2006, ISBN 978-0-321-48681-3.

[60] Microsoft, “Using the Compiler API (TypeScript wiki)”. Available: https://github.com/microsoft/TypeScript/wiki/Using-the-Compiler-API (accessed 21 October 2026).

[61] ts-morph contributors, “ts-morph: TypeScript Compiler API wrapper”. Available: https://ts-morph.com/ (accessed 21 October 2026).

[62] A. Feldthaus, M. Schäfer, M. Sridharan, J. Dolby and F. Tip, “Efficient construction of approximate call graphs for JavaScript IDE services”, *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, 2013, pp. 752-761. DOI: https://doi.org/10.1109/ICSE.2013.6606621.

[63] C. Le Goues, T. Nguyen, S. Forrest and W. Weimer, “GenProg: A Generic Method for Automatic Software Repair”, *IEEE Transactions on Software Engineering*, Vol. 38, No. 1, 2012, pp. 54-72. DOI: https://doi.org/10.1109/TSE.2011.104.

[64] M. Monperrus, “Automatic Software Repair: A Bibliography”, *ACM Computing Surveys*, Vol. 51, No. 1, 2018, pp. 17:1-17:24. DOI: https://doi.org/10.1145/3105906.

[65] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto *et al.*, “Evaluating Large Language Models Trained on Code”, *arXiv preprint arXiv:2107.03374*, 2021. Available: https://arxiv.org/abs/2107.03374 (accessed 21 October 2026).

[66] C. S. Xia, Y. Wei and L. Zhang, “Automated Program Repair in the Era of Large Pre-trained Language Models”, *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2023, pp. 1482-1494. DOI: https://doi.org/10.1109/ICSE48619.2023.00129.

[67] Y. Zhang, H. Ruan, Z. Fan and A. Roychoudhury, “AutoCodeRover: Autonomous Program Improvement”, *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024, pp. 1592-1604. DOI: https://doi.org/10.1145/3650212.3680384.

[68] C. S. Xia, Y. Deng, S. Dunn and L. Zhang, “Agentless: Demystifying LLM-based Software Engineering Agents”, *arXiv preprint arXiv:2407.01489*, 2024. DOI: https://doi.org/10.48550/arXiv.2407.01489.

[69] F. Reyes, M. Mahmoud, F. Bono, S. Nadi, B. Baudry and M. Monperrus, “Byam: Fixing Breaking Dependency Updates with Large Language Models”, *Empirical Software Engineering*, Vol. 31, No. 4, 2026. DOI: https://doi.org/10.1007/s10664-026-10835-1.

[70] L. Liu, X. Liu, Q. Zhou *et al.*, “MigrationBench: Repository-Level Code Migration Benchmark from Java 8”, *arXiv preprint arXiv:2505.09569*, 2025. Available: https://arxiv.org/abs/2505.09569 (accessed 21 October 2026).

[71] S. Nikolov, D. Codecasa, A. Sjovall *et al.*, “How is Google using AI for internal code migrations?”, *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2025. Available: https://arxiv.org/abs/2501.06972 (accessed 21 October 2026).

[72] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt and R. Karri, “Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions”, *2022 IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 754-768. DOI: https://doi.org/10.1109/SP46214.2022.9833571.

[73] OpenJS Foundation, “Node.js Documentation: Test runner (node:test)”. Available: https://nodejs.org/api/test.html (accessed 21 October 2026).

[74] GitHub, “GitHub Actions Documentation”. Available: https://docs.github.com/en/actions (accessed 21 October 2026).

[75] GitHub, “GitHub CLI Manual: gh pr create”. Available: https://cli.github.com/manual/gh_pr_create (accessed 21 October 2026).

APPENDIX A

RULE REFERENCE

Table A.1 gives every rule of the classifier with a concrete example of the specification edit that triggers it and the effect on a consumer. The formal predicates are in Table 3.4.

Table A.1: Classification Rules with Examples

Rule	Severity	Example edit (v1 → v2)	Effect on existing client
PATH_REMOVED	breaking	`/engines` deleted from `paths`	404 on every call
METHOD_REMOVED	breaking	`DELETE /pets/{id}` removed, `GET` kept	404 or 405 on the removed method
REQUIRED_PARAM_ADDED	breaking	new required body field `ownerId`	400 until the value is sent
PARAM_REMOVED	breaking	query parameter `sort` deleted	Value ignored or rejected
PARAM_RENAMED	breaking	query `limit` → `pageSize` (same location and type)	Old name ignored; behavior silently changes
PARAM_TYPE_CHANGED	breaking	`id`: integer → string	Validation failures or wrong matching
PARAM_MADE_REQUIRED	breaking	`prompt` optional → required	Calls that omit it fail
RESPONSE_FIELD_REMOVED	breaking	`tag` removed from 2xx schema	Code reads `undefined`
RESPONSE_FIELD_ TYPE_CHANGED	breaking	`created`: integer → string	Arithmetic and comparisons misbehave
ENUM_VALUE_REMOVED	breaking	`status` no longer allows `pending`	Requests rejected; old values unhandled
AUTH_SCHEME_CHANGED	breaking	`apiKey` header → HTTP bearer (existing scheme modified or removed)	401 on every call
PATH_DEPRECATED	warning	`deprecated: true` added	Still works; signals removal
PATH_ADDED	safe	new `/pets/{id}/owner`	None
METHOD_ADDED	safe	new `PUT /pets`	None
OPTIONAL_PARAM_ADDED	safe	new optional query `expand`	None
RESPONSE_FIELD_ADDED	safe	new response field `owner`	None for tolerant readers
ENUM_VALUE_ADDED	safe	`status` also allows `reserved`	None for tolerant readers
UNCLASSIFIED	unknown	no rule, no LLM review	Reported; never repaired
LLM_REVIEWED	as judged	no rule; LLM verdict	Repaired only if breaking and confidence ≥ 0.7

APPENDIX B

CONFIGURATION AND COMMAND REFERENCE

B.1 Configuration file

Listing B.1: Annotated watch.yaml

```
# The repo that gets scanned and patched, relative to this file.
repo: ./demo/demo-app
# How sma runs your test suite to verify an agent's edit. Defaults to `npm test`.
test: npm test

apis:
  - name: petstore                          # letters, digits, dot, dash, underscore
    ; unique
    spec: http://localhost:8787/openapi.json   # URL or local path (JSON or YAML)
    baseUrl: http://localhost:8787          # stripped from URLs found in code
  - name: github
    spec: https://raw.githubusercontent.com/github/rest-api-description/main/
    descriptions/api.github.com/api.github.com.json
    baseUrl: https://api.github.com
    sdk: "@octokit/rest"                    # optional: also match SDK method chains
    headers:                                # optional: for private specs; ${VAR}
    read from the environment
      Authorization: "Bearer ${SPEC_TOKEN}"
```

B.2 Exit codes and environment

Table B.1 lists the exit codes and the environment variables the tool reads.

Table B.1: Exit Codes and Environment Variables

Item	Meaning
Exit code 0	Nothing broken
Exit code 1	A breaking change reached this repository (after `check`: any breaking change; after `impact`: a used one; after `fix`: always, until the PR is merged)
Exit code 2	`sma` itself failed: bad configuration, network error, missing prerequisite
`ANTHROPIC_API_KEY`	Enables LLM review of leftover deltas
`SMA_MODEL`	Overrides the model used for leftover review (default `claude-opus-5`)
`SMA_CONFIG`	Path of the configuration file (after `--config`, before the upward search)
`GH_TOKEN` / `GITHUB_TOKEN`	Lets `gh` open pull requests

B.3 Minimal scheduled workflow for any repository

Listing B.2: GitHub Actions Workflow to Run sma Daily

```
on:
  schedule: [{ cron: '0 6 * * *' }]
  workflow_dispatch:
jobs:
  watch:
    runs-on: ubuntu-latest
    permissions: { contents: write, pull-requests: write }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm ci
      - run: npx sma run --fix
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

APPENDIX C

COMPLETE LIST OF UNIT TESTS

Table C.1 lists all 51 tests, which passed on Machine A (Node.js 22.23.2), with the durations reported by the Node.js test runner.

Table C.1: Unit Tests and Durations

File	Test	ms
auth	adding a new auth scheme is not breaking	2.5
auth	adding the first auth scheme to a previously open API is not breaking	0.4
auth	removing an auth scheme is still breaking	1.6
auth	changing an existing auth scheme is still breaking	0.4
auth	an unchanged scheme produces nothing	0.6
diff	v1 $\rightarrow$ v2 produces exactly the expected rule hits	97.8
diff	every breaking hit is breaking, and the added endpoint is safe	44.9
diff	the rules explain the whole diff; nothing is left for the LLM	57.7
diff	rename is detected structurally, not by name similarity	34.0
diff	no false positives: a spec compared against itself reports nothing	46.2
diff	reordered parameters and enum values are not a change	16.1
diff	a removed endpoint is breaking	17.8
diff	an optional param becoming required is breaking; a new optional one is not	16.2
diff	dropping an enum value is breaking, adding one is not	12.8
diff	a changed auth scheme is breaking	10.5
impact	a change matches its own endpoint, including one-hop callers	2.2
impact	call sites belonging to a different api are never matched	0.3
impact	an unused change is reported as not-affected rather than dropped	0.3
impact	an auth change reaches every call site	0.3
impact	an unclassified whole-spec delta claims nothing	0.3
impact	only breaking-and-used changes are repairable	0.3

File	Test	ms
impact	one upstream edit across several operations becomes one repair	0.6
impact	different changes stay separate repairs	0.2
inventory	finds every request and resolves its endpoint	882.6
inventory	template literals collapse to spec-comparable paths	461.7
inventory	the method comes from the options object, not the default	540.5
inventory	requests to other hosts are ignored	529.4
inventory	a runtime-computed URL is reported as unresolved, never silently dropped	493.8
inventory	one-hop callers are found and attributed to the endpoint they reach	297.9
inventory	import statements are not mistaken for call sites	297.0
repair	a file the agent created is reported, not silently dropped	48.8
repair	modified and created files are both reported	26.9
repair	every modified tracked file keeps its name, not just the first	35.8
repair	a created file in a new directory keeps its full path	29.0
repair	a rename is attributed to the new name	32.8
repair	a clean repo reports nothing touched	26.5
report	spec- and code-derived strings are escaped	18.3
report	the verdict says whether a breaking change reaches this repo	1.6
report	a repair run covering every affected change says so instead of raising the alarm	0.5
store	findConfig walks up to the nearest watch.yaml	2.7
store	findConfig returns null instead of looping at the filesystem root	0.5
store	repo is resolved against the config file, not the cwd	13.4
store	an api name that could escape .sma/specs/ is rejected	3.7
store	duplicate api names are rejected; snapshots are keyed by name	2.8
watcher	a local spec path resolves against the given base, not the cwd	12.7
watcher	a nested local spec path resolves against the base	4.9
watcher	an absolute local spec path is left alone	2.2
watcher	a missing local spec names the resolved path, not the bare string	1.1
watcher	a credential in a spec URL is masked; the rest of the URL is not	0.5
watcher	header values read ${VAR} from the environment and refuse an unset one	1.6

File	Test	ms
watcher	a 503 is retried, headers are sent, and the spec comes back	1046.6

APPENDIX D

PROMPT GIVEN TO THE REPAIR AGENT

Listing D.1 is the exact prompt written to `.sma/prompts/` and sent to the agent for the first repair group of the recorded run.

Listing D.1: Prompt for the PARAM_RENAMED Repair (Recorded Run of 18 October 2026)

```
An external API this codebase depends on has changed. Update the affected call
    sites.

API:        petstore
Change:     PARAM_RENAMED (breaking)
Endpoints:  GET /pets
What happened: query parameter 'limit' was renamed to 'pageSize'
Details:    {"from":"limit","to":"pageSize","in":"query"}

Affected call sites found by static analysis:
  - src/pets.js:4   fetch(`${BASE}/pets?limit=${limit}`)
  - test/api.test.mjs:16   const pets = await listPets(2)

Rules for this edit:
1. Change ONLY what is needed to keep working against the new API. Touch only the
    files
   listed above (and, if a listed change forces it, their direct callers).
2. Do not refactor, rename, reformat, add comments, or "improve" anything else.
3. Keep the existing code style.
4. If a caller has to pass new information (e.g. a newly required parameter),
    thread it
   through the function signature rather than hardcoding a value. If you must
    invent a
   placeholder, name it obviously and say so in your summary.
5. The repo may already contain fixes for other changes from the same spec update.
   Leave those alone.
6. Do not run any commands or tests - the harness runs them.

When you are done, reply with a short plain-text summary: what you changed in each
    file,
and anything a human reviewer must double-check.
```

APPENDIX E

PULL REQUEST PRODUCED BY THE SYSTEM

Listing E.1 is the pull request body that sma wrote to `.sma/pr/sma_petstore-20261018093325.md` during the recorded run (lightly wrapped to fit the page).

Listing E.1: Pull Request Body of the Recorded Repair Run

```
## `petstore` spec changed - 3 breaking change(s) patched

Spec: http://localhost:8787/openapi.json

### 1. PARAM_RENAMED - GET /pets
query parameter 'limit' was renamed to 'pageSize'
_Detected by the rule engine; details: `{"from":"limit","to":"pageSize","in":"query
    "}`_
Call sites:
- `src/pets.js:4` - `fetch(`${BASE}/pets?limit=${limit}`)`
- `test/api.test.mjs:16` - `const pets = await listPets(2)`
What the agent reported:
> I changed one file and left the other alone. I didn't run anything, as the rules
    asked.
> - src/pets.js:4: the request now sends `?pageSize=${limit}` instead of `?limit=${
    limit}`.
>   The function is still `listPets(limit = 10)`, so its callers don't change.
> - test/api.test.mjs:16: no change. `listPets(2)` still passes a number, and the
    function
>   now sends it as `pageSize`.
> For the reviewer: 1. The JavaScript argument is still called `limit`; renaming it
     is a
> separate cleanup. 2. Check that the petstore server used in the test runs the new
     API.

### 2. RESPONSE_FIELD_REMOVED - GET /pets, POST /pets, GET /pets/{id}
response field 'tag' was removed
_Detected by the rule engine; details: `{"field":"tag","code":"200"}`_
Call sites:
- `src/pets.js:4`, `test/api.test.mjs:16`, `src/pets.js:14`, `test/api.test.mjs:32`,

  `src/pets.js:9`, `src/report.js:7`, `test/api.test.mjs:22`, `test/api.test.mjs
    :28`
What the agent reported:
> I changed two files. src/pets.js: `describePet` now returns just `${pet.name}`.
> `createPet` still sends `tag` in the POST body, because the v2 spec still accepts
     it.
> test/api.test.mjs: expected strings updated to 'Rex' and '#2 Mia [pending]'.
> src/report.js: no change needed; it picks up the fix through `describePet`.
> For the reviewer: 1. Visible text has changed ("Rex" instead of "Rex (dog)"); if
    product
> still wants the species shown, that needs a decision. 2. One test name is now
    inaccurate.
```

```
> 3. I left the existing `pageSize` fix alone. 4. `ownerId` is outside this change.

### 3. REQUIRED_PARAM_ADDED - POST /pets
new required body parameter 'ownerId'
_Detected by the rule engine; details: `{"name":"ownerId","in":"body","type":"
    integer"}`_
Call sites:
- `src/pets.js:14` - `fetch(`${BASE}/pets`, {`
- `test/api.test.mjs:32` - `const pet = await createPet('Nix', 'ferret')`
What the agent reported:
> createPet(name, tag) now takes a third argument and passes ownerId into the POST
    body;
> nothing is hardcoded. The test calls createPet('Nix', 'ferret', 10); 10 is a real
     owner
> in the petstore server's starting data. For the reviewer: a caller that leaves
    ownerId
> out sends undefined and gets a 400 "ownerId is required".

### Diff
 src/pets.js       | 8 ++++----
 test/api.test.mjs | 6 ++++--
 2 files changed, 7 insertions(+), 7 deletions(-)

### Verification (run by the harness, not the agent)
- **PASS** - test suite
- every changed file was predicted

### Reviewer checklist
- [ ] The new contract matches the spec linked above
- [ ] No unrelated files were touched
- [ ] Any invented placeholder values are called out and replaced
- [ ] Tests assert the new behaviour, not just that nothing throws
```

APPENDIX F

SELECTED SOURCE CODE

The complete source code is available in the project repository [5]. The listings below show the parts referred to in Chapter 3.

Listing F.1: Reference Inlining with Cycle Guard (src/watcher.mjs)

```
export function resolveRefs(spec) {
  const at = (pointer) =>
    pointer.slice(2).split('/')
      .map((p) => p.replace(/~1/g, '/').replace(/~0/g, '~'))
      .reduce((node, key) => node?.[key], spec);

  const walk = (node, seen) => {
    if (Array.isArray(node)) return node.map((n) => walk(n, seen));
    if (!node || typeof node !== 'object') return node;
    const ref = node.$ref;
    if (typeof ref === 'string' && ref.startsWith('#/')) {
      if (seen.has(ref)) return { $ref: ref, $circular: true };
      const target = at(ref);
      if (target === undefined) return node;
      return walk(target, new Set([...seen, ref]));
    }
    return Object.fromEntries(Object.entries(node).map(([k, v]) => [k, walk(v, seen
    )]));
  };
  return walk(spec, new Set());
}
```

Listing F.2: Classification: Rules First, LLM for Leftovers, Confidence Floor (src/classify.mjs)

```
export async function classify(before, after, { apiName, useLlm = true } = {}) {
  const hits = applyRules(before, after);
  const leftovers = unexplainedDeltas(before, after, hits);
  if (!leftovers.length) return { changes: hits, reviewed: 0, skipped: 0 };

  if (!useLlm || !process.env.ANTHROPIC_API_KEY) {
    for (const d of leftovers) hits.push({ rule: 'UNCLASSIFIED', severity: 'unknown
    ',
      path: null, method: null, detail: { pointer: d.pointer, op: d.op },
      reason: `no rule covers ${d.pointer} and no LLM review was run`,
      pointers: [d.pointer], source: 'none' });
    return { changes: hits, reviewed: 0, skipped: leftovers.length };
  }
  const verdicts = await askClaude(apiName, leftovers);
  for (const v of verdicts) {
    const confident = v.confidence >= CONFIDENCE_FLOOR;          // 0.7
    hits.push({ rule: 'LLM_REVIEWED', severity: confident ? v.severity : 'unknown',
```

```
      path: null, method: null,
      detail: { pointer: v.pointer, confidence: v.confidence, claimed: v.severity },
      reason: confident ? v.reason : `${v.reason} (low confidence ${v.confidence}, not auto-patched)`,
      pointers: [v.pointer], source: 'llm' });
  }
  return { changes: hits, reviewed: verdicts.length, skipped: 0 };
}
```

Listing F.3: Impact Matching and Repair Grouping (src/impact.mjs)

```
const WHOLE_SPEC_RULES = new Set(['AUTH_SCHEME_CHANGED']);

function matches(change, site, opId) {
  if (site.api !== change.api) return false;
  if (site.kind === 'http' || site.kind === 'caller') {
    if (!change.path) return WHOLE_SPEC_RULES.has(change.rule);
    if (shape(site.path) !== shape(change.path)) return false;
    return !change.method || site.method === change.method;
  }
  if (site.kind === 'sdk') {
    if (!change.path) return WHOLE_SPEC_RULES.has(change.rule);
    if (!opId) return false;
    return site.symbol.toLowerCase().includes(opId.toLowerCase());
  }
  return false;
}

export function repairable(impacts) {
  const groups = new Map();
  for (const i of impacts.filter((x) => x.severity === 'breaking' && x.affected)) {
    const subject = i.detail.field ?? i.detail.name ?? i.detail.from ?? i.detail.scheme ?? i.path ?? '';
    const key = `${i.api}:${i.rule}:${subject}`;
    const endpoint = [i.method, i.path].filter(Boolean).join(' ') || '(whole spec)';
    const existing = groups.get(key);
    if (!existing) { groups.set(key, { ...i, endpoints: [endpoint] }); continue; }
    existing.endpoints.push(endpoint);
    for (const site of i.sites)
      if (!existing.sites.some((s) => s.file === site.file && s.line === site.line)) existing.sites.push(site);
  }
  return [...groups.values()];
}
```

Listing F.4: Independent Verification After the Agent Runs (src/repair.mjs)

```
export function finish(cfg, impacts, base, branch) {
  const changedFiles = touchedFiles(cfg.repoPath);          // git status --
    porcelain -uall
  if (!changedFiles.length) {
    git(cfg.repoPath, ['checkout', base]);
    git(cfg.repoPath, ['branch', '-D', branch], { allowFail: true });
    return { applied: false, base, branch, reason: 'the agent made no edits' };
  }
  const tests = runTests(cfg);                                 // cfg.test ?? 'npm
    test'
  const expected = new Set(impacts.flatMap((i) => i.sites.map((s) => s.file)));
  const outsideScope = changedFiles.filter((f) => !expected.has(f));
  return {
    applied: true, base, branch, changedFiles, outsideScope, tests,
    diffstat: (git(cfg.repoPath, ['add', '-A']), git(cfg.repoPath, ['diff', '--
    cached', '--stat'])),
    needsHuman: !tests.pass || outsideScope.length > 0,
  };
}
```

APPENDIX G

EXPERIMENT ARTIFACTS

All experiments of Chapter 4 are reproducible from the scripts and result files listed in Table G.1. Random choices use a fixed linear congruential generator (seed 42 for mutations, 7 for the call-site corpus).

Table G.1: Experiment Scripts and Result Files

Script	Purpose	Result file
`mutate.mjs`	24 mutation operators; 6,700 single-change trials; naive baseline	`mutation_results.json`, `mutation_summary.json`
`compound.mjs`	2,700 compound-release trials, $k \in \{1,2,3,4,6,8\}$	`compound_results.json`, `compound_summary.json`
`csbench.mjs`	Labeled corpus generator; AST scanner versus two grep baselines; scan timing by size	`csbench_40.json`, `scan_scale.txt`
`history.mjs`	Classifies every consecutive pair of `openapi.yaml` in OpenAI's repository	`openai_history.jsonl`
`scale.mjs`	Synthetic specifications, 30 to 12,000 operations; per-stage timing	`scale_results.json`, `scale_fits.json`
`g_exp.py`, `g_hist.py`	Produce every graph in Chapter 4	PDF graphs

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, **[PROF./DR. GUIDE NAME]**, Department of Computer Science and Engineering, [NAME OF THE INSTITUTE], for the constant guidance, encouragement and valuable suggestions given throughout this project. The direction received at every stage, from defining the problem to evaluating the results, shaped this work.

We are thankful to **[PROF./DR. HOD NAME]**, Head of the Department of Computer Science and Engineering, for providing the facilities and the support needed to carry out the project, and to **[DR. PRINCIPAL NAME]**, Principal, [NAME OF THE INSTITUTE], for providing an environment that encourages project work and research.

We thank all the faculty members and the non-teaching staff of the department for their help, and the maintainers of the open-source projects on which this work builds, in particular Node.js, ts-morph and the TypeScript compiler, and the API providers who publish their OpenAPI specifications in the open.

Finally, we thank our parents, family and friends for their patience, support and encouragement throughout the course of this work.

Place: [CITY] **[STUDENT NAME 1]**

[STUDENT NAME 2]

[STUDENT NAME 3]

Date: